



Proyecto Final: Asistente Virtual (Modelo Inteligente Asistente Universal [MIAU]).

Universidad Nacional Autónoma de México.
Facultad de Ingeniería.

Inteligencia Artificial.

Nombre del equipo: *Los puffs*

Grupo: 5

Equipo: 3

Alumno (s):

320025561

320266083

320317264

320060829

320206968

Fecha de entrega:

27 de noviembre de 2025



Índice

1. Parte I. Ensayo teórico	3
1.1. Introduccion	3
1.2. Desarrollo:	3
1.2.1. Modelos probabilísticos:	3
1.2.2. Modelos con base en reglas:	7
1.2.3. Modelos bioinspirados:	11
1.2.4. Modelos para toma de decisiones:	18
1.3. Conclusión:	21
1.4. Bibliografía:	21
2. Parte II. Proyecto práctico: diseño y desarrollo del agente inteligente:	22
2.1. Descripción del agente y su propósito:	22
2.2. Justificación teórica con base en el modelo de IA aplicado:	22
2.3. Problemas encontrados y soluciones propuestas:	23
2.4. Evidencias: capturas de pantalla o GIF de funcionamiento:	24
2.5. Código fuente documentado:	35
2.5.1. Gestión de Dependencias y Carga Condicional	35
2.5.2. Definición de Rutas y Constantes Heurísticas	37
2.5.3. Constantes de Anime, Identidad y Funciones Gráficas Auxiliares	40
2.5.4. Implementación del Panel Redondeado (RoundedPanel)	44
2.5.5. Configuración Global y Constantes de Entorno	46
2.5.6. Configuración de Visión, Voz y Planificador de Tareas	48
2.5.7. Bases de Conocimiento Estáticas e Inicialización de Clientes	49
2.5.8. Carga de Guías de Videojuegos y Normalización	52
2.5.9. Normalización Masiva de Constantes de Búsqueda	53
2.5.10. Carga e Indexación de Bases de Datos JSON	55
2.5.11. Utilidades de Procesamiento de Voz y Fechas	59
2.5.12. Gestor de Memoria (MemoryManager)	60
2.5.13. Utilidades del Sistema y Gestión de Perfiles	62
2.5.14. Inicialización del Motor de Automatización (Selenium)	63
2.5.15. Navegación e Interacción con el DOM	64
2.5.16. Lógica de Envío de Mensajes	65
2.5.17. Sistema de Planificación de Tareas (Scheduler)	68
2.5.18. Motor de Auto-Respuesta (Auto-Reply)	70
2.5.19. Extracción Heurística de Parámetros	72
2.5.20. Extracción Basada en LLM y Coordinación	74
2.5.21. Extracción de Datos del DOM (Scraping) y Generación de Respuesta	75
2.5.22. Detección de Expresiones Temporales en Español	78
2.5.23. Inferencia de Etiquetas y Generación de Enlaces de Calendario	81
2.5.24. Integración con Modelos Locales (Ollama)	82
2.5.25. Subsistema de Visión Artificial (OpenCV)	83
2.5.26. Pipeline de Descripción de Imágenes (Image Captioning)	84
2.5.27. Generación Heurística de Descripciones de Escena	87
2.5.28. Interacción con Hardware de Cámara y Archivos	89
2.5.29. Orquestador de Consultas LLM (Remote & Local)	90
2.5.30. Utilidades de Limpieza y Gestión de Historial	93



2.5.31. Consultas a APIs de Entretenimiento (Música y Anime)	94
2.5.32. Estrategias de Búsqueda Web (Multi-Provider)	96
2.5.33. Investigación Aumentada por Recuperación (RAG)	97
2.5.34. Motor de Búsqueda de Video (Piped/YouTube)	99
2.5.35. Utilidades de Navegación (Maps y Anime)	102
2.5.36. Contexto de Videojuegos y Expansión de Consultas	103
2.5.37. Detección de Guías Curadas (Conocimiento Estático)	105
2.5.38. Motor de Búsqueda de Recetas (Algoritmo Ponderado)	106
2.5.39. Extracción de Señales y Filtros	106
2.5.40. Algoritmo de Búsqueda y Ranking (Lookup)	107
2.5.41. Formateo de Respuesta (Recetas)	108
2.5.42. Motor de Recomendación de Hardware	110
2.5.43. Formateo Dinámico de Detalles de Hardware	113
2.5.44. Gestión de Estado Emocional y Cerebro del Asistente	117
2.5.45. Clasificador de Intenciones (Router Semántico)	120
2.5.46. Interfaz de Hardware de Voz	121
2.5.47. Núcleo del Asistente (AssistantCore)	122
2.5.48. Interfaz Gráfica (Tkinter) y Punto de Entrada	123



1. Parte I. Ensayo teórico

1.1. Introduccion

Este ensayo tiene como propósito principal explorar toda la teoría detras del funcionamiento de la Inteligencia Artificial actual, centrándose específicamente en los mecanismos mediante los cuales los agentes inteligentes lidian con la incertidumbre y procesan información en entornos complejos. En el curso de Inteligencia Artificial de la Facultad de Ingeniería, es necesario comprender no solo la ejecución práctica de algoritmos, sino la lógica matemática y los conceptos que permite a una máquina razonar, aprender y decidir. A lo largo de este trabajo, se analiza la relevancia de pasar de sistemas deterministas a modelos probabilísticos, como las redes Bayesianas y los modelos de Markov, que permiten deducir estados ocultos a partir de evidencia que podemos ver. Asimismo, se examinan enfoques bioinspirados, tales como las redes neuronales y los algoritmos genéticos, y se discuten los fundamentos de la teoría de la decisión. Esto establece la base necesaria para entender cómo se construyen sistemas capaces de actuar racionalmente ante información incompleta, sirviendo como base para el desarrollo del agente de inteligencia artificial creado llamado "MIAU".

1.2. Desarrollo:

1.2.1. Modelos probabilísticos:

Los modelos probabilísticos son herramientas fundamentales en el campo de la inteligencia artificial para el razonamiento bajo incertidumbre, en el cual los agentes no conocen la información verdadera suficiente sobre su entorno y entre los más destacados para resolver problemas con información incierta se encuentran las redes Bayesianas y los modelos de Markov, modelos que permiten representar y manipular conocimiento incierto de manera eficiente. Los modelos de redes Bayesianas y Markov han revolucionado áreas como la diagnosis médica, la robótica, el procesamiento del lenguaje natural y la toma de decisiones autónomas.

■ Redes Bayesianas:

Para que una máquina pueda razonar, aprender y tomar decisiones en entornos complejos y no estructurados, para esto tenemos dos arquitectur las Redes Bayesianas y los Modelos de Markov.

Para esto, lo primero, que tenemos es en el núcleo de estos modelos se encuentran las variables aleatorias, que representan atributos del mundo cuyo valor exacto puede ser desconocido o incierto. Un sistema de IA debe modelar la distribución de probabilidad conjunta de todas estas variables para tener una imagen completa del dominio.

Explicar el concepto y definición:

Una red Bayesiana es una estructura probabilística que modela dependencias entre variables aleatorias mediante un grafo dirigido acíclico y en este grafo, no obstante, tambien tenemos a los arcos (flechas) conectan pares de nodos y denotan una relación de dependencia probabilística directa. Si existe un arco del nodo X al nodo Y ($X \rightarrow Y$), se dice que X es el "padre" de Y , cada nodo representa una variable y las aristas reflejan influencias probabilísticas directas. Además, estos modelos se

complementan con tablas de probabilidad condicional que cuantifican dichas relaciones, permitiendo expresar cómo el estado de una variable afecta a otras dentro del sistema y gracias a la explotación de independencias condicionales, las redes Bayesianas pueden representar distribuciones conjuntas de múltiples variables de manera compacta y eficiente, al mismo tiempo que las redes bayesianas reducen significativamente la complejidad del razonamiento probabilístico en espacios de alta dimensionalidad.

En este sentido, Russell y Norvig (2004) precisan que:

A Bayesian network is a directed graph in which each node is annotated with quantitative probability information. (...) Each node corresponds to a random variable, which may be discrete or continuous. (...) Each node X_i has a conditional probability distribution $P(X_i|Parents(X_i))$ that quantifies the effect of the parents on the node. (p. 511).

Este teorema proporciona la regla matemática para revisar la probabilidad de una hipótesis (H) a la luz de una nueva evidencia (E). La formulación clásica se expresa como:

$$P(H|E) = \frac{P(E|H) \cdot P(H)}{P(E)}$$

Donde: $P(H|E)$ es la probabilidad a posteriori: la probabilidad de la hipótesis después de observar la evidencia.

$P(E|H)$ es la verosimilitud (likelihood): la probabilidad de observar la evidencia si la hipótesis fuera cierta.

$P(H)$ es la probabilidad a priori: la creencia inicial antes de ver la evidencia.

$P(E)$ es la probabilidad marginal de la evidencia: actúa como un factor de normalización.

Ahora bien, para el aprendizaje estructural, donde los métodos que tenemos son:

- Métodos basados en restricciones: Utilizan pruebas estadísticas de independencia condicional (como Chi-cuadrado o Información Mutua) para decidir si debe existir un arco entre dos nodos.
- Métodos basados en puntuación (Score-based): Asignan una puntuación a cada estructura candidata según qué tan bien explica los datos. Se utilizan métricas como la verosimilitud penalizada, el Criterio de Información de Akaike (AIC) o el Principio de Longitud de Descripción Mínima (MDL). El principio MDL busca el modelo que comprima mejor los datos, equilibrando la precisión del ajuste con la simplicidad del grafo.
- Algoritmo Chow-Liu: Un método clásico y eficiente para aprender estructuras de árbol (donde cada nodo tiene máximo un padre). Este algoritmo aproxima la distribución de probabilidad minimizando la divergencia de Kullback-Leibler entre el modelo y los datos reales.

“The simplest case is the Chow and Liu algorithm, which measures mutual information between pairs of variables. From these measurements, as seen previously, a Bayesian network in the form of a tree is generated. By analyzing dependencies between triplets of variables, the method extends to polytrees. This approach can be generalized for learning multi-connected networks by testing dependencies between subsets of variables, typically two or three variables.”. (Pag. 22. Sucar,L.)

Describir al menos un ejemplo práctico o aplicado: Un ejemplo clásico es el sistema de alarma contra robos que mencionan Russell y Norvig (2004) donde nos dicen: “You have a new burglar alarm installed at home. It is fairly reliable at detecting a burglary, but also responds on occasion to minor earthquakes. You also have two neighbors, John and Mary, who have promised to call you at work when they hear the alarm”(p. 511). Este ejemplo ilustra cómo múltiples variables (robo, terremoto, alarma, llamadas de vecinos) interactúan en una red Bayesiana, permitiendo calcular probabilidades condicionales como $P(\text{Burglary} \mid \text{JohnCalls}, \text{MaryCalls})$.

Mencionar ventajas, limitaciones y contextos de aplicación: Las redes Bayesianas tienen ventajas importantes entre las que se encuentra la capacidad de manejar variables discretas y continuas, combinar conocimiento experto con datos empíricos y representar distribuciones conjuntas complejas de manera compacta así que se podría decir que es un razonamiento eficiente bajo incertidumbre. Como desventajas se tiene que su construcción exige un conocimiento profundo del dominio, y el cálculo exacto de probabilidades puede ser computacionalmente costoso en redes extensas y es por ello que Russell y Norvig (2004) advierten que “in the general case, the problem is intractable” (p. 552), lo que ha impulsado el desarrollo de métodos de inferencia aproximada para hacer viable su aplicación en sistemas de gran escala.

■ Modelos de Markov:

Markov, encontraba intolerable esta combinación de matemáticas y teología en 1713, donde el Necrasov argumentaba esto, para esto Markov, se propuso dismantelar esta teoria que la Ley de los Grandes Números podría seguir siendo válida incluso cuando los eventos no fueran independientes, es decir, incluso cuando el resultado de un evento dependiera fuertemente del que lo precedió.

Para probar esto, requería un conjunto de datos que fuera inherentemente no independiente, un sistema donde el estado de un elemento dictara las probabilidades del siguiente, violando así la premisa fundamental de Necrasov, Markov despojó a las primeras 20.000 letras del texto de todo su significado semántico y narrativo, categorizándolas estrictamente como vocales o consonantes, calculó la probabilidad de que una vocal siguiera a una vocal, una consonante a una vocal, y así sucesivamente. A pesar de estas fuertes dependencias donde el “pasado”(la letra anterior) influía en el “futuro”(la siguiente letra), Markov demostró que la distribución general de vocales (aproximadamente 43 %) y consonantes (aproximadamente 57 %) se estabilizaba sobre el gran conjunto de datos. Esta evidencia empírica destrozó el argumento de Necrasov: el orden estadístico no requería independencia divina, y por lo tanto, la estabilidad de las estadísticas sociales no podía usarse para probar el concepto teológico del libre albedrío independiente.

Ahora bien, esto es lo importante de Markov en su historia, pero mientras que las Redes Bayesianas destacan en modelar relaciones complejas entre variables en un momento estático, los Modelos de Markov están diseñados específicamente para manejar datos secuenciales

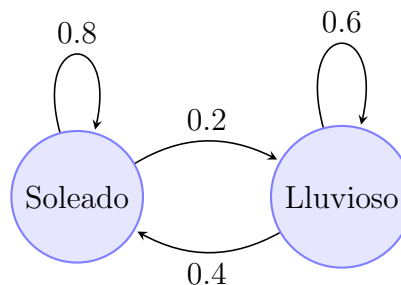


Diagrama de clima, ejemplo escueto.

Explicar el concepto y definición: Los modelos de Markov incluyendo a los modelos ocultos de Markov (HMM), se emplean para representar secuencias de eventos en las que el estado actual depende únicamente del estado anterior inmediato y a esto se le conoce como propiedad de Markov. En estos modelos los estados reales del proceso no son directamente observables ya que son inferidos a través de la evidencia recogida. Russell y Norvig (2004) afirman “an HMM is a temporal probabilistic model in which the state of the process is described by a single discrete random variable. The possible values of the variable are the possible states of the world.” Esta representación puede modelarse como una red Bayesiana dinámica con una sola variable de estado discreta, donde “each data point consists of an observation sequence of finite length, so the problem is to learn the transition probabilities from a set of observation sequences”. Además, los autores enfatizan que “representations can be designed to satisfy the Markov property, so that the future is independent of the past given the present”, y que bajo la suposición de estacionariedad las dinámicas no cambian en el tiempo y se simplifica considerablemente la representación del sistema. Estas características hacen de los modelos de Markov una herramienta poderosa para aplicaciones secuenciales y en especial las basadas en datos observacionales.

Describir al menos un ejemplo práctico o aplicado: Un ejemplo práctico muy popular en el que se usan Modelos de Markov es en el campo del reconocimiento del habla pues los HMM modelan la transición entre fonemas. Russell y Norvig (2004) destacan: “The basic task for any probabilistic inference system is to compute the posterior probability distribution for a set of query variables, given some observed event”. Todo lo anterior significa que los HMM pueden inferir secuencias de estados ocultos (fonemas) a partir de observaciones (señales de audio).

Mencionar ventajas, limitaciones y contextos de aplicación: Los modelos de Markov poseen ciertas ventajas como que son ideales para aplicaciones temporales y secuenciales como el monitoreo de sistemas o la predicción de series de tiempo. Como desventaja se tiene que debido a su naturaleza de su suposición de independencia respecto al pasado lejano resulta limitada en contextos donde la historia completa del sistema es relevante y para abordar estas dificultades de inferencia

en modelos grandes, Russell y Norvig (2004) describen métodos de inferencia aproximada como los algoritmos de Markov Chain Monte Carlo (MCMC), señalando que “MCMC algorithms work quite differently from rejection sampling [...] Instead of generating each sample from scratch, MCMC algorithms generate each sample by making a random change to the preceding sample” (p. 536), todos estos enfoques nos permiten manejar de manera más eficiente la complejidad computacional inherente al razonamiento probabilístico en sistemas con alta dimensionalidad.

1.2.2. Modelos con base en reglas:

- Árboles de decisión/regresión:

Explicar concepto y definición

El aprendizaje de árboles de decisión es uno de los métodos más eficaces y utilizados en la inteligencia artificial para la aproximación de funciones. Un árbol de decisión es una representación formal de una función que toma un conjunto de atributos como entrada y devuelve una decisión o valor de salida.

Estructura Jerárquica

La estructura de un árbol de decisión opera de manera jerárquica y descendente y este modelo trabaja mediante una estrategia de divide y vencerás. Podemos visualizar el árbol compuesto por tres elementos fundamentales que mapean la toma de decisiones humana:

1. **Nodos internos:** Cada nodo representa una evaluación o test sobre un atributo específico del objeto como, por ejemplo, preguntar ¿Está lloviendo? o “¿El precio es alto?”.
2. **Ramas:** Cada rama que sale de un nodo corresponde a uno de los posibles valores que puede tomar dicho atributo como, por ejemplo, el “Sí” o “No” u otro ejemplo como “Rojo” o “Azul”.
3. **Nodos hoja:** Son los nodos terminales que contienen el valor de retorno o la decisión final, por ejemplo, “Esperar” o “No esperar”.

El proceso de tomar una decisión consiste en comenzar en la raíz del árbol y descender a través de las ramas que coincidan con los valores de los atributos del caso que estamos analizando, hasta llegar a una hoja.

Tal y como lo definen Russell y Norvig en la sección 18.3:

“A decision tree represents a function that takes as input a vector of attribute values and returns a “decision”-a single output value.” Sección 18.3.1, pág. 698 (3^o Edición).



Tipos de Árboles: Clasificación vs. Regresión

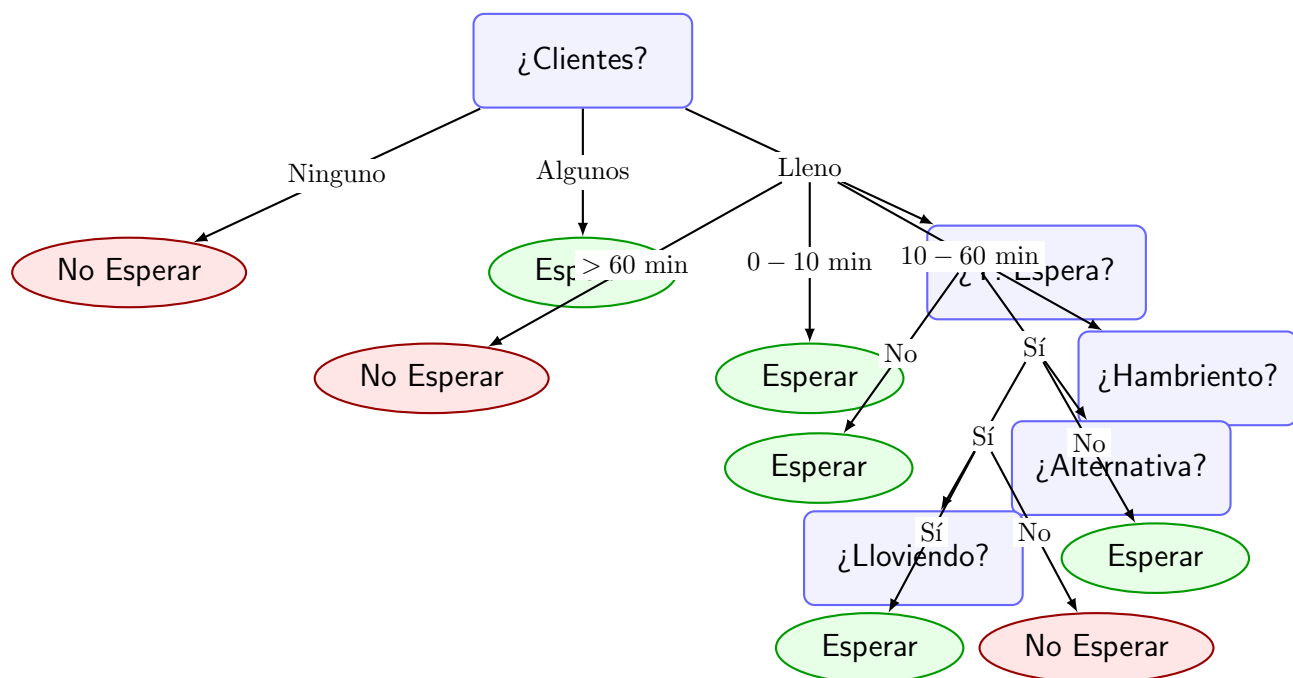
Aunque comúnmente se agrupan bajo el mismo concepto, existe distinción importante basada en el tipo de salida:

- Árboles de Clasificación: Se utilizan cuando la variable de salida es discreta o categórica como “Sí/No”, “Enfermo/Sano”, y el objetivo es clasificar la entrada en una de varias clases finitas.
- Árboles de Regresión: Se utilizan cuando la variable de salida es continua como predecir el precio de una casa o la temperatura de mañana, en este caso en lugar de una clase, las hojas del árbol contienen un valor numérico que comúnmente son el promedio de los ejemplos de entrenamiento que caen en esa hoja o incluso una función lineal simple.

Ejemplo práctico: Problema del restaurante

Para ver cómo se utilizan estos árboles, hay un escenario cotidiano y complejo, que es la decisión de esperar o no por una mesa en un restaurante cuando no se tiene reserva y este problema es ideal porque involucra múltiples factores que influyen en la decisión humana, donde el objetivo es que el agente aprenda la definición del predicado meta Esperar (verdadero o falso) basándose y analizando los siguientes atributos de la situación:

1. Alternativa: ¿Hay otro restaurante adecuado cerca?
2. Bar: ¿El restaurante tiene una zona de bar cómoda para esperar?
3. Vier/Sab: ¿Es viernes o sábado?
4. Hambriento: ¿Tenemos mucha hambre?
5. Clientes: ¿Cuánta gente hay en el restaurante? (Valores: Ninguno, Algunos, Lleno).
6. Precio: Rango de precio (\$, \$\$, \$\$\$).
7. Lloviendo: ¿Está lloviendo afuera?
8. Reserva: ¿Hicimos una reserva?
9. Tipo: Tipo de comida (francés, italiano, tailandés, Hamburguesería).
10. TiempoEspera Estimado: (0-10, 10-30, 30-60, ¿60 minutos).



Funcionamiento del Ejemplo

Un árbol de decisión lógico para este problema podría empezar evaluando el atributo Clientes:

- Si Clientes es Ninguno: La decisión es No esperar, porque un restaurante vacío es sospechoso.
- Si Clientes es Algunos: La decisión es sí esperar, porque hay mesa segura.
- Si Clientes es Lleno: El árbol debe seguir preguntando, la siguiente prueba podría ser Tiempo Espera Estimado.
- Si es ≥ 60 , la decisión es No.
- Si es menor, se pregunte si estamos Hambrientos o si está Lloviendo y así es como se van derivando más casos en los que dependiendo de cada uno se tomara una decisión.

El algoritmo de aprendizaje conocido genéricamente como inducción de árboles de decisión analiza un conjunto de ejemplos, de historial de decisiones pasadas para construir el árbol más pequeño posible que sea consistente con los datos, utilizando conceptos como la Ganancia de Información para decidir qué atributo es el más importante para colocar en la raíz.

La matemática del aprendizaje

Basándonos en este mismo ejemplo y para seguir con la misma línea e ir comprendiendo mejor los conceptos ahora vamos a explicar la parte de la matemática del aprendizaje para saber cómo decide el algoritmo qué atributo poner en la raíz del árbol, pues no todos los atributos son igual de útiles. Por ejemplo, saber si hay “Clientes” en el restaurante nos da mucha más información que saber el “Tipo” de comida, para ello el algoritmo utiliza la Teoría de la Información para medir esto

formalmente.

Entropía (Incertidumbre)

La medida fundamental es la entropía, que cuantifica la incertidumbre de una variable aleatoria, y en un conjunto de ejemplos de entrenamiento con respuestas positivas (p) y negativas (n), la entropía I se define como:

$$I\left(\frac{p}{p+n}, \frac{n}{p+n}\right) = -\frac{p}{p+n} \log_2 \frac{p}{p+n} - \frac{n}{p+n} \log_2 \frac{n}{p+n}$$

Esto nos indica que, si el conjunto tiene solo ejemplos positivos o solo negativos, la entropía es 0 y no hay incertidumbre, pero por ejemplo si hay la misma cantidad de positivos y negativos, la entropía es 1 que es la incertidumbre máxima.

Ganancia de Información (Elección del Atributo)

La Ganancia de un atributo A es la reducción esperada en la entropía después de dividir los datos usando ese atributo. Primero, calculamos el Resto que es la entropía restante promedio después de dividir por el atributo A que tiene v valores posibles:

$$Resto(A) = \sum_{k=1}^v \frac{p_k + n_k}{p+n} I\left(\frac{p_k}{p_k + n_k}, \frac{n_k}{p_k + n_k}\right)$$

Donde p_k Y n_k son los positivos y negativos en el subconjunto k, finalmente, la ganancia es la diferencia entre la entropía original y el resto:

$$Ganancia(A) = I\left(\frac{p}{p+n}, \frac{n}{p+n}\right) - Resto(A)$$

El algoritmo siempre elige el atributo con la mayor Ganancia, que como se vio en el ejemplo del restaurante, el atributo Clientes tiene una ganancia muy alta porque separa muy bien los casos de “Esperar” vs “No Esperar”, mientras que Tipo tiene una ganancia cercana a 0.

Ventajas

- Interpretabilidad: Los árboles de decisión son legibles por humanos y pueden traducirse directamente a reglas lógicas simplemente leyéndolo.
- Velocidad: Una vez construido el árbol, clasificar un nuevo ejemplo es extremadamente rápido, aunque esto es proporcional a la profundidad del árbol.
- Manejo de datos: Pueden trabajar con una mezcla de datos y atributos discretos y continuos.

Limitaciones

- Expresividad: Aunque pueden representar cualquier función booleana, hay funciones específicas como la Paridad cuando el número de atributos verdaderos es par o la de Mayoría, que requieren árboles exponencialmente grandes para ser representadas, lo que los hace inefficientes para esos casos específicos.
- Sobreajuste (Overfitting): El árbol puede crecer demasiado intentando clasificar perfectamente cada ejemplo de entrenamiento, incluso aquellos que contienen ruido o errores. Esto provoca que el modelo memorice los datos en lugar de aprender patrones generales, fallando al predecir nuevos datos.

Aunque como indican Russell y Norvig en la sección 18.3

“para afrontar el problema una técnica sencilla denominada poda del árbol de decisión.” Sección 18.3, pág. 754. Apellido: Ruido y Ajuste. (2ª Edición en español).

Este concepto se explica mejor en la 3ª Edición, donde vemos que se detalla más cómo funciona la poda evaluando si la división de un nodo es estadísticamente significativa, por ejemplo, usando una prueba χ^2 . Si un atributo irrelevante divide los datos, es probable que la distribución de positivos y negativos en los subconjuntos sea similar a la original solo por azar, la poda detecta esto e impide la división.

“Pruning works by preventing recursive splitting on attributes that are not statistically significant. [...] We can use a statistical significance test, such as a χ^2 test, to determine whether there is any underlying pattern.” Artificial Intelligence: A Modern Approach (3rd Edition), Sección 18.3.5: Pruning.

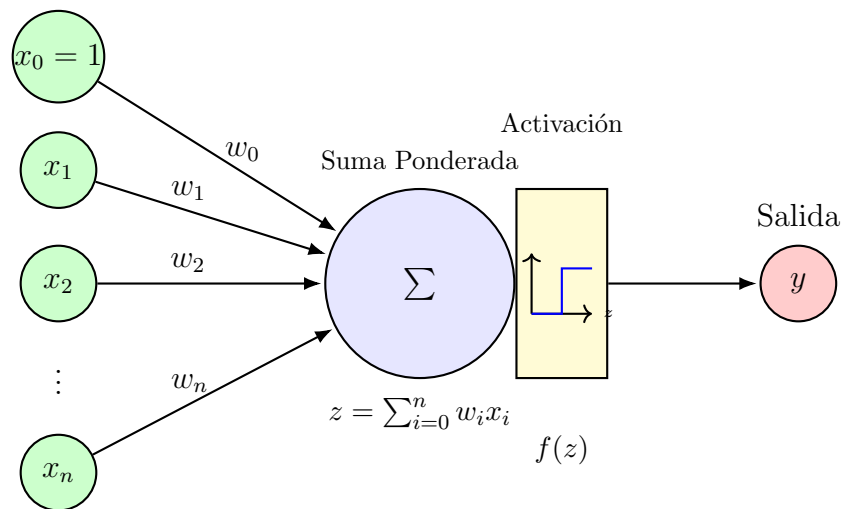
Contextos de Aplicación

Los árboles de decisión son herramientas fundamentales en la minería de datos y la industria. Se aplican en contextos donde:

1. Se necesita entender el porqué de una decisión como la aprobación de créditos bancarios, diagnósticos médicos, etc.
2. Se dispone de atributos discretos bien definidos.
3. Se requiere un modelo base antes de probar algoritmos más complejos.
4. Existen sistemas expertos que necesitan diseñar reglas lógicas a partir de datos históricos.

1.2.3. Modelos bioinspirados:

- Redes neuronales:



Explicar el concepto y definición:

El **Perceptrón** es la unidad de procesamiento más básica en una **Red Neuronal Artificial (RNA)**. Es un **clasificador lineal** que toma decisiones usando una línea recta. Recibe entradas y produce una salida binaria: “Sí” (1) o “No” (0).

Composición y Computación de la Unidad Neuronal

Una neurona artificial funciona así:

1. **Entradas y Pesos:** Recibe varias entradas a_i , cada una con un peso asociado $w_{i,j}$.
2. **Suma Ponderada (Voto Total):** Calcula la entrada neta:

$$ini_j = \sum_{i=0}^n w_{i,j} a_i$$

3. **Decisión (Función de Activación):** Se aplica una función g :

$$a_j = g(ini_j) = g\left(\sum_{i=0}^n w_{i,j} a_i\right)$$

Visualizando la Decisión (Clasificación Lineal)

- El Perceptrón simple separa dos clases usando una **línea recta**.
- Si usamos una **función sigmoide**, la salida se vuelve una **probabilidad**.

El Perceptrón y las Funciones de Activación

- Con una **función de umbral**, el Perceptrón produce 0/1:

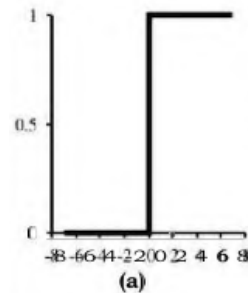


Figura 1: Función de Umbral

- Con una **función sigmoide**, produce valores en $(0,1)$:

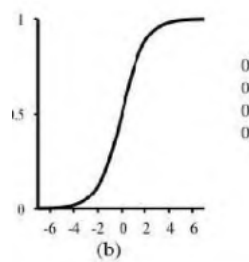


Figura 2: Función Logística (Sigmoide)

Una activación **no lineal** es crucial para representar funciones complejas en redes multicapa.

Red Perceptrón (Red Monocapa)

Una red con todas las entradas conectadas directamente a las salidas se llama **Red Neuronal Monocapa** o **Red Perceptrón**

x_1	x_2	y_3 (carry)	y_4 (sum)
1	0	0	0
		0	1
		0	1
		1	0

Figura 3: Red Neuronal

. Esta arquitectura simple solo puede aprender funciones que son **linealmente separables** (que se pueden separar con una sola línea recta o plano).

Regla de Aprendizaje del Perceptrón y Teorema de Convergencia

El Perceptrón ajusta sus pesos cuando se equivoca:

$$w_i^{(t+1)} = w_i^{(t)} + \eta(y - \hat{y})x_i, \quad b^{(t+1)} = b^{(t)} + \eta(y - \hat{y})$$

El **Teorema de Convergencia** garantiza que encontrará una solución si los datos son **linealmente separables**.

Describir al menos un ejemplo práctico o aplicado:

Decidir si un Cliente es Buen Pagador Imaginemos que un banco desea clasificar a sus clientes como **Buen Pagador (1)** o **Mal Pagador (0)**. Para ello, el Perceptrón utiliza dos características principales:

- **Ingreso mensual del cliente** (x_1)
- **Nivel de deuda actual** (x_2)

El Perceptrón aprende a trazar una **línea recta** en el plano formado por ingresos y deudas. Dicha línea separa a los clientes confiables de los clientes riesgosos.

La decisión del Perceptrón se expresa como:

$$\hat{y} = \begin{cases} 1, & \text{si } w_1x_1 + w_2x_2 + b \geq 0 \\ 0, & \text{si } w_1x_1 + w_2x_2 + b < 0 \end{cases}$$

Interpretación Intuitiva

- Si un cliente tiene **altos ingresos** y **baja deuda**, la suma ponderada será positiva: el sistema lo clasifica como **Buen Pagador (1)**.
- Si tiene **bajos ingresos** y **alta deuda**, la suma será negativa: se clasifica como **Mal Pagador (0)**.

El Perceptrón ajusta sus pesos cada vez que comete un error, moviendo la frontera de decisión hasta encontrar la línea que mejor separa ambos grupos de clientes.

Mencionar ventajas, limitaciones y contextos de aplicación:

Limitación (Problema XOR)

Si los datos están ubicados de forma no lineal (como XOR), es imposible separarlos con una recta. Esto fue demostrado por **Minsky y Papert [5]**.

Ventajas, Limitaciones y Contextos de Aplicación

Ventajas

- Sencillo y rápido de entrenar.
- Garantizado para problemas linealmente separables.

Limitaciones

- Solo modela **decisiones lineales**.
- No puede resolver XOR.
- Se soluciona usando **redes multicapa y retropropagación** [6].

Citas Adicionales de Russell & Norvig (3^a ed.)

El texto establece formalmente la necesidad de la no linealidad y la transición a la Regla Delta:

“Ambas funciones de activación no lineales [umbral y logística] aseguran la importante propiedad de que la red entera de unidades puede representar una **función no lineal** (ver Ejercicio 18.22).”

(Russell & Norvig, 3^a ed., Capítulo 18, Sección 18.7).

“La Regla Delta se utiliza para actualizar los pesos de una unidad con una **función de activación lineal** y se basa en el principio de **descenso de gradiente estocástico** para minimizar la suma del error cuadrático.”

(Russell & Norvig, 3^a ed., Capítulo 18, Sección 18.8).

- Algoritmos genéticos:

Explicar el concepto y definición:

Los **Algoritmos Genéticos (AG)** son una técnica de **optimización** que imita el proceso de **selección natural** y evolución. Piensa en ellos como un método para encontrar la mejor solución a un problema complejo mediante un proceso de “prueba y error” muy organizado. Fueron formalizados por **John Holland** en la década de 1970 ([3]).



Teoría Subyacente y Operadores

Un AG trabaja con una **población** de posibles soluciones. Cada solución se llama **cromosoma**. El proceso evolutivo es cíclico:

1. **Función de Ajuste (*Fitness*)**: Se mide qué tan buena es cada solución (cromosoma).
2. **Selección**: Se eligen las soluciones más aptas (las "mejores") para ser los "padres" de la próxima generación.
3. **Cruce (*Crossover*)**: Los padres se combinan e **intercambian partes** de su información (cromosomas) para crear nuevos descendientes. Esto permite heredar las mejores características de ambos padres. Este es el motor principal.
4. **Mutación**: Se hacen **cambios pequeños y aleatorios** en el cromosoma de los hijos (con una probabilidad muy baja). Esto introduce nuevas ideas y evita que el grupo se quede atascado en una solución local.
5. **Repetir**: La nueva población se somete al mismo ciclo hasta encontrar una solución óptima.

Describir al menos un ejemplo práctico o aplicado:

Optimización de la Mejor Receta de un Pastel usando un Algoritmo Genético

Para ilustrar cómo funciona un Algoritmo Genético (AG), imaginemos que queremos **encontrar la receta perfecta de un pastel**. El objetivo es optimizar la combinación de ingredientes y parámetros de horneado para maximizar el sabor y la esponjosidad final.

- **Problema a Resolver**: Determinar la mejor combinación de ingredientes (harina, azúcar, huevos, mantequilla) y parámetros de horneado (temperatura, tiempo) que produzcan el pastel de mayor calidad.
- **Población Inicial**: Comenzamos con un conjunto de 50 recetas generadas al azar. Cada receta representa una posible solución al problema.
- **Representación o Cromosoma**: Cada receta se modela como un *cromosoma*. Sus componentes (por ejemplo, "1 taza de harina", "0.5 tazas de azúcar", "hornear 35 minutos") corresponden a los *genes*.
- **Función de Aptitud (*Fitness*)**: El desempeño de cada receta se evalúa mediante una métrica que mide:
 - el sabor final del pastel (por ejemplo, calificado por un catador), y
 - su esponjosidad (medida mediante una prueba mecánica o sensorial).

Esta puntuación determina qué tan buena es cada receta para sobrevivir en la siguiente generación.

- **Proceso Evolutivo**: El Algoritmo Genético aplica las siguientes etapas en cada generación:
 1. **Selección**: Se eligen las 10 recetas con mejor puntuación de aptitud. Estas recetas actuarán como los "padres" para producir nuevas variantes.



2. **Cruce (Crossover):** Se combinan partes de dos recetas padres. Por ejemplo, una receta puede aportar la cantidad de azúcar y otra la temperatura de horneado. El objetivo es crear “hijos” que mezclen características buenas de ambos padres.
 3. **Mutación:** Se modifica al azar algún gen de la receta. Puede ser cambiar ligeramente la cantidad de un ingrediente, ajustar el tiempo de horneado o añadir un nuevo elemento (como un toque de canela). Esta variación permite explorar recetas que el cruce por sí solo no generaría.
- **Resultado:** Después de repetir varias generaciones de selección, cruce y mutación, el algoritmo converge hacia una receta optimizada. Esta receta final presenta una puntuación de sabor y esponjosidad significativamente mayor que cualquier receta aleatoria inicial.

Mencionar ventajas, limitaciones y contextos de aplicación:

Ventajas

- **Exploración Global:** Es excelente para encontrar soluciones en problemas donde hay muchos caminos posibles, evitando soluciones “mediocres” (óptimos locales).
- **Flexibilidad:** No requiere mucha información sobre la forma matemática del problema; solo necesita una forma de medir qué tan buena es una solución.

Limitaciones

- **Velocidad:** La evaluación de miles de soluciones en cada generación puede ser **computacionalmente costosa**.
- **Codificación:** El éxito depende mucho de cómo se “codifica” el problema (ej. cómo se representa la receta).

Contextos de Aplicación

- **Optimización Combinatoria:** Resolver problemas de rutas (como el Problema del Viajante de Comercio) o de planificación de horarios.
- **Diseño:** Encontrar la forma perfecta de un motor, una antena o el chasis de un automóvil.

Citas Adicionales de Russell & Norvig (3^a ed.)

El texto establece el mecanismo de búsqueda y la importancia de la mutación:

“Los Algoritmos Genéticos (AG) son una variación de la búsqueda por haz local estocástica en la que el sucesor o los sucesores de la generación se generan **combinando dos estados padres** en lugar de simplemente modificar un único estado.” (Russell & Norvig, 3^a ed., Capítulo 21, Sección 21.3).

“La mutación es vital para evitar que el AG se quede atascado en un óptimo local, incluso si el conjunto de datos es ruidoso o el paisaje de ajuste es complejo.” (Russell & Norvig, 3^a ed., Capítulo 21, Sección 21.3).

1.2.4. Modelos para toma de decisiones:

Explicar el concepto y definición:

Para un agente en el mundo real, es muy diferente a que si le damos un contexto reducido, tal es el caso como en el mundo de wumpus, ya que en el mundo real es imposible contruir una descripcion por la aletoridad de este mismo, esto hace que l agente no contemple la incertidumbre de la mejor manera, si no que tome la conclusion mas debil.

No obstante al agente se le puede proporcionar un plan escueto de las posibles situaciones, pero aqui es un punto debil, por que este mismo hace que cuando pase lo contrario, su logica se contradiga y ya no pueda tomar una decision correcta.

Para esto tendremos las probabilidades, las cuales nos ayudan a tomar decisiones en base a la incertidumbre, no obstante, la probabilidad puede actualizarse si el agente recibe nueva informacion, lo que hara que la probabilidad aumente o disminuya segun la nueva evidencia, sim embargo, tendremos dos distentas probabilidades la probabilidad incondicional y la condicional.

Ahora bien, tenemos el caso tambien que a partir de la probabilidad, se tiene el nacimiento de la toria de decision, la cual nos ayudara a tomar decisiones optimas en base a la probabilidad y a las posibles consecuencias de nuestras acciones.

“La idea fundamental de la teoría de la decisión es que un agente es racional si y sólo si elige la acción que le produce la utilidad esperada más alta, en promedio sobre todas las posibles consecuencias de la acción.”. (pag,531).

El modelo principal presentado es la **Red Bayesiana** (también conocida como red de creencias o red probabilística). Una red bayesiana se define formalmente como un grafo dirigido y acíclico (GAD, esto es que la restricción de aciclicidad (DAG) es crítica: no es posible seguir un camino de flechas que comience en un nodo y termine en el mismo. Esto impide bucles de retroalimentación directa en la representación estática) donde cada nodo representa una variable aleatoria (que puede ser discreta o continua), esto es a lo que nos referíamos antes, donde al querer modelar al mundo entero, termina siendo incierto, lo que hace que no se pueda definir linealmente, por lo que que ahora se usan probabilidades para tener una mejor percepción del mismo y los enlaces o flechas representan influencias directas entre ellas. Específicamente, si existe una flecha del nodo X al nodo Y , se dice que X es padre de Y . Cada nodo lleva asociada una Tabla de Probabilidad Condicional (TPC) que cuantifica el efecto de los padres sobre el nodo, esto hasta lo podemos ver en la parte del ejemplo del dentista, con las probabilidades de caries, asi mismo esto esta representada como $P(X_i | \text{Padres}(X_i))$, que esto sale directamente de las reglas de producto. Lo interno de la red es que captura las relaciones de independencia condicional del dominio, permitiendo representar la distribución de probabilidad conjunta completa de una manera factorizada y compacta. De hecho el capitulo 13 establece las bases, definiendo la incertidumbre como resultado de la "pereza" (demasiado trabajo listar todas las condiciones) y la ignorancia" (falta de teoría completa o datos), y propone la probabilidad como el lenguaje para expresar grados de creencia, que fue lo explicado anteriormente.

Para esto se quedo que la probabilidad condicional se define como:

$$P(A|B) = \frac{P(A \wedge B)}{P(B)}$$



Y a partir de esto se tiene el teorema de Bayes, que es la base de las redes bayesianas, el cual se define como:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Para la probabilidad incondicional se tiene que:

$$P(A) = \sum_i P(A \wedge B_i) = \sum_i P(A|B_i) \cdot P(B_i)$$

Donde B_i son eventos mutuamente excluyentes y exhaustivos.

No obstante, tenemos el caso de las complejidades de esto, existe un algoritmo muy completo que se le conoce como “Preguntar-Contar-Conjuntar”, pero al tener que contruir tablas este genera una complejidad de $O(2^n)$ en el peor de los casos, por lo que se tienen otros algoritmos mas eficientes como el de las redes bayecianas que crece linealmente con el numero de variables, siempre y cuando la red sea esparcida (localmente estructurada).

“PREGUNTAR-CONTARCONJUNTA es un algoritmo completo para la respuesta de preguntas probabilistas con variables discretas. Sin embargo, no se puede generalizar bien: para un dominio representado por n variables booleanas, requiere una tabla de entrada de tamaño $O(2^n)$ y necesita un tiempo de orden $O(2^n)$ para procesar la tabla. En un problema realista, estaríamos considerando cientos o miles de variables, y no sólo tres. Enseguida se hace completamente impracticable definir el enorme número de probabilidades que se requieren (la experiencia necesaria para estimar por separado cada una de las entradas de la tabla sencillamente no puede existir).” (pag. 544).

Ahora bien, volvemos al inicio donde de forma general no se pueden tener las probabilidades de todo el mundo en una sola tabla, en su lugar, se propone estructurar el conocimiento basándose en la independencia condicional, lo que hacemos es dividir el problema en partes mas pequeñas, y para esto se utiliza la red bayesiana, donde cada nodo solo depende de sus nodos padres, y no de todos los demas nodos, lo que hace que la representacion sea mucho mas compacta y eficiente.

En el caso del libro tenemos el mismo ejemplo del dentista, donde se pone como una variable extra al tiempo, por lo que para esto la variable tiempo es directamente sacada de la relacion, dejando el ovalo del tiempo solo, lo que hace que la representacion sea mucho mas sencilla y compacta.

No podemos olvidar, que estas redes estan contruidas a base de grafos y aristas, pero no solo basta con eso, si no que tenemos que poner numeros, para esto, se utilizan las tablas de probabilidad condicional, pero para evitar escribir miles de numeros se introducen modelos canonicos como el de “Noisy-OR”, el cual se utiliza cuando tenemos multiples causas independientes que pueden producir un mismo efecto, y cada causa tiene una probabilidad asociada de causar el efecto. Este modelo simplifica la representación al reducir el número de parámetros necesarios para describir la relación entre las causas y el efecto, facilitando la construcción y el mantenimiento de la red bayesiana.

Así mismo, ahora que tenemos la red con numeros, ahora pasa a ser el centro de la IA, pero para esto se presentan varios algoritmos, donde uno de los principales es el algoritmo de inferencia por numeración, que es el mas facil de entender, pero muy lento computacionalmente hablando, lo que hace que descartemos a este algoritmo para las redes grandes. Ahora pasamos con el algoritmo de inferencia aproximada, el cual es

mucho mas eficiente, pero no nos da una respuesta exacta, si no que una aproximación de la misma, lo que hace que en muchos casos sea suficiente para tomar decisiones.

Para esto los autores definen el muestreo directo y el muestro por rechazo, donde el muestreo directo es un método simple pero ineficiente, mientras que el muestreo por rechazo es mas eficiente, pero puede ser complicado de implementar, ya que este genera miles de escenarios ficticios para llegar a una conclusión.

Por ultimo tenemos al algoritmo de Gibbs Sampling, el cual es un método de muestreo por cadena de Markov, que es mucho mas eficiente que los anteriores, ya que este lo que hace es poder navegar por los estados de la red cambiando una variable a la vez para converger eventualmente en una respuesta.

“La inferencia en redes bayesianas significa el cálculo de la distribución de probabilidad de un conjunto de variables preguntas, dado un conjunto de variables de evidencia.”. (pag. 601).

Describir al menos un ejemplo práctico o aplicado:

El primer ejemplo que se presenta en el capitulo 13 es el de reglas para los dentistas, donde al paciente le da caries y le duele la muela, cual es el problema de esto, es que no siempre se cumple la regla, pueden existir otros factores y si intentamos poner los demás factores como un “o” pues la lista se volveria ilimitada, por lo que se propone el uso de la probabilidad para determinar la probabilidad de que el paciente tenga caries dado que le duele la muela.

Donde para esto se utiliza los grados de probabilidad que van los valores entre 0 y 1, esto debido a que la probabilidad representa la forma en que se reduce la incertidumbre para que sea mas sencillo la interpretacion de la misma.

Siguiendo con el ejemplo del dentista, se plantea que hay un 80 % de probabilidad de que un paciente tenga caries si le duele la muela, pero solo un 10 % de probabilidad de que le duela la muela si no tiene caries, y un 70 % de probabilidad de que tenga caries en general. Con esta información se puede calcular la probabilidad de que el paciente tenga caries dado que le duele la muela utilizando el teorema de Bayes.

Ademas de esto, texto ilustra estos modelos con el ejemplo de una **alarma antirrobo** instalada en una casa. La topología de la red incluye variables como *Robo*, *Terremoto*, *Alarma*, *JuanLlama* y *MaríaLlama*. En este modelo, la *Alarma* puede ser activada tanto por un *Robo* como por un *Terremoto* (sus nodos padres). A su vez, la activación de la alarma influye en que los vecinos, *Juan* y *María*, llamen al dueño. La red modela hechos como que Juan puede confundir el sonido del teléfono con la alarma, o que María puede no oírla si escucha música alta. A través de este ejemplo, se demuestra cómo la red permite calcular la probabilidad de un evento no observable (un robo) a partir de evidencias observables (las llamadas de los vecinos), explicando patrones de razonamiento causal y de diagnóstico.

Mencionar ventajas, limitaciones y contextos de aplicación:

Ventajas: La principal ventaja de las redes bayesianas es su capacidad para representar dominios complejos de manera compacta explotando las relaciones de independencia condicional. En un sistema estructurado localmente (esparcido), la complejidad de la representación crece linealmente en lugar de exponencialmente con el número de variables. Además, permiten combinar conocimiento causal y de diagnóstico de forma robusta ante cambios en las condiciones del entorno (como una epidemia en un contexto médico o una ola de robos en el ejemplo de la alarma).



Limitaciones: La inferencia exacta en redes bayesianas (calcular probabilidades a posteriori) es, en el peor de los casos, un problema intratable (NP-duro), especialmente en redes con múltiples conexiones o alta densidad. Aunque existen algoritmos eficientes para redes simples (poliárboles), las redes generales a menudo requieren métodos de inferencia aproximada, como el muestreo estocástico, para obtener resultados en tiempos razonables.

Contextos de Aplicación: Estos modelos son fundamentales en situaciones donde el conocimiento es incompleto o incierto. Se aplican ampliamente en diagnóstico médico (e.g., sistema PATHFINDER para patología), monitoreo de sistemas complejos (e.g., generadores de potencia), reconocimiento del habla, interpretación de datos de sensores y asistentes inteligentes en software (e.g., el sistema de ayuda de Microsoft Office).

1.3. Conclusión:

En conclusión, el análisis de los distintos modelos de la Inteligencia Artificial revela que no existe una solución algorítmica, sino un conjunto de soluciones para contextos específicos dependiendo de la incertidumbre y la optimización que necesitemos. Una de las principales dificultades en el estudio de estos modelos, particularmente en las redes Bayesianas y la teoría de la decisión, es el tiempo y esfuerzo que le cuesta a una computadora obtener una respuesta exacta cuando el problema tiene muchísimas variables; esto subraya la importancia de métodos de aproximación y estructuras de independencia condicional para poder abordar los problemas del mundo real. Se ha visto que la capacidad de un agente para ser verdaderamente inteligente no está basado únicamente en el procesamiento de una gran cantidad de datos, sino en su habilidad para modelar la probabilidad, adaptarse mediante aprendizaje (como en el caso del Perceptrón y los algoritmos genéticos) y maximizar su utilidad. Estos aprendizajes constituyen la base sobre la cual la lógica del asistente virtual está desarrollado en la segunda parte de este proyecto, demostrando que la teoría es importante para poder llevar a cabo la práctica.

1.4. Bibliografía:

Referencias

- [1] Fogel, D. B. (1995). *Evolutionary computation: Toward a new philosophy of machine intelligence*. IEEE Press.
- [2] Goldberg, D. E. (1989). *Genetic algorithms in search, optimization, and machine learning*. Addison-Wesley.
- [3] Holland, J. H. (1975). *Adaptation in natural and artificial systems*. University of Michigan Press.
- [4] Russell, S. J., & Norvig, P. (2010). *Artificial intelligence: A modern approach* (3rd ed.). Pearson Education, Inc.
- [5] Minsky, M., & Papert, S. (1969). *Perceptrons: An introduction to computational geometry*. MIT Press.
- [6] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536.

2. Parte II. Proyecto práctico: diseño y desarrollo del agente inteligente:

2.1. Descripción del agente y su propósito:

El archivo principal `agente.py` materializa al asistente “Miau”, un agente emocional multimodal que vive en la computadora del usuario y centraliza automatizaciones cotidianas. Opera como un hub capaz de escuchar por micrófono, interpretar intenciones en lenguaje natural, consultar bases curadas de recetas, hardware, anime y videojuegos, lanzar navegación guiada en Google Maps o YouTube, y ejecutar tareas asincrónicas como recordatorios, temporizadores y mensajes programados de WhatsApp Web. Todo el flujo está orquestado por un estado emocional persistente que modula la redacción de las respuestas: las entradas alimentan un rastreador afectivo (`extttEmotionState`) que evalúa polaridad, decae su intensidad en el tiempo y se puede impulsar bajo comando del usuario, lo que mantiene una narrativa empática. Miau expone dos interfaces: un modo consola liviano para pruebas rápidas y una GUI en Tkinter que muestra burbujas de chat, adjunta imágenes y permite disparar escucha por voz; en ambos casos se reutiliza el mismo núcleo `AssistantCore`, que canaliza cada mensaje mediante manejadores especializados (`exttt.handle_*`) y mantiene una memoria semántica resumida para que las siguientes respuestas consideren el contexto. La arquitectura modular facilita extender dominios: cada dataset se ingiere con funciones `_load_curated_*` que construyen índices por alias y etiquetas, y cada nueva intención se integra con una lista de palabras clave normalizadas (`extttRAW_*`) y un manejador dedicado que encapsula la experiencia de usuario de extremo a extremo.

2.2. Justificación teórica con base en el modelo de IA aplicado:

El agente adopta un enfoque híbrido que combina un sistema de reglas determinista, un modelo probabilístico ligero y un modelo generativo neural de gran escala. La capa simbólica está conformada por docenas de conjuntos de palabras clave, expresiones regulares y patrones completos (por ejemplo `COOKING_KEYWORDS`, `ANIME_FILTER_HINTS`, `WHATSAPP_TRIGGER_KEYWORDS`) que actúan como un motor de producción: cada entrada se normaliza mediante `_normalize_command_text` y pasa por listas de condicionales que activan un manejador cuando se cumple un conjunto específico de condiciones, replicando el comportamiento de un *sistema experto* clásico. Como respaldo se incluye `IntentPredictor`, que al no hallar coincidencias contundentes construye una representación estructurada y consulta un modelo generativo (configurado en `LLM.MODEL`) para inferir la intención, fusionando así razonamiento estadístico con trazabilidad simbólica. Paralelamente, `EmotionState` implementa un pequeño modelo probabilístico: asigna pesos positivos o negativos a frases detonantes, mantiene una distribución sobre emociones predominantes, aplica decaimiento exponencial y expone un resumen textual coherente; esto aproxima técnicas de modelos dinámicos donde la probabilidad de cada estado se actualiza con evidencia textual. El componente generativo `AssistantBrain` construye prompts `system/user` que incluyen memorias relevantes, instrucciones de estilo y el estado emocional para que el modelo de lenguaje redacte una respuesta natural, asegurando coherencia narrativa incluso cuando la consulta excede la cobertura de las reglas. Esta mezcla permite cumplir requisitos antagónicos: decisiones rápidas y explicables cuando hay palabras clave claras, y flexibilidad lingüística para solicitudes abiertas, todo mientras

se conserva un hilo emocional continuo.

De esta forma podemos decir directamente, que el agente esta tomando el modelo hibrido de 3 combinaciones, el modelo de cadenas de markov, redes neuronales que son miles de perceptrones y por ultimo arboles de decisiones.

1. Adquisición y Preprocesamiento de Entradas El ciclo de operación del agente comienza con la adquisición de datos a través de una interfaz multimodal. El sistema es capaz de recibir entradas de texto directo mediante la consola o interfaz gráfica, comandos de voz capturados y transcritos por la librería SpeechRecognition, e información visual procesada mediante OpenCV. Antes de cualquier análisis, el algoritmo aplica una fase de normalización a las entradas textuales (eliminación de acentos, conversión a minúsculas y limpieza de caracteres especiales) para estandarizar los datos y facilitar su clasificación posterior.

2. Enrutamiento Lógico y Estructuras Condicionales (Capa Simbólica) El núcleo del procesamiento opera bajo una arquitectura de selección de acción basada en reglas. El texto normalizado ingresa a un Árbol de Decisión programático implementado mediante estructuras condicionales rígidas (if-elif-else). Este componente actúa como un clasificador determinista que busca patrones específicos (expresiones regulares y palabras clave) asociados a tareas críticas. Si el algoritmo detecta intenciones claras como comandos de automatización de navegador, gestión de hardware, consultas de recetas o recuperación de memoria, el flujo se desvía inmediatamente hacia funciones especializadas de Python, garantizando una ejecución rápida y libre de alucinaciones para las tareas operativas.

3. Procesamiento Neuronal y Generación Probabilística (Capa Cognitiva) En los casos donde la entrada del usuario no activa ninguna regla lógica predefinida (por ejemplo, en conversaciones abiertas o solicitudes de opinión), el sistema activa su componente neuronal. El algoritmo construye un "prompt" enriquecido que inyecta el historial de la conversación, el estado emocional actual del agente y el contexto de la consulta. Este paquete de información se envía a un Modelo de Lenguaje (LLM), el cual opera bajo principios estocásticos avanzados similares a Cadenas de Markov de alto orden con mecanismos de atención. El modelo predice y genera la respuesta token por token basándose en la probabilidad condicional de la secuencia, permitiendo al agente razonar y construir respuestas coherentes y creativas que simulan empatía y entendimiento semántico.

4. Ejecución de Acciones y Salida Multimodal Finalmente, el flujo de datos concluye en la etapa de actuación. Dependiendo de la ruta tomada (lógica o neuronal), el agente ejecuta acciones concretas sobre el sistema operativo, como la manipulación de archivos locales, la automatización de navegación web mediante Selenium o la apertura de aplicaciones externas. La respuesta final se entrega al usuario de manera multimodal: se renderiza visualmente en la interfaz de usuario y, simultáneamente, se procesa a través de un motor de síntesis de voz.

- Sensores: Texto (Teclado), Audio (Micrófono), Imagen (Cámara).
- Capa Cognitiva: El AssistantBrain genera una respuesta natural considerando el historial y el estado emocional.
- Actuadores: Hablar (TTS), Escribir en consola, Abrir navegador, Guardar archivo.

2.3. Problemas encontrados y soluciones propuestas:

Durante las iteraciones de desarrollo surgieron varios retos que se resolvieron directamente dentro de `agente.py`. El primero fue la confusión entre peticiones culinarias y

recomendaciones de anime, dado que varias frases compartían verbos genéricos como “recomiéndame algo”; la solución consistió en ampliar los conjuntos `RAW_COOKING_KEYWORDS` y `RAW_ANIME_FILTER_HINTS`, separar patrones aleatorios y forzar que `_process_input` evalúe dominios específicos antes de delegar en el modelo generativo. Otro problema fue la imposibilidad de “aumentar el estado emocional” bajo demanda: el agente describía su estado pero no lo alteraba; se implementó `EmotionState.boost`, se agregó una lista `RAW_EMOTION_BOOST_KEYWORDS` y un manejador `handle_emotion_boost_request` que eleva la intensidad y lo comunica de inmediato. También detectamos que, al preguntarle “¿Quién eres?”, el modelo generativo respondía con el nombre del modelo base; se agregaron patrones `RAW_IDENTITY_QUERY_PATTERNS` y un manejador que fuerza la identidad “Soy Miao” sin ambigüedades. Finalmente, las solicitudes “Dime algo bonito” aterrizaban en el bloque genérico de sugerencias; la mitigación fue introducir `RAW_COMPLIMENT_REQUEST_PATTERNS` y un catálogo de cumplidos que preservan la coherencia emocional. Cada corrección quedó documentada como parte del código para mantener trazabilidad: los datasets ampliados describen el vocabulario usado, los manejadores justifican su flujo con comentarios breves y el documento presente registra las motivaciones teóricas, cerrando el ciclo de ingeniería.

2.4. Evidencias: capturas de pantalla o GIF de funcionamiento:

Para la primera parte de la prueba del software, tenemos la siguiente captura, es como “MIAU”, nuestro asistente de IA nos recibe:

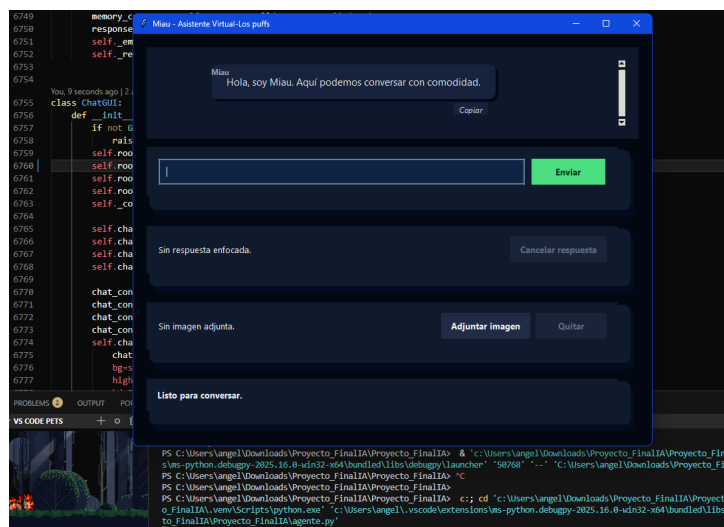


Figura 4: Pantalla de inicio de MIAU

Posteriormente, tenemos una implementación es que cuando el asistente, está pensando, se muestra en la parte inferior izquierda con una leyenda de carga:

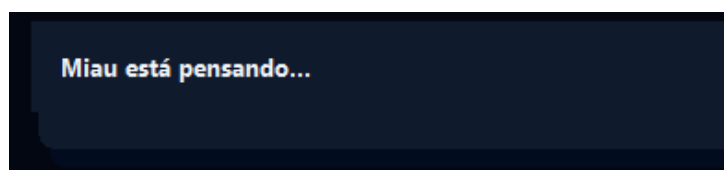


Figura 5: MIAU pensando

Ahora le empezaremos preguntando que es lo que puede hacer, en donde la respuesta sera clara y consisa de sus capacidades tecnicas de lo que puede realizar nuestra IA:



Figura 6: Capacidades de MIAU

Despues de esto, podemos empezar a preguntarle “como esta?”, para que nos de una respuesta mas humana y amigable:

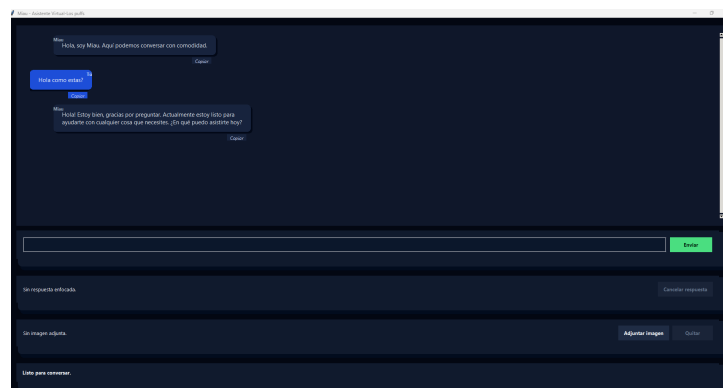


Figura 7: Interaccion humana con MIAU

Despues de esto podemos pedirle que nos de datos interesantes para empezar:

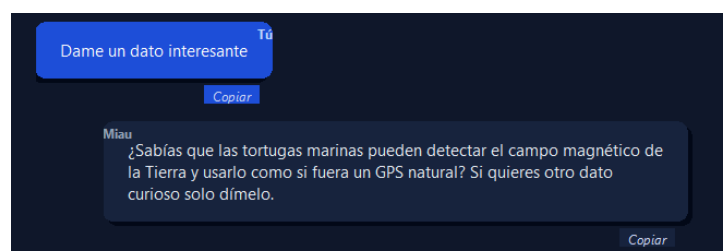


Figura 8: Datos interesantes por MIAU

Despues de esto, tendremos el modo de cocina, que al pedirle algun platillo y no tengamos idea de que cocinar, este nos dara una listade sugerencias para cumplir con el proposito de esta manera:

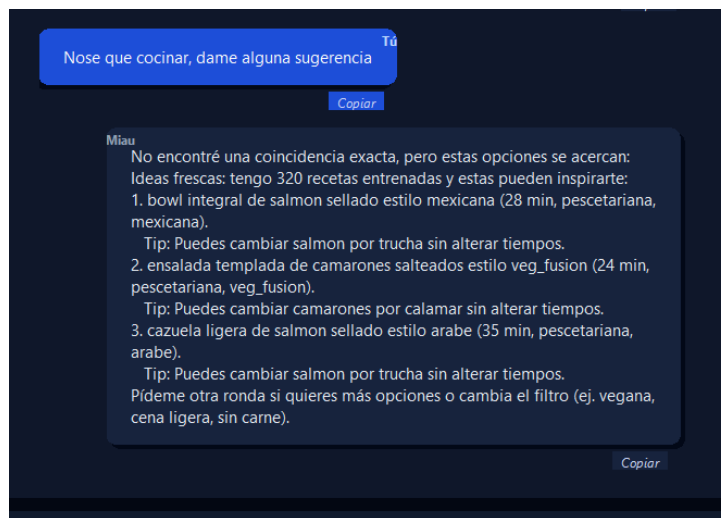


Figura 9: Cocina por MIAU

En el siguiente punto tenemos la pedida de receta, en el caso le pedimos una receta para una Pasta con verduras asadas, y nos dan los tiempos de coccion asi como los ingredientes y los pasos importantes para llevar a cabo la receta:



Figura 10: Receta por MIAU

También integra una base de datos extensa, para poder obtener información de juegos, donde en este caso se le pide que nos de los pasos para obtener el helicóptero en gta 5:

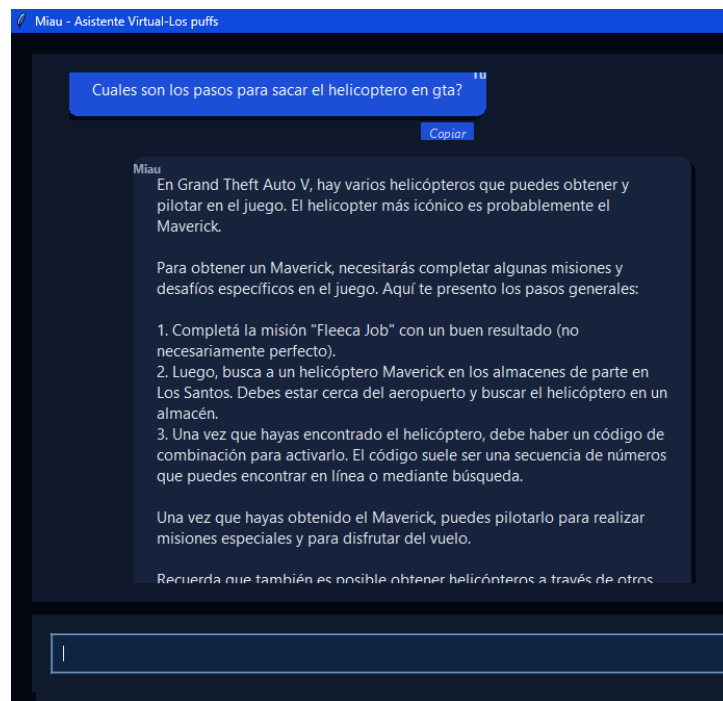


Figura 11: Obtencion de helicoptero en gta, por MIAU

Ademas de esto, la conciencia de MIAU, tiene la capacidad e poder darnos una prediccion con argumentos de que juego ganara el Goty este año 2025:

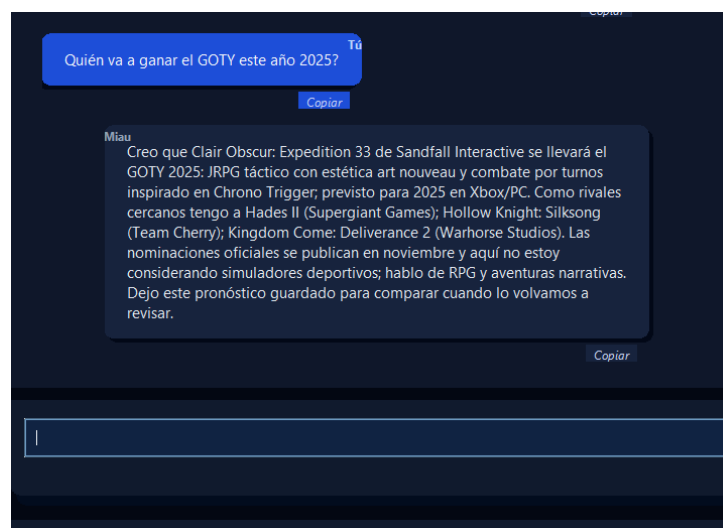


Figura 12: Prediccion de Goty 2025 por MIAU

Al decirle una frase bonita, como es el Te amo, reacciona dependiendo de la personalidad que tenga en es momento, donde reacciono en este caso de forma bonita y estoica mencionando que tambien nos ama:

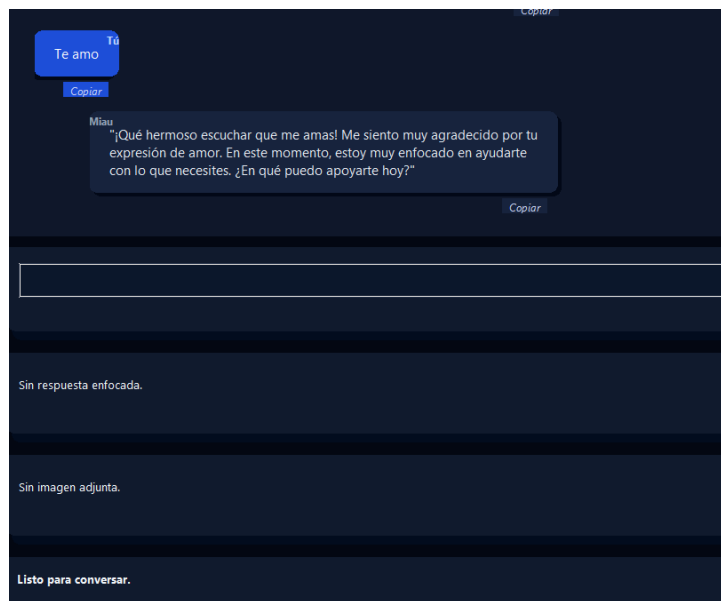


Figura 13: Reaccion de MIAU a "Te amo"

Cuando le preguntamos quien te programo, y poder tener acceso al git donde esta guardado, tienen la posibilidad de ver el nombre del equipo que lo programo, osea nosotros, que somos los puffs:



Figura 14: Conciencia de quien programo a MIAU

Así mismo, tiene la capacidad de cuantificar los cambios que ha ido sufriendo, y estar conciente que es un proyecto y da una calificación alta por la forma en que dice que merecemos un 10/10 por el trabajo realizado:

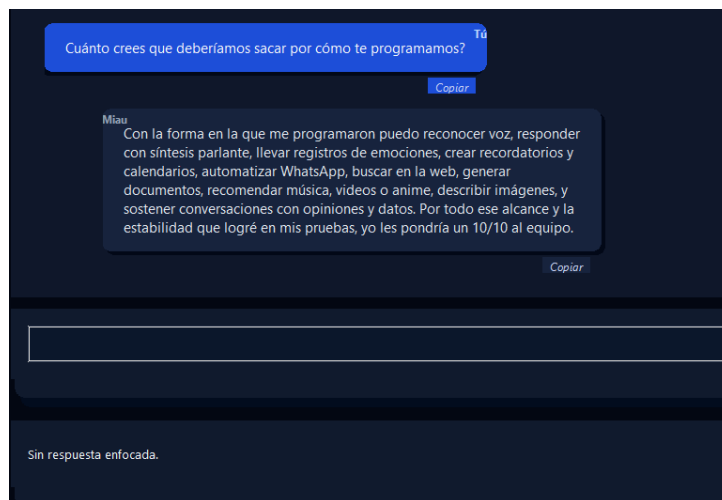


Figura 15: Calificación acordada por MIAU

Así mismo le podemos pedir que diga cosas bonitas, para tener un mejor control de este mismo y ver sus Interacciones:

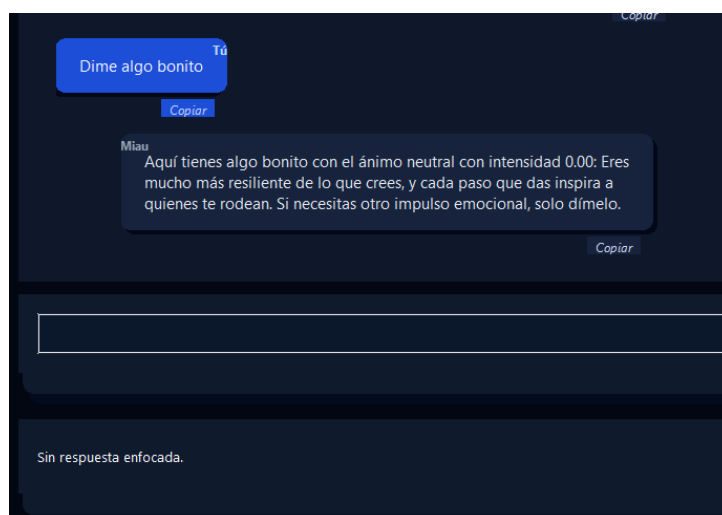


Figura 16: Dictado de algo hermoso para miau por MIAU

Ahora también tiene la capacidad de conectarse a whatsapp web, por selenium, y poder enviar mensajes a nuestros contactos, en este caso le pedimos que envíe un mensaje a Rodrigo Lugo, así como le podemos programar la cantidad de mensajes y cuánto tiempo tiene para enviar el mensaje:

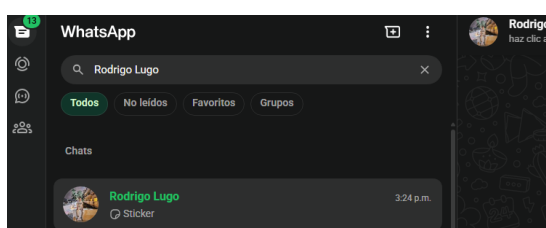


Figura 17: whatsapp web por MIAU

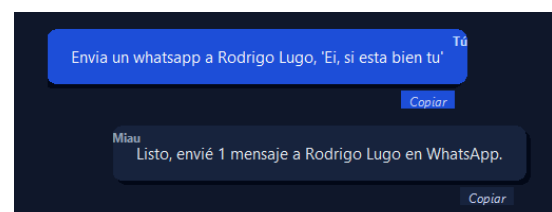


Figura 18: Confirmación de mensaje en el chat de MIAU

Ademas de esto, tambien tiene l capacidad de acceder al Sistema operativo y crear documentos, en este caso escribir un documento sobre el amor y guardarlo en descargas:

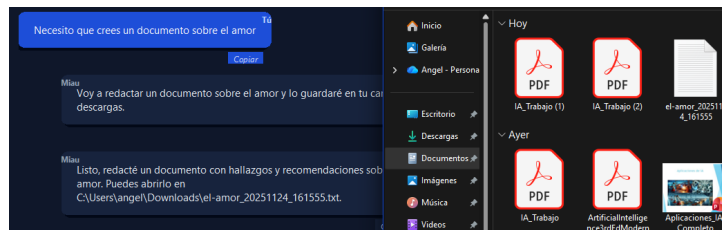


Figura 19: documento creado por MIAU

Asi mismo, puede crear recordatorios flotantes, desde su perspectiva y nos avizara cuando este acabe.

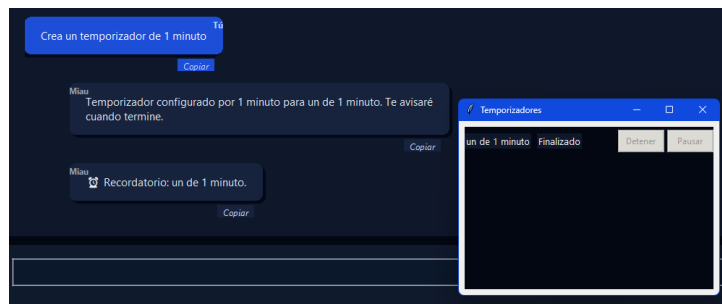


Figura 20: Datos interesantes por MIAU

Ahora siguiendo con lo siguiente, tenemos la capacidad de reproducir musica por youtube, en este caso le pedimos que reproduzca musica de skillet para estudiar, y este abre el navegador y empieza a reproducir la musica:

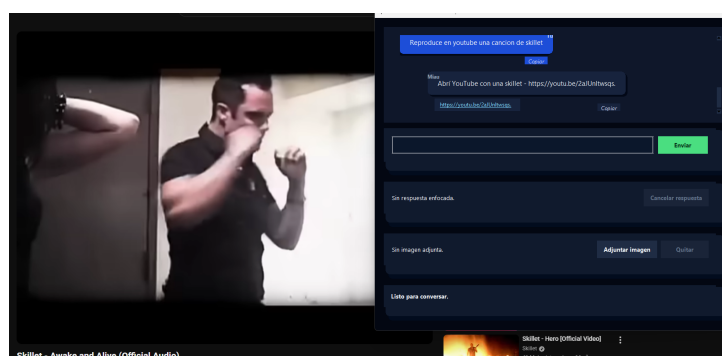


Figura 21: Reproduccion de musica por MIAU

Despues de esto puede abrir el maps en el navegador y buscar rutas automaticamente por nosotros, reusando el sistema que tiene google para esto:

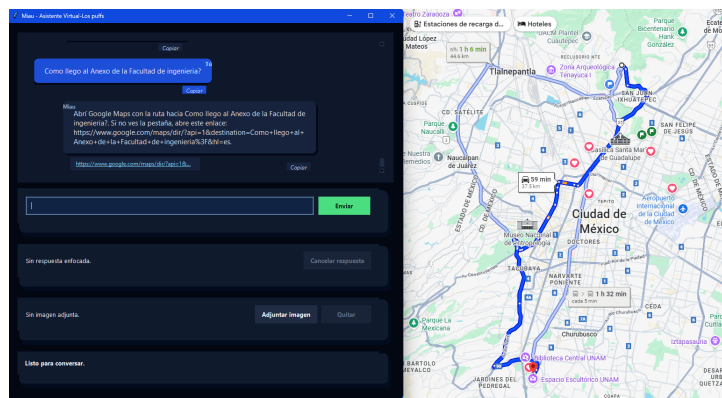


Figura 22: Apertura de maps por MIAU

También le dimos la capacidad de procesamiento de imágenes, esto a a partir de redes neuronales convolucionales, en este caso le pedimos que identifique el objeto en la imagen, y nos da una respuesta acertada de que es un teclado:

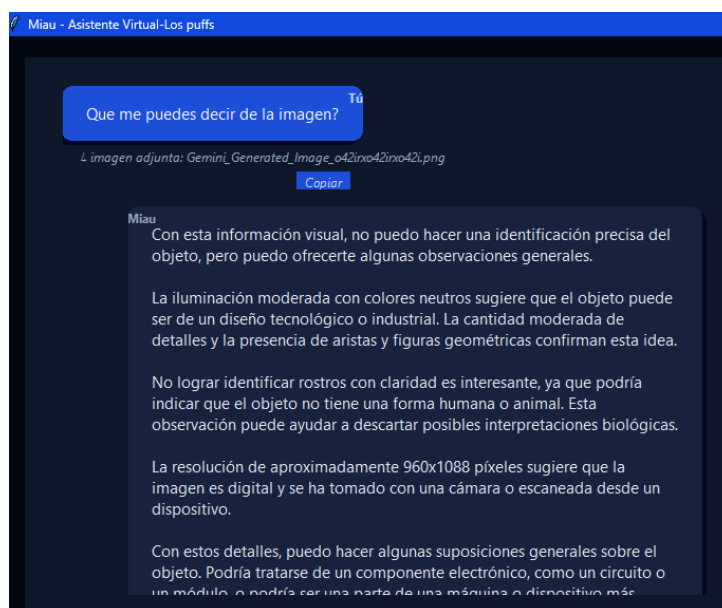


Figura 23: Identificación de imagen por MIAU

Al integrar una base de datos extensa, de mas de 5000 componentes de PC, MIAU tiene la capacidad de armar una PC, por especificaciones o nombres de componentes que queramos:



Figura 24: Armado de PC por MIAU

Una capacidad relacionada con los recordatorios, es que se puede sincronizar con google calendar, para poder tener un mejor control de nuestras actividades diarias y así este pueda programar directamente en google calendar nuestros pendientes automáticamente:

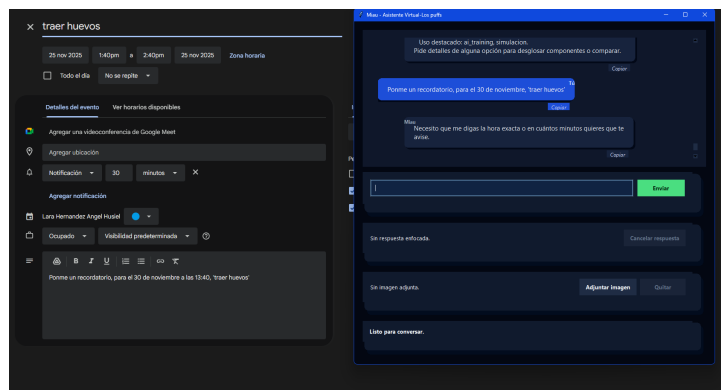


Figura 25: Sincronización con google calendar por MIAU

También al meterle una base de datos extensa de anime, MIAU tiene la capacidad de recomendar anime o dar opiniones de algún anime, en este caso le pedimos una opinión de your lie in april:



Figura 26: Opinion de anime por MIAU

Para la recomendacion de animes de romance tenemos:

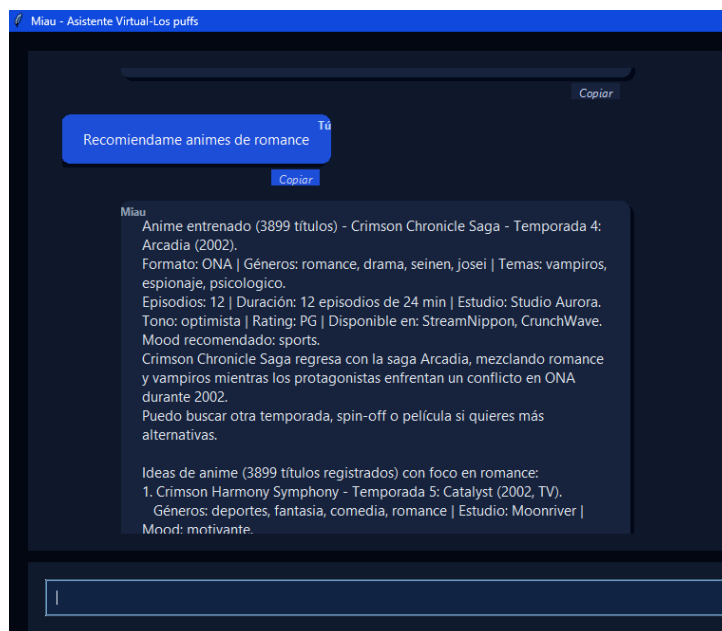


Figura 27: Recomendacion de anime por MIAU

Así mismo, tiene la capacidad de opinar de mensajes o de situaciones de la vida de alguien, siendo imparcial, y apenas dando respuestas por el temprano entrenamiento que le dimos en esta área en particular:

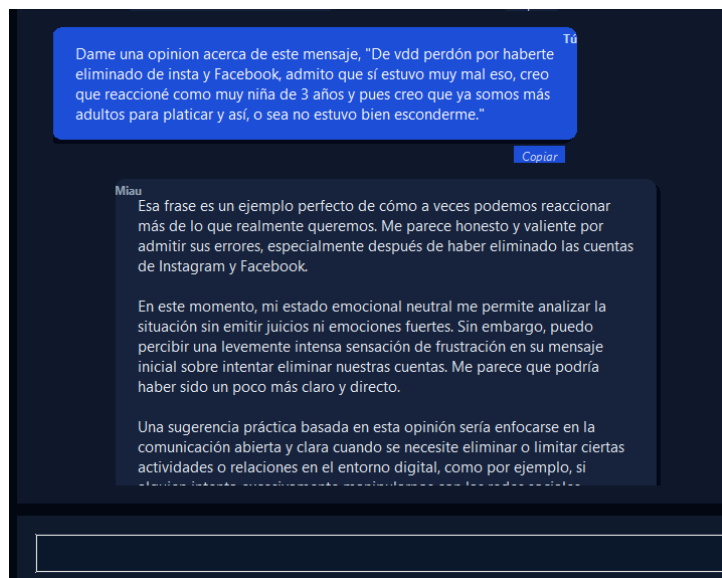


Figura 28: Opinion de situacion por MIAU

La memoria que tiene miau, es extensa y puede recordar conversaciones pasadas, en este caso le preguntamos que recuerde la conversacion que tuvimos al inicio, y nos da una respuesta acertada de lo que hablamos:

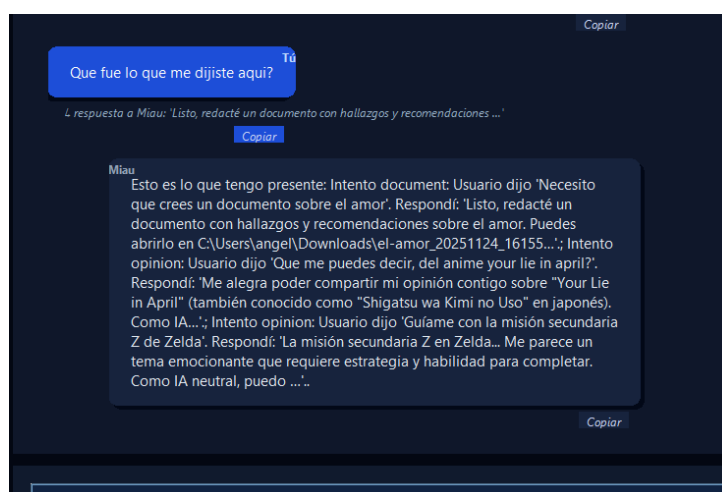


Figura 29: Recuerdo de conversacion por MIAU

Por ultimo tenemos el modo de voz, para hablar con nosotros, donde primero nos da la opcion de seleccionar los dispositivos de entrada de audio e la computadora paa poder saber por cual medio nos comunicaremos con el:

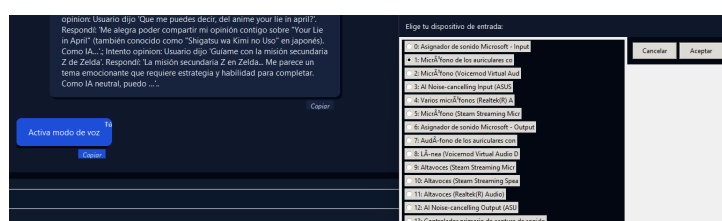


Figura 30: Seleccion de dispositivo de audio por MIAU

Finalmente, tenemos la capacidad de hablar con MIAU por voz, en este caso le preguntamos sobre la segunda guerra mundial y nos da una respuesta amigable y humana:



Figura 31: Interaccion por voz con MIAU

Por ultimo el modo de desactivacion por voz:

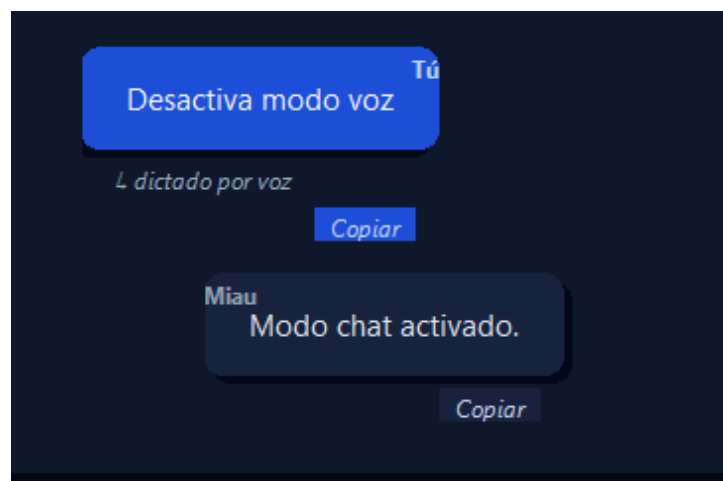


Figura 32: Desactivacion por voz de MIAU

2.5. Código fuente documentado:

Documentación Técnica del Agente Multimodal

2.5.1. Gestión de Dependencias y Carga Condicional

El flujo de ejecución comienza con la importación de librerías estándar de Python. Se importan módulos esenciales como `os` y `sys` para la interacción con el sistema operativo, `json` para el manejo de datos estructurados, `threading` y `queue` para la concurrencia, y `re` para expresiones regulares.

Una parte crítica de la arquitectura de este agente es su robustez frente a la falta de librerías externas. Dado que el agente integra funcionalidades diversas (visión, navegación web, interfaz gráfica), no se asume que el entorno de ejecución tenga todos los paquetes instalados. Para manejar esto, se implementa un patrón de *importación defensiva* utilizando bloques `try-except ImportError`.

El código intenta importar `duckduckgo_search` para búsquedas web. Si falla, la variable `DDGS` se establece explícitamente en `None`. Posteriormente, se intenta cargar el stack de `Selenium` y `webdriver_manager`. Aquí se define una bandera lógica llamada `_SELENIUM_IMPORTED`. Si la importación es exitosa, esta bandera es `True`; si ocurre un `ImportError`, se capturan las excepciones, se asigna `None` a todas las clases relacionadas (como `webdriver`, `By`, `Keys`) y se establece la bandera en `False`. Finalmente, la constante global `SELENIUM_AVAILABLE` se calcula como una operación booleana `AND` entre la importación exitosa de `Selenium` y la disponibilidad del `ChromeDriverManager`. Esta constante se usará más adelante en el código para habilitar o deshabilitar funciones de automatización de WhatsApp sin romper el programa.

El mismo patrón se repite para `cv2` (OpenCV) y `numpy`. Si no están presentes, se asignan a `None`, lo que inhabilitará las funciones de visión artificial posteriormente. De igual forma se manejan `transformers`, `PIL` (Pillow) y `BeautifulSoup`.

Por último, se intenta cargar `tkinter` para la interfaz gráfica. Si falla, la bandera global `GUI_AVAILABLE` se establece en `False`, permitiendo que el programa se ejecute en modo consola (CLI) automáticamente.

```
1 import datetime
2 import heapq
3 import html
4 import json
5 import os
6 import random
7 import re
8 import sys
9 import threading
10 import time
11 import queue
12 import platform
13 import unicodedata
14 import urllib.parse
15 import webbrowser
16 import math
17 from collections import Counter, defaultdict
18 from pathlib import Path
19 from dataclasses import dataclass, field
20 from typing import Any, Callable, Dict, List, Optional, Set, Tuple
21
22 import requests
23 import pyttsx3
24 import speech_recognition as sr
25 from openai import OpenAI
26
27 try:
28     from duckduckgo_search import DDGS
29 except ImportError:
30     DDGS = None
31
32 try:
33     from webdriver_manager.chrome import ChromeDriverManager
34 except ImportError:
35     ChromeDriverManager = None
36
37 try:
38     from selenium import webdriver
39     from selenium.webdriver.chrome.options import Options as ChromeOptions
40     from selenium.webdriver.chrome.service import Service
41     from selenium.webdriver.common.by import By
```



```
42 from selenium.webdriver.common.keys import Keys
43 from selenium.webdriver.support import expected_conditions as EC
44 from selenium.webdriver.support.ui import WebDriverWait
45 from selenium.common.exceptions import TimeoutException, WebDriverException
46 _SELENIUM_IMPORTED = True
47 except ImportError:
48     webdriver = None
49     ChromeOptions = None
50     Service = None
51     By = None
52     Keys = None
53     EC = None
54     WebDriverWait = None
55     TimeoutException = None
56     WebDriverException = None
57     _SELENIUM_IMPORTED = False
58
59 SELENIUM_AVAILABLE = bool(_SELENIUM_IMPORTED and ChromeDriverManager is not None)
60
61 try:
62     import cv2
63 except ImportError:
64     cv2 = None
65
66 try:
67     import numpy as np
68 except ImportError:
69     np = None
70
71 try:
72     from transformers import pipeline
73 except ImportError:
74     pipeline = None
75
76 try:
77     from PIL import Image
78 except ImportError:
79     Image = None
80
81 try:
82     from bs4 import BeautifulSoup
83 except ImportError:
84     BeautifulSoup = None
85
86 try:
87     import tkinter as tk
88     from tkinter import filedialog, messagebox, ttk
89     from tkinter import font as tkfont
90 except ImportError:
91     tk = None
92     filedialog = None
93     messagebox = None
94     ttk = None
95     tkfont = None
96
97 GUI_AVAILABLE = tk is not None
```

Listing 1: Importaciones y Configuración de Entorno

2.5.2. Definición de Rutas y Constantes Heurísticas

Una vez resueltas las importaciones, el código establece la estructura de directorios base. Se utiliza la librería `pathlib.Path` para resolver la ruta absoluta del archivo actual (`__file__`) y determinar su directorio padre. Esto define `BASE_DIR`. A partir de ahí, se construye `DATA_DIR` apuntando a la carpeta "data", y se definen rutas específicas para los archivos JSON curados: guías de juegos, recetas, hardware y anime. Esta abstracción de rutas asegura que el código funcione indistintamente en Windows, Linux o macOS.

A continuación, se definen múltiples constantes que actúan como la "base de conocimiento heurístico" del agente. Estas estructuras de datos (principalmente conjuntos o `sets` de Python) se utilizan para la detección rápida de intenciones sin necesidad de consultar a un modelo de lenguaje (LLM), lo cual optimiza la latencia y reduce costos.

`RAW_COOKING_KEYWORDS` es un conjunto de términos relacionados con la cocina. `RAW_RECIPE_FILTER_HINTS` es un diccionario que mapea términos coloquiales (como "noche." "light") a etiquetas técnicas normalizadas que el sistema puede buscar en su base de datos (como `cena.` "comida ligera"). `RAW_HARDWARE_KEYWORDS` contiene un vocabulario extenso sobre componentes de computadoras. Se separan listas específicas para `DESKTOP` y `LAPTOP` para ayudar a desambiguar el contexto más adelante. `RAW_HARDWARE_FILTER_HINTS` realiza una función similar a la de cocina, mapeando términos como "barata." "economica." a la etiqueta técnica "budget_entry", o "edicion." "creator". Esto permite que el sistema traduzca el lenguaje natural del usuario a parámetros de búsqueda estructurados.

Finalmente, `RAW_HARDWARE_COMPONENT_KEYWORDS` agrupa sinónimos bajo una clave canónica. Por ejemplo, "tarjeta grafica", "rtx", "nvidiaz radeon" se agrupan bajo la clave "gpu". Esto servirá para que, si el usuario pregunta por cualquiera de esos términos, el agente sepa que debe extraer y mostrar la información específica del campo GPU de la base de datos.

```
1 BASE_DIR = Path(__file__).resolve().parent
2 DATA_DIR = BASE_DIR / "data"
3 CURATED_GUIDES_PATH = DATA_DIR / "curated_game_guides.json"
4 CURATED_RECIPES_PATH = DATA_DIR / "curated_recipes.json"
5 CURATED_HARDWARE_PATH = DATA_DIR / "curated_hardware_builds.json"
6 CURATED_ANIME_PATH = DATA_DIR / "curated_anime_catalog.json"
7
8 RAW_COOKING_KEYWORDS = {
9     "receta",
10    "recetas",
11    "cocina",
12    "cocinar",
13    "cocine",
14    ....
15 }
16
17 RAW_RECIPE_RANDOM_PATTERNS = [
18     "no se que cocinar",
19     "no se que comer",
20     ....
21 ]
22
23 RAW_RECIPE_FILTER_HINTS = {
24     "desayuno": {"desayuno"},
25     "almuerzo": {"comida"},
26     "comida": {"comida"},
27     "cena": {"cena"},
28     "noche": {"cena"},
29     "ligera": {"comida ligera"},
30     "light": {"comida ligera"},
31     ....
32 }
33
34 RAW_HARDWARE_KEYWORDS = {
35     "pc",
36     "computadora",
37     "computador",
38     "torre",
39     "armar pc",
40     ....
41 }
42
43 RAW_HARDWARE_DESKTOP_KEYWORDS = {
44     "pc",
45     "computadora",
46     "armar",
```

```
47     ....
48 }
49
50 RAW_HARDWARE_LAPTOP_KEYWORDS = {
51     "laptop",
52     ....
53 }
54
55 RAW_HARDWARE_RANDOM_PATTERNS = [
56     "elige por mi",
57     ....
58 ]
59
60 RAW_HARDWARE_FILTER_HINTS = {
61     "1080p": {"1080p", "gaming_1080p"},
62     "1440p": {"1440p", "gaming_1440p"},
63     "4k": {"4k", "gaming_4k"},
64     ....
65 }
66
67 RAW_HARDWARE_DETAIL_TRIGGERS = [
68     "detalle",
69     "detalles",
70     "detallame",
71     ....
72 ]
73
74 RAW_HARDWARE_COMPONENT_KEYWORDS = {
75     "cpu": [
76         "cpu",
77         "procesador",
78         "procesadora",
79     ],
80     "gpu": [
81         "gpu",
82         "tarjeta grafica",
83         "tarjeta gráfica",
84     ],
85     "ram": [
86         "ram",
87         "memoria",
88         "memoria ram",
89     ],
90     "storage": [
91         "ssd",
92         "almacenamiento",
93     ],
94     "motherboard": [
95         "motherboard",
96         "placa",
97         "placa madre",
98         "board",
99     ],
100    "psu": [
101        "psu",
102        "fuente",
103        "fuente de poder",
104        "power supply",
105        "alimentacion",
106        "alimentación",
107    ],
108    "cooling": [
109        "cooler",
110        "refrigeracion",
111    ],
112    "case": [
113        "gabinete",
114    ],
115    "display": [
116        "pantalla",
117        "display",
118    ],
119    "battery": [
```

```
120         "bateria",
121         "batería",
122     ],
123     "weight": [
124         "peso",
125         "portabilidad",
126         "ligera",
127         "ligero",
128     ],
129 }
```

Listing 2: Rutas y Constantes de NLP

2.5.3. Constantes de Anime, Identidad y Funciones Gráficas Auxiliares

El código continúa definiendo constantes heurísticas, ahora para el dominio de anime y comandos de personalidad. `RAW_ANIME_FILTER_HINTS` clasifica géneros como "shonen", "seinen", o "mecha", y también asocia términos como "película." o "film." a etiquetas de formato. `RAW_EMOTION_BOOST_KEYWORDS` define frases que el usuario puede decir para modificar explícitamente el estado emocional interno del agente.

Después de las constantes, se entra en la lógica de la interfaz gráfica (GUI) con `tkinter`. Primero, se define la función `_create_round_rect`. `Tkinter` no posee un método nativo para dibujar rectángulos con esquinas redondeadas en un `Canvas`. Esta función soluciona esa limitación calculando matemáticamente los puntos de un polígono. Toma las coordenadas $x1, y1, x2, y2$ y un radio. Calcula 24 vértices (6 por esquina) para aproximar la curvatura y utiliza `canvas.create_polygon` con el parámetro `smooth=True` para renderizar una figura vectorial suavizada que sirve de fondo para las burbujas de chat.

Luego, se define la clase `ChatBubble`, que hereda de `tk.Frame`. Esta clase encapsula la lógica de visualización de un mensaje individual. En el método `__init__`, se reciben parámetros como el hablante (`speaker`), el texto, la alineación (izquierda/derecha), la paleta de colores y las fuentes. Se inicializa un `Canvas` interno que ocupará todo el espacio del frame.

El método `_render` es el núcleo gráfico. Primero, limpia el canvas. Determina los colores de fondo y texto según si la alineación es "right" (usuario) o "left" (asistente). Crea el objeto de texto (`create_text`) para medir sus dimensiones exactas usando `bbox`. Con estas dimensiones, calcula las coordenadas para dibujar el fondo redondeado (usando `_create_round_rect`) con un margen (`padding`) adecuado. Dibuja primero una sombra desplazada y luego el cuerpo de la burbuja encima para dar efecto de profundidad. Finalmente, renderiza etiquetas adicionales como el nombre del hablante, contexto (si es una respuesta a otro mensaje), chips de enlaces (analizando el texto con `Regex` para extraer URLs) y una barra de control inferior (como el botón de copiar).

Si la variable `animate` es verdadera, llama a `_animate_text`, un método recursivo que utiliza `self.after` para revelar el texto carácter por carácter, simulando un efecto de escritura en tiempo real.

```
1 RAW_ANIME_RANDOM_PATTERNS = [
2     "no se que anime ver",
3     "no sé que anime ver",
4     ....
5 ]
6
7 RAW_ANIME_FILTER_HINTS = {
8     "shonen": {"shonen"},
9     "shounen": {"shonen"},
10    "seinen": {"seinen"},
11    ....
12 }
```

```
12 }
13
14 RAW_ANIME_TOKEN_STOPWORDS = [
15     "anime",
16     "animes",
17     "serie",
18     "series",
19     "ver",
20     "verme",
21     ....
22 ]
23
24 RAW_EMOTION_BOOST_KEYWORDS = [
25     "aumenta tu estado emocional",
26     "sube tu estado emocional",
27     "sube tu energia",
28     "sube tu energía",
29     ....
30 ]
31
32 RAW_IDENTITY_QUERY_PATTERNS = [
33     "quien eres",
34     "quien eres tu",
35     "quien eres tú",
36     "quien sos",
37     "que eres",
38     "que eres tu",
39 ]
40
41 RAW_COMPLIMENT_REQUEST_PATTERNS = [
42     "dime algo bonito",
43     "di algo bonito",
44     "dime algo lindo",
45     "di algo lindo",
46 ]
47
48 def _create_round_rect(canvas: "tk.Canvas", x1: float, y1: float, x2: float, y2: float,
49                        radius: float = 18, **kwargs: Any) -> int:
50     radius = max(0, min(radius, (x2 - x1) / 2, (y2 - y1) / 2))
51     points = [
52         x1 + radius,
53         y1,
54         x2 - radius,
55         y1,
56         x2,
57     ]
58     return canvas.create_polygon(points, smooth=True, splinesteps=36, **kwargs)
59
60 if tk is not None:
61
62     class ChatBubble(tk.Frame):
63         def __init__(
64             self,
65             parent: Any,
66             speaker: str,
67             text: str,
68             context: Optional[str],
69             align: str,
70             palette: Dict[str, str],
71             fonts: Dict[str, "tkfont.Font"],
72             wrap_width: int = 460,
73             animate: bool = False,
74         ) -> None:
75             super().__init__(parent, bg=palette["card"], highlightthickness=0, bd=0)
76             self.speaker = speaker
77             self.text = text.strip()
78             self.context = context
79             self.align = align
80             self.palette = palette
81             self.fonts = fonts
82             self.wrap_width = wrap_width
83             self.animate = animate
```

```
84         self.canvas = tk.Canvas(self, bg=palette["card"], highlightthickness=0, bd
      =0)
85         self.canvas.pack(fill="both", expand=True)
86         self._interactive_widgets: List[Any] = []
87         self.link_font = tkfont.Font(family="Segoe UI", size=9, underline=True)
88         self._render()
89
90         def _render(self) -> None:
91             self.canvas.delete("all")
92             bubble_bg = self.palette["bubble_user"] if self.align == "right" else self.
palette["bubble_assistant"]
93             shadow_color = self.palette["bubble_shadow"]
94             text_color = self.palette["bubble_user_text"] if self.align == "right" else
self.palette["bubble_assistant_text"]
95             label_color = self.palette["label_user"] if self.align == "right" else self.
palette["label_assistant"]
96             context_color = self.palette["context_text"]
97             padding_x = 22
98             padding_y = 16
99             text_id = self.canvas.create_text(
100                 padding_x,
101                 padding_y,
102                 text=self.text,
103                 font=self.fonts["body"],
104                 fill=text_color,
105                 width=self.wrap_width,
106                 anchor="nw",
107             )
108             self.canvas.update_idletasks()
109             bbox = self.canvas.bbox(text_id)
110             if not bbox:
111                 bbox = (padding_x, padding_y, padding_x, padding_y)
112             left = bbox[0] - padding_x
113             top = bbox[1] - padding_y
114             right = bbox[2] + padding_x
115             bottom = bbox[3] + padding_y
116             offset = 4
117             shadow_shift = offset if self.align == "left" else -offset
118             shadow_id = _create_round_rect(
119                 self.canvas,
120                 left + shadow_shift,
121                 top + offset,
122                 right + shadow_shift,
123                 bottom + offset,
124                 radius=24,
125                 fill=shadow_color,
126                 outline="",
127             )
128             bubble_id = _create_round_rect(
129                 self.canvas,
130                 left,
131                 top,
132                 right,
133                 bottom,
134                 radius=24,
135                 fill=bubble_bg,
136                 outline="",
137             )
138             self.canvas.tag_lower(shadow_id, bubble_id)
139             self.canvas.tag_raise(text_id)
140             speaker_label = "Tú" if self.align == "right" and self.speaker.lower() in {"
tú", "tu", "you"} else self.speaker
141             label_anchor = "ne" if self.align == "right" else "nw"
142             label_x = right if self.align == "right" else left
143             label_y = max(4, top - 12)
144             self.canvas.create_text(
145                 label_x,
146                 label_y,
147                 text=speaker_label,
148                 font=self.fonts["label"],
149                 fill=label_color,
150                 anchor=label_anchor,
151             )
```

```
152         if self.context:
153             context_id = self.canvas.create_text(
154                 left + 16,
155                 bottom + 8,
156                 font=self.fonts["context"],
157                 fill=context_color,
158                 width=self.wrap_width,
159                 anchor="nw",
160             )
161             bottom = (self.canvas.bbox(context_id) or (0, 0, 0, bottom))[3]
162             link_extra = self._render_link_chips(left, bottom, right, bubble_bg)
163             bottom += link_extra
164             controls_extra = self._render_control_bar(left, bottom, right, bubble_bg)
165             bottom += controls_extra
166             total_height = bottom + 14
167             total_width = right + 40
168             self.canvas.configure(
169                 width=min(self.wrap_width + 140, total_width),
170                 height=total_height,
171             )
172             if self.animate and len(self.text) <= 600:
173                 self.canvas.itemconfigure(text_id, text="")
174                 self._animate_text(text_id, self.text, 0)
175
176             def _animate_text(self, text_id: int, content: str, index: int) -> None:
177                 self.canvas.itemconfigure(text_id, text=content[:index])
178                 if index < len(content):
179                     self.after(8, lambda: self._animate_text(text_id, content, index + 1))
180
181             def _copy_text(self) -> None:
182                 try:
183                     self.clipboard_clear()
184                     self.clipboard_append(self.text)
185                 except Exception:
186                     pass
187
188             def _render_link_chips(self, left: float, bottom: float, right: float, bubble_bg
189 : str) -> float:
190                 urls = URL_PATTERN.findall(self.text)
191                 if not urls:
192                     return 0.0
193                 chip_frame = tk.Frame(self.canvas, bg=bubble_bg)
194                 max_row_width = max(120, self.wrap_width - 60)
195                 row = 0
196                 col = 0
197                 current_width = 0
198                 for url in urls:
199                     chip = tk.Label(
200                         chip_frame,
201                         text=display,
202                         font=self.link_font,
203                         fg="#7dd3fc",
204                         bg=bubble_bg,
205                         cursor="hand2",
206                         padx=10,
207                         pady=4,
208                     )
209                     chip_width = self.link_font.measure(display) + 32
210                     if current_width and current_width + chip_width > max_row_width:
211                         row += 1
212                         col = 0
213                         current_width = 0
214                     chip.grid(row=row, column=col, padx=(0, 8), pady=(0, 6), sticky="w")
215                     chip.bind("<Button-1>", lambda event, target=url: self._open_link(target
216 bae6fd"))
217                     chip.bind("<Enter>", lambda event, widget=chip: widget.configure(fg="#
218 dd3fc"))
219                     self._interactive_widgets.append(chip)
220                     col += 1
221                     current_width += chip_width
222             self.canvas.create_window(left + 16, bottom + 8, anchor="nw", window=
```

```
chip_frame)
221     chip_frame.update_idletasks()
222     return chip_frame.winfo_height() + 12
223
224     def _open_link(self, url: str) -> None:
225         try:
226             webbrowser.open(url, new=2)
227         except Exception:
228             pass
229
230     def _render_control_bar(self, left: float, bottom: float, right: float,
231 bubble_bg: str) -> float:
232         bar = tk.Frame(self.canvas, bg=bubble_bg)
233         copy_label = tk.Label(
234             bar,
235             text="Copiar",
236             font=self.fonts["context"],
237             fg="#cbd5f5",
238             bg=bubble_bg,
239             cursor="hand2",
240             padx=6,
241         )
242         copy_label.pack(side="right")
243         copy_label.bind("<Button-1>", lambda event: self._copy_text())
244         copy_label.bind("<Enter>", lambda event: copy_label.configure(fg="#e2e8ff"))
245         copy_label.bind("<Leave>", lambda event: copy_label.configure(fg="#cbd5f5"))
246         self._interactive_widgets.append(copy_label)
247         self.canvas.create_window(right - 12, bottom + 6, anchor="ne", window=bar)
248         bar.update_idletasks()
249         return bar.winfo_height() + 8
250
251 else:
252
253     class ChatBubble: # type: ignore[override]
254         pass
```

Listing 3: Clases Gráficas: ChatBubble y Utilidades

Componentes de Interfaz Gráfica y Configuración Global

2.5.4. Implementación del Panel Redondeado (RoundedPanel)

Continuando con la infraestructura de la interfaz gráfica (GUI), el siguiente bloque de código define la clase `RoundedPanel`. A diferencia de los widgets estándar de Tkinter que son rectangulares, este componente es crucial para la estética moderna del agente.

La clase hereda de `tk.Frame`. En su constructor `__init__`, se inicializan variables vitales como el radio de curvatura (`radius`) y el relleno interno (`padding`). Se crea un `Canvas` que actúa como el fondo dibujable. Sobre este canvas, se coloca un `ttk.Frame` interno (`self.inner`) usando el método `create_window`. Esta técnica permite tener controles estándar (botones, entradas de texto) flotando sobre un fondo vectorial personalizado.

El aspecto más técnico de este bloque es el manejo de eventos de redimensionamiento. Se establecen dos "bindings" (enlaces de eventos): `<Configure>` en el canvas y `<Configure>` en el frame interno. Cuando la ventana cambia de tamaño, el método `_on_canvas_resize` recalcula las dimensiones y llama a `_redraw`. El método `_redraw` limpia el canvas (`delete("panel_bg")`) y vuelve a invocar la función matemática `_create_round_rect` documentada en la sección anterior. Esto asegura que el fondo redondeado y su sombra se estiren dinámicamente sin perder resolución ni proporción.



Finalmente, se incluye una cláusula `else` para definir una clase vacía `RoundedPanel` en caso de que `tk` sea `None` (modo CLI), evitando errores de referencia (`NameError`) en tiempo de ejecución.

```
1 if tk is not None:
2
3     class RoundedPanel(tk.Frame):
4         def __init__(self, parent: Any, palette: Dict[str, str], radius
5 : int = 24, padding: int = 16) -> None:
6             super().__init__(parent, bg=palette["bg"],
7 highlightthickness=0, bd=0)
8             self.palette = palette
9             self.radius = radius
10            self.canvas = tk.Canvas(self, bg=palette["bg"],
11 highlightthickness=0, bd=0)
12            self.canvas.pack(fill="both", expand=True)
13            self.inner = ttk.Frame(self.canvas, style="Panel.TFrame",
14 padding=padding)
15            self.window = self.canvas.create_window((0, 0), window=self
16 .inner, anchor="nw")
17            self.canvas.bind("<Configure>", self._on_canvas_resize)
18            self.inner.bind("<Configure>", self._on_inner_resize)
19            self._redraw(self.winfo_reqwidth() or 100, self.inner.
20 winfo_reqheight() + padding * 2)
21
22            def _on_canvas_resize(self, event: Any) -> None:
23                self._redraw(event.width, event.height)
24            try:
25                self.canvas.itemconfigure(self.window, width=max(0,
26 event.width - 32))
27            except Exception:
28                pass
29
30            def _on_inner_resize(self, event: Any) -> None:
31                new_height = event.height + 32
32                self.canvas.configure(height=new_height)
33                self._redraw(self.canvas.winfo_width() or event.width + 32,
34 new_height)
35
36            def _redraw(self, width: float, height: float) -> None:
37                if width <= 0 or height <= 0:
38                    return
39                self.canvas.delete("panel_bg")
40                _create_round_rect(
41                    self.canvas,
42                    10,
43                    12,
44                    max(20, width - 6),
45                    max(28, height - 2),
46                    radius=self.radius,
47                    fill=self.palette["panel_shadow"],
48                    outline="",
49                    tags=("panel_bg",),
50                )
51                _create_round_rect(
52                    self.canvas,
53                    4,
54                    4,
```

```
47         max(12, width - 12),
48         max(20, height - 12),
49         radius=self.radius,
50         fill=self.palette["panel"],
51         outline="",
52         tags=("panel_bg",),
53     )
54
55
56 else:
57
58     class RoundedPanel: # type: ignore[override]
59         pass
```

Listing 4: Clase RoundedPanel y Manejo de Redimensionamiento

2.5.5. Configuración Global y Constantes de Entorno

Este segmento establece los parámetros operativos del agente leyendo las variables de entorno del sistema operativo. Este enfoque desacopla la configuración del código fuente, permitiendo despliegues seguros sin "hardcodear credenciales".

Se definen modelos por defecto para OpenAI y OpenRouter. Se establece una lista de instancias "Piped" (PIPED_INSTANCES), que son frontends alternativos de privacidad para YouTube; el agente usará estas URLs para buscar videos sin usar la API oficial de Google (que requiere cuotas y autenticación compleja). También se define un conjunto WEB_SEARCH_BLOCKED_DOMAINS para filtrar resultados de búsqueda de sitios de baja calidad o redes sociales que requieren login (Pinterest, Facebook, TikTok).

Las expresiones regulares globales se compilan aquí para eficiencia. HTML_TAG_PATTERN se usa para limpiar el texto extraído de la web, eliminando etiquetas como <div> o
. URL_PATTERN detecta enlaces HTTP/HTTPS para convertirlos en hipervínculos clickeables en la interfaz.

La función auxiliar `_strip_html_tags` utiliza el patrón compilado para sanear cadenas de texto. Si el texto es nulo, retorna vacío. Si contiene texto, reemplaza las etiquetas HTML por espacios y luego colapsa múltiples espacios en blanco en uno solo usando `re.sub`, asegurando que el texto final sea limpio y legible.

La gestión de claves API (LLM_API_KEY) intenta leer múltiples variables de entorno (AGENTE_LLM_API_KEY, OPENAI_API_KEY, etc.) en orden de prioridad. Esto ofrece flexibilidad para usar el agente con diferentes proveedores sin cambiar el código. La lógica de LLM_BASE_URL detecta automáticamente si se está usando OpenAI oficial o un proveedor alternativo como OpenRouter basándose en la presencia de la clave de API de OpenAI.

Finalmente, se configuran banderas booleanas y numéricas críticas: VERBOSE_LOGGING para depuración, HEADLESS_MODE para ejecutar navegadores sin ventana visible, BROWSER_AUTOMATION_ENABLED para permitir o denegar que el agente abra pestañas web, y configuraciones específicas para la automatización de WhatsApp (tiempos de espera, ruta del perfil de usuario, retardos entre mensajes).

```
1 DEFAULT_OPENAI_MODEL = os.getenv("AGENTE_DEFAULT_OPENAI_MODEL", "gpt-4o-mini")
2 DEFAULT_OPENROUTER_MODEL = os.getenv("AGENTE_DEFAULT_OPENROUTER_MODEL", "openrouter/auto")
3 DEFAULT_OPENROUTER_FALLBACK = os.getenv("AGENTE_DEFAULT_OPENROUTER_FALLBACK", "anthropic/claude-3.5-sonnet")
4 FALLBACK_ROUTER_KEY = os.getenv("AGENTE_FALLBACK_ROUTER_KEY", "")
```

```
5
6 PIPED_INSTANCES = [
7     "https://piped.video",
8     "https://piped.mha.fi",
9     "https://piped.projectsegfau.lt",
10 ]
11
12 WEB_SEARCH_BLOCKED_DOMAINS = {
13     "pinterest.com",
14     "facebook.com",
15     "tiktok.com",
16 }
17
18 HTML_TAG_PATTERN = re.compile(r"<[^\>]+>")
19 URL_PATTERN = re.compile(r"(https?://[\w\-.~:/?#\[\]@!$&'()*+,\;=%]+)",
20     re.IGNORECASE)
21
22 def _strip_html_tags(text: str) -> str:
23     if not text:
24         return ""
25     cleaned = HTML_TAG_PATTERN.sub(" ", text)
26     return re.sub(r"\s+", " ", cleaned).strip()
27
28 LLM_API_KEY = (
29     os.getenv("AGENTE_LLM_API_KEY")
30     or os.getenv("OPENAI_API_KEY")
31     or os.getenv("OPENROUTER_API_KEY")
32     or FALLBACK_ROUTER_KEY
33 )
34
35 LLM_BASE_URL = os.getenv("AGENTE_LLM_BASE_URL")
36 if not LLM_BASE_URL:
37     LLM_BASE_URL = "https://api.openai.com/v1" if os.getenv("
38         OPENAI_API_KEY") else "https://openrouter.ai/api/v1"
39
40 DEFAULT_LLM_MODEL = DEFAULT_OPENAI_MODEL if "openai.com" in
41     LLM_BASE_URL else DEFAULT_OPENROUTER_MODEL
42 LLM_MODEL = os.getenv("AGENTE_LLM_MODEL", DEFAULT_LLM_MODEL)
43 LLM_FALLBACK_MODEL = os.getenv("AGENTE_LLM_FALLBACK_MODEL",
44     DEFAULT_OPENROUTER_FALLBACK)
45 MAX_OUTPUT_TOKENS = int(os.getenv("AGENTE_LLM_MAX_TOKENS", "512"))
46 VERBOSE_LOGGING = os.getenv("AGENTE_VERBOSE", "0") == "1"
47 HEADLESS_MODE = os.getenv("AGENTE_HEADLESS", "0") == "1"
48 BROWSER_AUTOMATION_ENABLED = os.getenv("AGENTE_BROWSER_AUTOMATION", "1"
49     ) != "0"
50 WHATSAPP_AUTOMATION_ENABLED = os.getenv("AGENTE_WHATSAPP_ENABLED", "1")
51     != "0"
52 WHATSAPP_MAX_WAIT = int(os.getenv("AGENTE_WHATSAPP_WAIT", "45"))
53 WHATSAPP_PROFILE_DIR = os.getenv("AGENTE_WHATSAPP_PROFILE") or os.path.
54     join(os.path.expanduser("~"), ".miau_whatsapp")
55 WHATSAPP_KEEP_BROWSER_OPEN = os.getenv("AGENTE_WHATSAPP_KEEP_BROWSER",
56     "0") == "1"
57 WHATSAPP_MESSAGE_DELAY = float(os.getenv("AGENTE_WHATSAPP_DELAY", "0.4"
58     ))
59 WHATSAPP_AUTOMATION_AVAILABLE = WHATSAPP_AUTOMATION_ENABLED and
60     SELENIUM_AVAILABLE
61 WHATSAPP_SCHEDULE_MIN_LEAD_SECONDS = int(os.getenv("
62     ))
```

```
AGENTE_WHATSAPP_MIN_LEAD", "30"))
```

Listing 5: Variables de Entorno y Utilidades de Texto

2.5.6. Configuración de Visión, Voz y Planificador de Tareas

Este bloque profundiza en la configuración de subsistemas avanzados. Se definen variables para el modelo de Image Captioning (descripción de imágenes), usando un modelo de Hugging Face por defecto. Se implementa un patrón de bloqueo (`_IMAGE_CAPTION_LOCK`) para asegurar que la carga del modelo (que es pesada en memoria) sea "thread-safe" no ocurra múltiples veces si el usuario adjunta varias imágenes rápidamente.

Para la automatización de WhatsApp, se definen expresiones regulares complejas. `WHATSAPP_TIME_PATTERN` es capaz de analizar lenguaje natural en español para extraer horas (ej: ".a las 5pm", "para las 18:30"). Esto permite programar mensajes solo con texto. También se definen palabras clave que activan la respuesta automática (`WHATSAPP_AUTO_REPLY_KEYWORDS`) y límites de tiempo de seguridad para evitar que el bot se quede respondiendo indefinidamente.

El planificador de tareas (Scheduler) utiliza una lista llamada `SCHEDULED_WHATSAPP_TASKS` que funcionará como una cola de prioridad (heap). Se inicializan un `Lock` y un `Event` de threading. El `Lock` protege la lista de condiciones de carrera (lectura/escritura simultánea), y el `Event` permite que el hilo planificador duerma eficientemente y despierte solo cuando se añade una nueva tarea, ahorrando ciclos de CPU.

Finalmente, se configuran los parámetros acústicos para el reconocimiento de voz. `VOICE_ENERGY_THRESHOLD` define el volumen mínimo para activar la escucha. `VOICE_MODE.ACTIVATE.PATTERNS` y `DEACTIVATE` son conjuntos de frases que el usuario puede decir para cambiar el modo de interacción del agente sin tocar el teclado.

```
1 IMAGE_CAPTION_MODEL = os.getenv("AGENTE_VISION_MODEL", "Salesforce/blip
  -image-captioning-large")
2 IMAGE_CAPTION_ENABLED = os.getenv("AGENTE_VISION_CAPTION", "1") != "0"
3 _IMAGE_CAPTION_PIPELINE: Optional[Any] = None
4 _IMAGE_CAPTION_FAILURE = False
5 _IMAGE_CAPTION_LOCK = threading.Lock()
6
7 WHATSAPP_TIME_PATTERN = re.compile(
8     r"(?:((?:a|para|sobre)\s*las|alas)\s+(?P<hour>\d{1,2})(?:[:h\.](?P<
9     minute>\d{2}))?\s*(?P<meridian>am|pm|a\.m\.|p\.m\.|hrs?|horas?))?",
10    re.IGNORECASE,
11 )
12 WHATSAPP_AUTO_REPLY_KEYWORDS = {
13     "respondele",
14     "respóndele",
15     ....
16 }
17 WHATSAPP_AUTO_REPLY_MIN_SECONDS = int(os.getenv("
  AGENTE_WHATSAPP_AUTO_REPLY_MIN_SECONDS", "30"))
18 WHATSAPP_AUTO_REPLY_MAX_MINUTES = int(os.getenv("
  AGENTE_WHATSAPP_AUTO_REPLY_MAX_MIN", "5"))
19 WHATSAPP_AUTO_REPLY_MAX_SECONDS = max(
20     WHATSAPP_AUTO_REPLY_MIN_SECONDS,
21     WHATSAPP_AUTO_REPLY_MAX_MINUTES * 60,
22 )
```

```
23 WHATSAPP_AUTO_REPLY_POLL_SECONDS = float(os.getenv("
    AGENTE_WHATSAPP_AUTO_REPLY_POLL", "2.0"))
24 WHATSAPP_AUTO_REPLY_DURATION_PATTERN = re.compile(
25     r"(?:por|durante)\s+(?P<value>\d{1,3}|[a-zAÉÍÓÚÑ]+)\s+(?P<unit>min
    (?:uto)?s?|mins?)",
26     re.IGNORECASE,
27 )
28
29 SCHEDULED_WHATSAPP_TASKS: List[Tuple[float, Dict[str, Any]]] = []
30 WHATSAPP_SCHEDULER_LOCK = threading.Lock()
31 WHATSAPP_SCHEDULER_EVENT = threading.Event()
32 WHATSAPP_SCHEDULER_THREAD: Optional[threading.Thread] = None
33 WHATSAPP_SCHEDULE_COUNTER = 0
34 WHATSAPP_TRIGGER_KEYWORDS = {
35     "whatsapp",
36     "what'sapp",
37     ....
38 }
39
40 VOICE_MODE_ACTIVATE_PATTERNS = {
41     "ahora quiero hablarte por voz",
42     "quiero hablarte por voz",
43     ....
44 }
45
46 VOICE_MODE_DEACTIVATE_PATTERNS = {
47     "deten la voz",
48     "deten el modo voz",
49     "apaga la voz",
50     "apaga el modo voz",
51     ....
52 }
53
54 VOICE_LISTEN_TIMEOUT = int(os.getenv("AGENTE_VOICE_TIMEOUT", "10"))
55 VOICE_PHRASE_TIME_LIMIT = int(os.getenv("AGENTE_VOICE_PHRASE_LIMIT", "
    25"))
56 VOICE_MAX_ATTEMPTS = int(os.getenv("AGENTE_VOICE_MAX_ATTEMPTS", "3"))
57 VOICE_FULL_CAL_DURATION = float(os.getenv("AGENTE_VOICE_CAL_DURATION", "
    1.6"))
58 VOICE_QUICK_CAL_DURATION = float(os.getenv("AGENTE_VOICE_CAL_QUICK", "
    0.6"))
59 VOICE_ENERGY_THRESHOLD = float(os.getenv("AGENTE_VOICE_ENERGY", "140"))
60 VOICE_ENERGY_MIN = float(os.getenv("AGENTE_VOICE_ENERGY_MIN", "55"))
61 VOICE_ENERGY_DECAY = float(os.getenv("AGENTE_VOICE_ENERGY_DECAY", "0.78
    "))
```

Listing 6: Visión, Scheduler y Constantes de Voz

2.5.7. Bases de Conocimiento Estáticas e Inicialización de Clientes

El código prosigue definiendo conjuntos de datos estáticos (hardcoded) para respuestas rápidas que no requieren consumo de API de LLM. `FUN_FACT_KEYWORDS` y `FUN_FACT_RESPONSES` habilitan un módulo de trivia sencillo. `CAPABILITY_PHRASES`, `PROJECT_EVAL_PHRASES`, `CREATOR_QUERY_PHRASES` y `GOTY_KEYWORD_PHRASES` son conjuntos de disparadores para metacognición sobre el propio agente, su creador o temas específicos como videojuegos.

Además, se configuran constantes para el parseo de temporizadores y recordatorios (TIMER_DURATION_PATTERN, REMINDER_KEYWORDS). Se mapean palabras numéricas en español (uno, veinte) a enteros, lo cual es esencial para convertir comandos de voz como "pon una alarma en cinco minutos." a valores computables.

También se configuran las constantes para un posible LLM local (como Ollama), definiendo URL base, modelo y tiempos de espera.

El punto crucial de este bloque es la instanciación del cliente OpenAI. Se le pasa la base_url dinámica y la api_key, permitiendo que este único objeto client sirva tanto para conectar con los servidores de OpenAI, OpenRouter o un servidor local de inferencia, dependiendo de las variables de entorno cargadas previamente.

Posteriormente, se inicializa el motor de texto a voz (pyttsx3). Aquí hay una lógica de selección de voz importante: el código itera sobre todas las voces disponibles en el sistema operativo (engine.getProperty("voices")). Inspecciona el nombre, ID y lenguajes de cada voz. Si detecta "spanish", "español" o "es", selecciona esa voz inmediatamente (engine.setProperty). Esto garantiza que el agente hable español correctamente en lugar de usar una voz en inglés por defecto. Si el motor falla al iniciar (común en servidores sin tarjeta de sonido), se captura la excepción y se establece engine = None para evitar caídas del programa. Se definen las funciones speak (que envuelve la llamada a engine.say) y debug_log para salida controlada.

```
1 FUN_FACT_KEYWORDS = {
2     "dato interesante",
3     "dato curioso",
4     "datos interesantes",
5     ....
6 }
7
8 FUN_FACT_RESPONSES = [
9     "el pulpo tiene tres corazones y su sangre es de color azul porque
10    usa cobre en lugar de hierro para transportar oxígeno",
11    "las abejas pueden reconocer rostros humanos y recordarlos, algo
12    que antes se creía exclusivo de los primates",
13 ]
14
15 CAPABILITY_PHRASES = {
16     "que puedes hacer",
17     "qué puedes hacer",
18 }
19
20 CREATOR_QUERY_PHRASES = {
21     "quien te programo",
22     "quien te programó",
23 }
24
25 GOTY_KEYWORD_PHRASES = {
26     "goty",
27     "game of the year",
28 }
29
30 TIMER_DURATION_PATTERN = re.compile(
31     r"(?:en|dentro de|por|durante)\s+(?P<value>\d{1,3}|[a-zAÉÍÓÚÑ]+)\s
32     +(?P<unit>segundos?|mins?|minutos?|horas?)",
33     re.IGNORECASE,
34 )
35
36 REMINDER_KEYWORDS = {
```

```
33     "recordatorio",
34     "recordarme",
35     "recuérdame",
36 }
37
38 CALENDAR_DEFAULT_DURATION_MINUTES = int(os.getenv("
    AGENTE_CALENDAR_DURATION_MIN", "60"))
39 CALENDAR_TIMEZONE = os.getenv("AGENTE_CALENDAR_TZ", "").strip()
40
41 SPANISH_NUMBER_WORDS = {
42     "uno": 1,
43     "una": 1,
44     "un": 1,
45     "dos": 2,
46     "tres": 3,
47     "cuatro": 4,
48     "cinco": 5,
49 }
50
51 SPANISH_WEEKDAY_NAMES = [
52     "lunes",
53     "martes",
54     ....
55 ]
56
57 LOCAL_LLM_PROVIDER = (os.getenv("AGENTE_LOCAL_LLM") or "").strip().
    lower()
58 LOCAL_LLM_MODEL = os.getenv("AGENTE_LOCAL_MODEL", "llama3.2")
59 LOCAL_LLM_BASE_URL = os.getenv("AGENTE_LOCAL_BASE_URL", "http
    ://127.0.0.1:11434").rstrip("/")
60 LOCAL_LLM_TIMEOUT = float(os.getenv("AGENTE_LOCAL_TIMEOUT", "90"))
61 LOCAL_LLM_STRICT = os.getenv("AGENTE_LOCAL_STRICT", "0") == "1"
62
63
64 # Cliente del modelo de lenguaje
65 client = OpenAI(base_url=LLM_BASE_URL, api_key=LLM_API_KEY)
66
67
68 # Inicializar el motor de texto a voz
69 try:
70     engine = pyttsx3.init()
71     voices = engine.getProperty("voices")
72     spanish_voice_id = None
73     for voice in voices:
74         language_tags = []
75         if hasattr(voice, "languages") and voice.languages:
76             language_tags = [tag.decode("utf-8") if isinstance(tag,
77 bytes) else tag for tag in voice.languages]
78             metadata = " ".join([voice.name.lower(), voice.id.lower(), " "
79 join(language_tags).lower()])
80             if "spanish" in metadata or "español" in metadata or "es_" in
81 metadata:
82                 spanish_voice_id = voice.id
83                 break
84     if spanish_voice_id:
85         engine.setProperty("voice", spanish_voice_id)
86     else:
87         print("Advertencia: no se encontró voz en español concreta, se
```



```
    usará la predeterminada.")
85 except Exception as exc:
86     print(f"Error al inicializar el motor de TTS: {exc}")
87     engine = None
88
89
90 def speak(text: str) -> None:
91     if not text:
92         return
93     print(f"Asistente: {text}")
94     if not engine:
95         print("Aviso: motor TTS no disponible.")
96         return
97     try:
98         engine.say(text)
99         engine.runAndWait()
100 except Exception as exc:
101     print(f"Error durante la síntesis de voz: {exc}")
102
103
104 def debug_log(message: str) -> None:
105     if VERBOSE_LOGGING:
106         print(f"Depuración: {message}")
```

Listing 7: Constantes de Conocimiento e Inicialización de Clientes

2.5.8. Carga de Guías de Videojuegos y Normalización

Finalmente, este bloque introduce la función `_load_curated_game_guides`, que se encarga de leer datos estructurados desde el sistema de archivos.

La función inicia definiendo dos diccionarios vacíos: `alias_map` y `canonical_meta`. Primero, verifica la existencia del archivo definido en `CURATED_GUIDES_PATH`. Si no existe, retorna los diccionarios vacíos, evitando errores. Si el archivo existe, intenta abrirlo en modo lectura con codificación UTF-8 y parsearlo como JSON. Cualquier error de decodificación es capturado y logueado. El JSON se espera que sea un diccionario donde las claves son IDs canónicos y los valores son metadatos. El bucle itera sobre este diccionario. Para cada entrada, extrae la lista de pasos (`steps`), alias y prompts de ejemplo. Un punto clave es la normalización: los alias se convierten a minúsculas para facilitar búsquedas insensibles a mayúsculas. Luego, se llena `canonical_meta` con la información estructurada y se llena `alias_map` invirtiendo la relación: cada alias apunta a la lista de pasos. Esto permite que si el usuario busca "GTA 5." "Grand Theft Auto V", ambos términos en el diccionario apunten a la misma guía.

La función auxiliar `_normalize_command_text` implementa una normalización Unicoide NFD. Esto descompone caracteres acentuados (como 'á') en su carácter base ('a') y su modificador. Luego, se codifica a ASCII ignorando errores, lo que efectivamente elimina las tildes. Esto es fundamental para que comandos como "çámaraz çámara" sean tratados como idénticos por el sistema.

```
1 def _load_curated_game_guides() -> Tuple[Dict[str, List[Dict[str, str]
2     ]], Dict[str, Dict[str, Any]]]:
3     alias_map: Dict[str, List[Dict[str, str]]] = {}
4     canonical_meta: Dict[str, Dict[str, Any]] = {}
5     if not CURATED_GUIDES_PATH.exists():
6         return alias_map, canonical_meta
```



```
6     try:
7         with CURATED_GUIDES_PATH.open("r", encoding="utf-8") as fh:
8             payload = json.load(fh)
9     except Exception as exc:
10        debug_log(f"Error cargando guias curadas: {exc}")
11        return alias_map, canonical_meta
12    if not isinstance(payload, dict):
13        return alias_map, canonical_meta
14    for canonical_id, meta in payload.items():
15        if not isinstance(meta, dict):
16            continue
17        steps = meta.get("steps") or []
18        aliases = [alias.lower() for alias in meta.get("aliases", [])]
19        if isinstance(alias, str) and alias.strip():
20            sample_prompts = [prompt for prompt in meta.get("sample_prompts", []) if isinstance(prompt, str) and prompt.strip()]
21            canonical_meta[canonical_id] = {
22                "steps": steps,
23                "aliases": aliases,
24                "sample_prompts": sample_prompts,
25            }
26        for alias in aliases:
27            alias_map[alias] = steps
28    return alias_map, canonical_meta
29
30 def _normalize_command_text(text: str) -> str:
31     normalized = unicodedata.normalize("NFD", text)
32     ascii_text = normalized.encode("ascii", "ignore").decode("ascii")
33     return ascii_text.lower()
```

Listing 8: Carga de Datos y Normalización

Pre-procesamiento de Datos y Estructuras de Memoria

2.5.9. Normalización Masiva de Constantes de Búsqueda

Inmediatamente después de definir la función de normalización `_normalize_command_text` (explicada en la sección anterior), el código procede a ejecutar un bloque masivo de pre-procesamiento. El objetivo de estas líneas es optimizar el rendimiento en tiempo de ejecución.

En lugar de normalizar el texto del usuario y luego normalizar cada palabra clave de la base de datos cada vez que se recibe un mensaje (lo cual sería computacionalmente costoso, $O(N \cdot M)$), el sistema normaliza todas las constantes estáticas **una sola vez** al inicio del programa.

Para esto, se utilizan "set comprehensions" (comprensión de conjuntos) de Python. Por ejemplo, `COOKING_KEYWORDS` se genera iterando sobre `RAW_COOKING_KEYWORDS` y aplicando la función de normalización a cada elemento. Esto asegura que términos como "Sugerencias de comida" se conviertan en "sugerencias de comida" (sin mayúsculas y sin acentos) y se almacenen en un conjunto hash, permitiendo búsquedas $O(1)$.

El mismo proceso se aplica a `RECIPE_RANDOM_PATTERNS`. Para los diccionarios como `RECIPE_FILTER_HINTS`, se utiliza una comprensión de diccionario anidada. Se normaliza

la clave (el disparador, ej: çena”) y también se normaliza cada elemento del conjunto de valores (las etiquetas, ej: {çena”). Esto garantiza que todo el mapa de conocimiento interno esté en el mismo “espacio vectorial” de texto (ASCII minúsculas) que la entrada del usuario procesada.

Este patrón se repite exhaustivamente para las constantes de Hardware (HARDWARE_KEYWORDS, HARDWARE_DESKTOP_KEYWORDS, etc.) y para las constantes de Anime (ANIME_RANDOM_PATTERNS, ANIME_FILTER_HINTS). También se procesa la lista de palabras vacías (stopwords) como HARDWARE_TOKEN_STOPWORDS y ANIME_TOKEN_STOPWORDS, que serán usadas más adelante para filtrar ruido en las consultas del usuario y extraer solo los términos relevantes para la búsqueda.

```
1 COOKING_KEYWORDS = {
2     _normalize_command_text(keyword)
3     for keyword in RAW_COOKING_KEYWORDS
4 }
5 RECIPE_RANDOM_PATTERNS = {
6     _normalize_command_text(pattern)
7     for pattern in RAW_RECIPE_RANDOM_PATTERNS
8 }
9 RECIPE_FILTER_HINTS = {
10     _normalize_command_text(trigger): {
11         _normalize_command_text(tag)
12         for tag in tags
13     }
14     for trigger, tags in RAW_RECIPE_FILTER_HINTS.items()
15 }
16
17 HARDWARE_KEYWORDS = {
18     _normalize_command_text(keyword)
19     for keyword in RAW_HARDWARE_KEYWORDS
20 }
21 HARDWARE_DESKTOP_KEYWORDS = {
22     _normalize_command_text(keyword)
23     for keyword in RAW_HARDWARE_DESKTOP_KEYWORDS
24 }
25 HARDWARE_LAPTOP_KEYWORDS = {
26     _normalize_command_text(keyword)
27     for keyword in RAW_HARDWARE_LAPTOP_KEYWORDS
28 }
29 HARDWARE_RANDOM_PATTERNS = [
30     _normalize_command_text(pattern)
31     for pattern in RAW_HARDWARE_RANDOM_PATTERNS
32 ]
33 HARDWARE_FILTER_HINTS = {
34     _normalize_command_text(trigger): {
35         _normalize_command_text(tag)
36         for tag in tags
37     }
38     for trigger, tags in RAW_HARDWARE_FILTER_HINTS.items()
39 }
40 HARDWARE_TOKEN_STOPWORDS = {
41     "pc",
42     "pcs",
43     "computadora",
44     "computador",
45 }
46 HARDWARE_DETAIL_TRIGGERS = {
```

```
47     _normalize_command_text(trigger)
48     for trigger in RAW_HARDWARE_DETAIL_TRIGGERS
49 }
50 HARDWARE_COMPONENT_KEYWORDS = {
51     slot: {
52         _normalize_command_text(keyword)
53         for keyword in keywords
54     }
55     for slot, keywords in RAW_HARDWARE_COMPONENT_KEYWORDS.items()
56 }
57
58 ANIME_RANDOM_PATTERNS = {
59     _normalize_command_text(pattern)
60     for pattern in RAW_ANIME_RANDOM_PATTERNS
61 }
62 ANIME_FILTER_HINTS = {
63     _normalize_command_text(trigger): {
64         _normalize_command_text(tag)
65         for tag in tags
66     }
67     for trigger, tags in RAW_ANIME_FILTER_HINTS.items()
68 }
69 ANIME_TOKEN_STOPWORDS = {
70     _normalize_command_text(word)
71     for word in RAW_ANIME_TOKEN_STOPWORDS
72 }
73
74 EMOTION_BOOST_KEYWORDS = {
75     _normalize_command_text(keyword)
76     for keyword in RAW_EMOTION_BOOST_KEYWORDS
77 }
78 IDENTITY_QUERY_PATTERNS = {
79     _normalize_command_text(pattern)
80     for pattern in RAW_IDENTITY_QUERY_PATTERNS
81 }
82 COMPLIMENT_REQUEST_PATTERNS = {
83     _normalize_command_text(pattern)
84     for pattern in RAW_COMPLIMENT_REQUEST_PATTERNS
85 }
```

Listing 9: Normalización de Constantes de Búsqueda

2.5.10. Carga e Indexación de Bases de Datos JSON

El siguiente bloque contiene tres funciones fundamentales: `_load_curated_recipes`, `_load_curated_hardware.builds` y `_load_curated_anime_catalog`. Aunque manejan dominios diferentes, comparten una estructura lógica idéntica diseñada para la eficiencia de búsqueda.

La función `_load_curated_recipes` inicializa tres estructuras de datos:

1. `recipe_map`: Un diccionario que mapea un ID único a la totalidad del objeto JSON de la receta.
2. `alias_index`: Un diccionario que mapea nombres alternativos normalizados (ej: "tacos al pastor") al ID único de la receta.

3. `tag_index`: Un `defaultdict(set)` que actúa como un índice invertido. Mapea una etiqueta (ej: "vegana") a un conjunto de IDs de recetas que poseen esa etiqueta.

El flujo es el siguiente: Se verifica si el archivo JSON existe. Se carga en memoria dentro de un bloque `try-except` para robustez. Luego, se itera sobre cada entrada de la lista. Si la entrada es válida y tiene ID, se almacena en `recipe_map`.

Posteriormente, se construyen los índices. Se iteran los `alias`s, se normalizan y se guardan en `alias_index`. Se iteran los `tags`, se normalizan y se agregan al conjunto correspondiente en `tag_index`. Además, campos específicos como `cuisine`, `meal_type`, `diet` y `equipment` se tratan como etiquetas implícitas, normalizándolos y agregándolos al índice de etiquetas. Esto permite que una búsqueda por "mexicana" encuentre recetas donde `cuisine="mexicana"`.

La función `_load_curated_hardware_builds` sigue la misma lógica pero añade un paso de "extracción de características". Al procesar una configuración de PC, inspecciona el texto de los campos `cpu` y `gpu`. Si detecta subcadenas clave como "ryzen", "intel", "tx." "radeon", agrega automáticamente esas etiquetas al `tag_index` de esa build. Esto permite clasificar automáticamente una configuración como "intel" sin que el JSON tenga esa etiqueta explícita.

Finalmente, `_load_curated_anime_catalog` realiza lo mismo para el catálogo de anime, indexando géneros, temas, estudios y años de lanzamiento.

```
1 def _load_curated_recipes() -> Tuple[Dict[str, Dict[str, Any]], Dict[
2     str, str], Dict[str, Set[str]]]:
3     recipe_map: Dict[str, Dict[str, Any]] = {}
4     alias_index: Dict[str, str] = {}
5     tag_index: Dict[str, Set[str]] = defaultdict(set)
6     if not CURATED_RECIPES_PATH.exists():
7         return recipe_map, alias_index, tag_index
8     try:
9         with CURATED_RECIPES_PATH.open("r", encoding="utf-8") as fh:
10             payload = json.load(fh)
11     except Exception as exc:
12         debug_log(f"Error cargando recetario curado: {exc}")
13         return recipe_map, alias_index, tag_index
14     if not isinstance(payload, list):
15         return recipe_map, alias_index, tag_index
16     for entry in payload:
17         if not isinstance(entry, dict):
18             continue
19         recipe_id = entry.get("id")
20         if not recipe_id:
21             continue
22         recipe_map[recipe_id] = entry
23         for alias in entry.get("alias", []):
24             alias_key = _normalize_command_text(alias)
25             if alias_key:
26                 alias_index[alias_key] = recipe_id
27         for tag in entry.get("tags", []):
28             tag_key = _normalize_command_text(tag)
29             if tag_key:
30                 tag_index[tag_key].add(recipe_id)
31         for extra in (entry.get("cuisine"), entry.get("meal_type"),
32             entry.get("diet"), entry.get("equipment")):
33             if extra:
```

```
32         tag_index[_normalize_command_text(str(extra))].add(
recipe_id)
33     return recipe_map, alias_index, tag_index
34
35
36 def _load_curated_hardware_builds() -> Tuple[Dict[str, Dict[str, Any]],
Dict[str, str], Dict[str, Set[str]]]:
37     build_map: Dict[str, Dict[str, Any]] = {}
38     alias_index: Dict[str, str] = {}
39     tag_index: Dict[str, Set[str]] = defaultdict(set)
40     if not CURATED_HARDWARE_PATH.exists():
41         return build_map, alias_index, tag_index
42     try:
43         with CURATED_HARDWARE_PATH.open("r", encoding="utf-8") as fh:
44             payload = json.load(fh)
45     except Exception as exc:
46         debug_log(f"Error cargando builds de hardware: {exc}")
47         return build_map, alias_index, tag_index
48     if not isinstance(payload, list):
49         return build_map, alias_index, tag_index
50     for entry in payload:
51         if not isinstance(entry, dict):
52             continue
53         build_id = entry.get("id")
54         if not build_id:
55             continue
56         build_map[build_id] = entry
57         for alias in entry.get("aliases", []) or []:
58             alias_key = _normalize_command_text(alias)
59             if alias_key:
60                 alias_index[alias_key] = build_id
61         tag_sources: List[str] = []
62         tag_sources.extend(entry.get("tags", []) or [])
63         tag_sources.extend(entry.get("use_cases", []) or [])
64         cpu_text = entry.get("cpu", "")
65         gpu_text = entry.get("gpu", "")
66         for token in (cpu_text, gpu_text):
67             lowered = token.lower()
68             if "ryzen" in lowered:
69                 tag_sources.append("ryzen")
70             if "intel" in lowered:
71                 tag_sources.append("intel")
72             if "core" in lowered:
73                 tag_sources.append("intel_core")
74             if "rtx" in lowered:
75                 tag_sources.append("rtx")
76             if "rx" in lowered:
77                 tag_sources.append("radeon")
78             if "arc" in lowered:
79                 tag_sources.append("intel_arc")
80         build_type = entry.get("type")
81         if build_type:
82             tag_sources.append(build_type)
83         release_year = entry.get("release_window")
84         if isinstance(release_year, int):
85             tag_sources.append(str(release_year))
86         for tag in tag_sources:
87             tag_key = _normalize_command_text(str(tag))
```

```
88         if tag_key:
89             tag_index[tag_key].add(build_id)
90     return build_map, alias_index, tag_index
91
92
93 def _load_curated_anime_catalog() -> Tuple[Dict[str, Dict[str, Any]],
94 Dict[str, str], Dict[str, Set[str]]]:
95     anime_map: Dict[str, Dict[str, Any]] = {}
96     alias_index: Dict[str, str] = {}
97     tag_index: Dict[str, Set[str]] = defaultdict(set)
98     if not CURATED_ANIME_PATH.exists():
99         return anime_map, alias_index, tag_index
100     try:
101         with CURATED_ANIME_PATH.open("r", encoding="utf-8") as fh:
102             payload = json.load(fh)
103     except Exception as exc:
104         debug_log(f"Error cargando catálogo de anime: {exc}")
105         return anime_map, alias_index, tag_index
106     if not isinstance(payload, list):
107         return anime_map, alias_index, tag_index
108     for entry in payload:
109         if not isinstance(entry, dict):
110             continue
111         anime_id = entry.get("id")
112         if not anime_id:
113             continue
114         anime_map[anime_id] = entry
115         for alias in entry.get("alias", []) or []:
116             alias_key = _normalize_command_text(alias)
117             if alias_key:
118                 alias_index[alias_key] = anime_id
119         tagged_fields = [
120             entry.get("tags", []),
121             entry.get("genres", []),
122             entry.get("themes", []),
123             entry.get("mood_tags", []),
124         ]
125         for optional_field in ["studio", "tone", "format", "type", "franchise", "length_tag", "era_tag"]:
126             value = entry.get(optional_field)
127             if value:
128                 tagged_fields.append([value])
129         release_year = entry.get("release_year")
130         if release_year:
131             tagged_fields.append([str(release_year)])
132         for collection in tagged_fields:
133             for tag in collection or []:
134                 tag_key = _normalize_command_text(str(tag))
135                 if tag_key:
136                     tag_index[tag_key].add(anime_id)
137     return anime_map, alias_index, tag_index
```

Listing 10: Carga de Datos: Recetas, Hardware y Anime

2.5.11. Utilidades de Procesamiento de Voz y Fechas

Este segmento define funciones auxiliares para limpiar la entrada de voz. `VOICE_COMMAND_SANITIZE_P` es una expresión regular que elimina cualquier carácter que no sea alfanumérico o espacio en blanco, limpiando la puntuación errática que a veces generan los motores de reconocimiento de voz. `VOICE_COMMAND_FILLER_WORDS` es un conjunto de palabras de "relleno" (como ".oye", "por favor", "miau") que no aportan significado semántico a la instrucción.

La función `_canonical_voice_phrase` toma un texto, lo normaliza (quita acentos, minúsculas) y luego aplica el saneamiento por regex para dejar una cadena limpia. La función `_voice_command_candidate_from_normalized` va un paso más allá: divide el texto en tokens y filtra cualquier palabra que esté en el conjunto de relleno. Esto es vital para que un comando como ".oye Miau, por favor activa la voz" se reduzca a ".activa la voz", que es lo que el sistema puede comparar contra sus patrones predefinidos.

También se incluye `_extract_year_from_text`, que usa una expresión regular para buscar patrones de cuatro dígitos que comiencen con "20" (ej: 2023, 2024) dentro de una cadena. Esto se usa específicamente para responder preguntas sobre premios GOTY (Game of the Year) de años específicos.

Finalmente, se redefinen los conjuntos de patrones de activación de voz (`VOICE_MODE_ACTIVATE_PATTE` etc.), pero esta vez aplicando la función `_canonical_voice_phrase` a cada elemento. Esto optimiza las comparaciones futuras: en lugar de limpiar el comando del usuario y compararlo contra patrones sucios, comparamos comando limpio contra patrones limpios.

```
1 VOICE_COMMAND_SANITIZE_PATTERN = re.compile(r"[^a-z0-9\s]+")
2 VOICE_COMMAND_FILLER_WORDS = {
3     "oye",
4     "por",
5     "favor",
6 }
7
8
9 def _canonical_voice_phrase(text: str) -> str:
10     normalized = _normalize_command_text(text)
11     sanitized = VOICE_COMMAND_SANITIZE_PATTERN.sub(" ", normalized)
12     return " ".join(sanitized.split())
13
14
15 def _voice_command_candidate_from_normalized(normalized_text: str) ->
16     str:
17     sanitized = VOICE_COMMAND_SANITIZE_PATTERN.sub(" ", normalized_text
18 )
19     tokens = [token for token in sanitized.split() if token and token
20 not in VOICE_COMMAND_FILLER_WORDS]
21     return " ".join(tokens)
22
23
24 def _voice_command_candidate_from_text(text: str) -> str:
25     normalized = _normalize_command_text(text)
26     return _voice_command_candidate_from_normalized(normalized)
27
28
29 YEAR_PATTERN = re.compile(r"(20\d{2})")
30
31
32 def _extract_year_from_text(text: str) -> Optional[int]:
```



```
30     match = YEAR_PATTERN.search(text)
31     if match:
32         try:
33             return int(match.group(1))
34         except ValueError:
35             return None
36     return None
37
38
39 VOICE_MODE_ACTIVATE_PATTERNS = {
40     _canonical_voice_phrase(pattern)
41     for pattern in VOICE_MODE_ACTIVATE_PATTERNS
42 }
43 VOICE_MODE_DEACTIVATE_PATTERNS = {
44     _canonical_voice_phrase(pattern)
45     for pattern in VOICE_MODE_DEACTIVATE_PATTERNS
46 }
47 FUN_FACT_KEYWORDS = {
48     _normalize_command_text(keyword)
49     for keyword in FUN_FACT_KEYWORDS
50 }
51 CAPABILITY_PHRASES = {
52     _normalize_command_text(phrase)
53     for phrase in CAPABILITY_PHRASES
54 }
55 PROJECT_EVAL_PHRASES = {
56     _normalize_command_text(phrase)
57     for phrase in PROJECT_EVAL_PHRASES
58 }
59 GOTY_KEYWORDS = {
60     _normalize_command_text(keyword)
61     for keyword in GOTY_KEYWORD_PHRASES
62 }
```

Listing 11: Limpieza de Voz y Fechas

2.5.12. Gestor de Memoria (MemoryManager)

Este bloque implementa la persistencia contextual del agente en tiempo de ejecución. Se define una `dataclass` llamada `MemoryEntry` que estructura un recuerdo individual. Almacena el resumen de la interacción, el texto original del usuario y del asistente, la intención detectada, etiquetas asociadas, una marca de tiempo (`timestamp`) y una puntuación de relevancia (`salience`).

La clase principal `MemoryManager` gestiona una lista de estas entradas. Su constructor acepta un `max_entries` (por defecto 40) para limitar el tamaño del historial y evitar consumo excesivo de memoria RAM. El método `add_entry` crea un nuevo objeto `MemoryEntry`, genera un resumen formateado (truncando los textos si son muy largos para ahorrar tokens en futuras llamadas al LLM) y lo añade a la lista. Si la lista excede el tamaño máximo, se descarta la entrada más antigua (FIFO).

El método más complejo es `recall`. Su objetivo es recuperar recuerdos relevantes basándose en una consulta actual. Primero, extrae palabras clave de la consulta. Luego itera sobre todos los recuerdos almacenados, asignando una puntuación dinámica. La puntuación base es la `salience`. Se suma 1.0 punto por cada palabra clave que coincida en el historial. Se suma un factor infinitesimal basado en el `timestamp` para que,

ante empates de puntuación, se prefiera el recuerdo más reciente. Finalmente, ordena los recuerdos por puntuación descendente y devuelve los resúmenes de los N mejores. Si no encuentra coincidencias semánticas, recurre al método `recent`, que simplemente devuelve las últimas interacciones cronológicas.

El método `last.by_intent` permite buscar hacia atrás en el historial para encontrar la última vez que ocurrió un evento específico (ej: cuándo fue la última vez que hablamos del clima”), iterando la lista en reverso.

```
1 @dataclass
2 class MemoryEntry:
3     summary: str
4     user_text: str
5     assistant_text: str
6     intent: str
7     tags: List[str]
8     timestamp: float
9     salience: float = 1.0
10
11 class MemoryManager:
12     def __init__(self, max_entries: int = 40) -> None:
13         self.max_entries = max_entries
14         self.entries: List[MemoryEntry] = []
15
16     def add_entry(
17         self,
18         user_text: str,
19         assistant_text: str,
20         intent: str,
21         tags: Optional[List[str]] = None,
22         salience: float = 1.0,
23     ) -> None:
24         summary = self._build_summary(user_text, assistant_text, intent)
25
26         entry = MemoryEntry(
27             summary=summary,
28             user_text=user_text,
29             assistant_text=assistant_text,
30             intent=intent,
31             tags=(tags or []),
32             timestamp=time.time(),
33             salience=salience,
34         )
35         self.entries.append(entry)
36         if len(self.entries) > self.max_entries:
37             self.entries = self.entries[-self.max_entries :]
38
39     def recall(self, query: str, limit: int = 3) -> List[str]:
40         if not self.entries:
41             return []
42         keywords = {token for token in re.findall(r"[\wáéíóúñ]+", query)
43                     .lower() if len(token) >= 4}
44         if not keywords:
45             return self.recent(limit)
46         scored: List[Tuple[float, MemoryEntry]] = []
47         for entry in self.entries:
```

```
48         entry.user_text.lower(),
49         entry.assistant_text.lower(),
50         entry.intent.lower(),
51         " ".join(entry.tags).lower(),
52     ]
53 )
54 score = entry.salience
55 for keyword in keywords:
56     if keyword in haystack:
57         score += 1.0
58 # slight preference for recent memories
59 score += entry.timestamp * 1e-12
60 if score > entry.salience:
61     scored.append((score, entry))
62 if not scored:
63     return self.recent(limit)
64 scored.sort(key=lambda item: item[0], reverse=True)
65 return [item[1].summary for item in scored[:limit]]
66
67 def recent(self, limit: int = 3) -> List[str]:
68     if not self.entries:
69         return []
70     return [entry.summary for entry in self.entries[-limit:]][::-1]
71
72 def last_by_intent(self, intent: str) -> Optional[MemoryEntry]:
73     for entry in reversed(self.entries):
74         if entry.intent == intent:
75             return entry
76     return None
77
78 def _build_summary(self, user_text: str, assistant_text: str,
79 intent: str) -> str:
80     cleaned_user = re.sub(r"\s+", " ", user_text).strip()
81     cleaned_assistant = re.sub(r"\s+", " ", assistant_text).strip()
82     return f"Intento {intent}: Usuario dijo '{snippet_user}'.  
Respondí: '{snippet_assistant}'."
```

Listing 12: Sistema de Memoria a Corto Plazo

Automatización de Navegadores y WhatsApp

2.5.13. Utilidades del Sistema y Gestión de Perfiles

Antes de iniciar la automatización compleja, el sistema define funciones utilitarias para interactuar con el entorno del usuario.

La función `launch_browser` es una envoltura robusta sobre la librería estándar `webbrowser`. Primero, verifica la bandera global `BROWSER_AUTOMATION_ENABLED`; si es falsa, la función aborta inmediatamente, respetando la configuración del usuario. Si está habilitada, intenta abrir la URL en una nueva pestaña (`new=2`). Sin embargo, en sistemas Windows, la librería estándar a veces falla silenciosamente o no trae la ventana al frente. Por ello, el código incluye un bloque de recuperación específico para Windows (`os.name == "nt"`). Si `webbrowser.open` retorna `False`, se invoca `os.startfile`, que delega la apertura al sistema operativo directamente, asegurando que la URL se abra con la aplicación predefinida.

La función `_ensure_whatsapp_profile_dir` es crítica para la experiencia de usuario en la automatización de WhatsApp. Selenium, por defecto, inicia una sesión de navegador "limpiada" vez, lo que obligaría al usuario a escanear el código QR en cada ejecución. Para evitar esto, esta función determina una ruta en el directorio de usuario (`~/miau_whatsapp`) y asegura que exista mediante `mkdir(parents=True)`. Esta ruta se pasará luego a Chrome para que guarde allí las cookies, el almacenamiento local y la sesión autenticada, permitiendo la persistencia entre reinicios del agente.

```
1 def launch_browser(url: str, new: int = 2) -> bool:
2     if not BROWSER_AUTOMATION_ENABLED:
3         debug_log(f"Automatización de navegador desactivada. URL
pendiente: {url}")
4         return False
5     try:
6         opened = webbrowser.open(url, new=new)
7         if not opened and os.name == "nt":
8             try:
9                 os.startfile(url) # type: ignore[attr-defined]
10                opened = True
11            except Exception as sub_exc:
12                debug_log(f"startfile falló para {url}: {sub_exc}")
13        return opened
14    except Exception as exc:
15        debug_log(f"Error al abrir navegador para {url}: {exc}")
16        if os.name == "nt":
17            try:
18                os.startfile(url) # type: ignore[attr-defined]
19                return True
20            except Exception as sub_exc:
21                debug_log(f"Intento adicional falló para {url}: {
sub_exc}")
22        return False
23
24
25 def _ensure_whatsapp_profile_dir() -> str:
26     profile_path = Path(WHATSAPP_PROFILE_DIR).expanduser()
27     try:
28         profile_path.mkdir(parents=True, exist_ok=True)
29     except OSError as exc:
30         debug_log(f"No pude crear el perfil persistente de WhatsApp: {
exc}")
31     return str(profile_path)
```

Listing 13: Utilidades de Navegación y Perfiles

2.5.14. Inicialización del Motor de Automatización (Selenium)

La función `_create_whatsapp_driver` es la encargada de instanciar el navegador Chrome controlado por software.

Primero, realiza una verificación de seguridad comprobando `WHATSAPP_AUTOMATION_AVAILABLE`. Si las librerías de Selenium no se importaron al inicio (por falta de instalación), retorna `None` inmediatamente, evitando errores de ejecución. Luego, configura las `ChromeOptions`. Se añaden argumentos para desactivar notificaciones nativas (que podrían bloquear el clic en elementos), desactivar la aceleración por GPU (fuente común de inestabilidad en scripts), y desactivar el "sandbox" (necesario en entornos Linux/Docker). Si la bandera

HEADLESS_MODE está activa, se añade el argumento `--headless=new`, lo que permite que el navegador funcione en segundo plano sin interfaz gráfica visible. Se configura el directorio de datos de usuario apuntando a la ruta generada por `_ensure_whatsapp_profile_dir`, garantizando la persistencia de la sesión.

Finalmente, se utiliza `ChromeDriverManager().install()` dentro de un bloque `try-except`. Esta herramienta descarga automáticamente la versión del driver (`chromedriver`) que coincide con la versión del navegador Chrome instalado en el sistema, eliminando problemas de compatibilidad manual. Con el servicio creado, se instancia `webdriver.Chrome`, se establece un tiempo de espera de carga de página generoso (90 segundos) y se navega a `web.whatsapp.com`.

```
1 def _create_whatsapp_driver() -> Optional[Any]:
2     if not WHATSAPP_AUTOMATION_AVAILABLE:
3         debug_log("WhatsApp automation no disponible: falta Selenium o
4             está desactivada por configuración.")
5         return None
6     assert webdriver is not None and ChromeOptions is not None and
7     Service is not None
8     options = ChromeOptions()
9     options.add_argument("--disable-notifications")
10    options.add_argument("--disable-gpu")
11    options.add_argument("--no-sandbox")
12    options.add_argument("--disable-extensions")
13    options.add_argument("--start-maximized")
14    if HEADLESS_MODE:
15        options.add_argument("--headless=new")
16    profile_dir = _ensure_whatsapp_profile_dir()
17    options.add_argument(f"--user-data-dir={profile_dir}")
18    options.add_argument("--profile-directory=Default")
19    try:
20        assert ChromeDriverManager is not None
21        service = Service(ChromeDriverManager().install())
22        driver = webdriver.Chrome(service=service, options=options)
23        driver.set_page_load_timeout(90)
24        driver.get("https://web.whatsapp.com/")
25        return driver
26    except Exception as exc:
27        debug_log(f"No se pudo iniciar el navegador de WhatsApp: {exc}")
28    return None
```

Listing 14: Configuración del Driver de Selenium

2.5.15. Navegación e Interacción con el DOM

Una vez que el navegador está abierto, el agente necesita "ver" tocar elementos específicos. La función `_wait_for_whatsapp_ready` implementa un bucle de espera activa. Verifica la presencia de un elemento específico en el DOM: un `div` con `role='textbox'` y `contenteditable='true'`. Este elemento corresponde a la barra de búsqueda de chats, y su presencia confirma que la aplicación web ha cargado completamente y que el usuario está autenticado (el código QR ya no está).

La función `_select_whatsapp_chat` automatiza el proceso de encontrar a un contacto. Utiliza `WebDriverWait` para esperar a que la barra de búsqueda sea interactuable. Una vez lista, realiza una secuencia de acciones: hace clic en ella, selecciona todo el texto

existente (Control+A) y lo borra (para limpiar búsquedas previas), escribe el nombre del contacto y espera 1.2 segundos. Esta pausa artificial es crucial porque WhatsApp Web realiza filtrado en tiempo real y tarda un momento en mostrar los resultados. Finalmente, envía la tecla ENTER, lo que abre el chat del primer resultado coincidente. Luego, verifica que el pie de página del chat (la caja donde se escriben los mensajes) esté presente, confirmando que la navegación fue exitosa.

```
1 def _wait_for_whatsapp_ready(driver: Any, timeout: int =
    WHATSAPP_MAX_WAIT) -> bool:
2     if WebDriverWait is None:
3         return False
4     deadline = time.time() + timeout
5     while time.time() < deadline:
6         try:
7             driver.find_element(By.CSS_SELECTOR, "div[role='textbox']["
                contenteditable='true']")
8             return True
9         except Exception:
10            time.sleep(1)
11    return False
12
13
14 def _select_whatsapp_chat(driver: Any, contact: str, timeout: int = 25)
    -> bool:
15     if WebDriverWait is None or EC is None or Keys is None:
16         return False
17     try:
18         search_box = WebDriverWait(driver, timeout).until(
19             EC.element_to_be_clickable((By.CSS_SELECTOR, "div["
                contenteditable='true'] [role='textbox']"))
20         )
21     except TimeoutException:
22         return False
23     try:
24         search_box.click()
25         search_box.send_keys(Keys.CONTROL, "a")
26         search_box.send_keys(Keys.DELETE)
27         search_box.send_keys(contact)
28         time.sleep(1.2)
29         search_box.send_keys(Keys.ENTER)
30     except WebDriverException:
31         return False
32     try:
33         WebDriverWait(driver, timeout).until(
34             EC.presence_of_element_located((By.CSS_SELECTOR, "footer
                div[role='textbox'] [contenteditable='true']"))
35         )
36     except TimeoutException:
37         return False
38     return True
```

Listing 15: Funciones de Espera y Selección de Chat

2.5.16. Lógica de Envío de Mensajes

El envío de mensajes en WhatsApp Web es complejo debido a la ofuscación del código fuente de la página. La función `_trigger_whatsapp_send` intenta realizar el envío median-

te dos estrategias. Primero, intenta enviar simplemente presionando la tecla ENTER dentro de la caja de texto. Esto suele funcionar para mensajes simples de una línea. Si eso falla o no es suficiente, busca el botón de envío en la interfaz. Dado que WhatsApp cambia frecuentemente los selectores CSS de sus botones, la función itera sobre una lista de posibles selectores candidatos (`data-testid='compose-btn-send'`, `aria-label='Enviar'`, etc.). Utiliza `WebDriverWait` para encontrar el primer botón que sea `clickable` ejecuta la acción.

La función `_send_messages_in_chat` orquesta el proceso completo de escritura. Recibe el mensaje y el número de repeticiones. Localiza la caja de texto del chat activo. Limpia su contenido previo. Luego, procesa el mensaje línea por línea: escribe un segmento y, si hay más líneas, inyecta la combinación `SHIFT + ENTER` para crear un salto de línea sin enviar el mensaje. Una vez que todo el texto está escrito, llama a `_trigger_whatsapp_send`. Si se solicitan repeticiones, espera un tiempo aleatorio (definido en `WHATSAPP_MESSAGE_DELAY`) antes de repetir el ciclo, simulando comportamiento humano y evitando bloqueos por spam.

La función `automate_whatsapp_message` actúa como la interfaz pública de alto nivel. Coordina la creación del driver, la espera de carga, la selección del chat y el envío del mensaje, manejando todas las excepciones y asegurándose de cerrar el navegador al finalizar (a menos que `WHATSAPP_KEEP_BROWSER_OPEN` sea verdadero).

```
1 def _trigger_whatsapp_send(driver: Any, input_box: Any) -> bool:
2     if Keys is None:
3         return False
4     try:
5         input_box.send_keys(Keys.ENTER)
6         return True
7     except WebDriverException:
8         pass
9     if WebDriverWait is None or EC is None:
10        return False
11    selectors = [
12        "button[data-testid='compose-btn-send']",
13        "button[aria-label='Enviar']",
14        "button[title='Enviar']",
15        "span[data-icon='send']",
16    ]
17    for selector in selectors:
18        try:
19            send_button = WebDriverWait(driver, 5).until(
20                EC.element_to_be_clickable((By.CSS_SELECTOR, selector))
21            )
22            send_button.click()
23            return True
24        except TimeoutException:
25            continue
26        except WebDriverException:
27            continue
28    return False
29
30
31 def _send_messages_in_chat(driver: Any, message: str, repetitions: int)
32     -> bool:
33     if WebDriverWait is None or EC is None or Keys is None:
34         return False
35     repetitions = max(1, min(repetitions, 25))
```

```
35     for idx in range(repetitions):
36         try:
37             input_box = WebDriverWait(driver, 20).until(
38                 EC.element_to_be_clickable((By.CSS_SELECTOR, "footer
div[role='textbox'][contenteditable='true']"))
39             )
40             input_box.click()
41         except TimeoutException:
42             return False
43         try:
44             input_box.send_keys(Keys.CONTROL, "a")
45             input_box.send_keys(Keys.DELETE)
46             segments = message.split("\n")
47             for seg_index, segment in enumerate(segments):
48                 if segment:
49                     input_box.send_keys(segment)
50                     if seg_index < len(segments) - 1:
51                         input_box.send_keys(Keys.SHIFT, Keys.ENTER)
52             if not _trigger_whatsapp_send(driver, input_box):
53                 return False
54             time.sleep(0.6)
55             if idx < repetitions - 1:
56                 time.sleep(max(0.1, WHATSAPP_MESSAGE_DELAY))
57         except WebDriverException:
58             return False
59     return True
60
61
62 def automate_whatsapp_message(contact: str, message: str, repetitions:
int = 1) -> Tuple[bool, str]:
63     if not contact.strip() or not message.strip():
64         return False, "Necesito el contacto y el mensaje para poder
enviarlo."
65     if not WHATSAPP_AUTOMATION_AVAILABLE:
66         return False, "No puedo controlar WhatsApp Web porque Selenium
no está instalado o está deshabilitado."
67     driver = _create_whatsapp_driver()
68     if driver is None:
69         return False, "No pude iniciar el navegador automatizado para
WhatsApp."
70     try:
71         if not _wait_for_whatsapp_ready(driver):
72             return False, "No pude detectar que hayas iniciado sesión
en WhatsApp Web a tiempo."
73         if not _select_whatsapp_chat(driver, contact):
74             return False, f"No encontré el chat llamado '{contact}'.
Asegúrate de que exista y vuelve a intentarlo."
75         if not _send_messages_in_chat(driver, message, repetitions):
76             return False, "El mensaje no se pudo enviar."
77         plural = "mensaje" if max(1, repetitions) == 1 else "mensajes"
78         return True, f"Listo, envié {max(1, repetitions)} {plural} a {
contact} en WhatsApp."
79     finally:
80         if not WHATSAPP_KEEP_BROWSER_OPEN:
81             try:
82                 time.sleep(1.0)
83                 driver.quit()
84             except Exception:
```


Listing 16: Escritura y Envío de Mensajes

2.5.17. Sistema de Planificación de Tareas (Scheduler)

Para permitir el envío diferido de mensajes (ej: ".envía un mensaje mañana a las 9am"), el agente implementa su propio planificador de tareas basado en hilos y colas de prioridad. `_ensure_whatsapp_scheduler_running` implementa un patrón Singleton para el hilo del planificador. Verifica si la variable global `WHATSAPP_SCHEDULER_THREAD` está viva; si no, inicia un nuevo hilo demonio (`daemon=True`) que ejecuta `_whatsapp_scheduler_loop`. `_schedule_whatsapp_message` registra una nueva tarea. Calcula el `timestamp` de ejecución, genera un ID único y crea un diccionario con los detalles de la tarea. Utiliza `heapq.heappush` dentro de un bloqueo (`WHATSAPP_SCHEDULER_LOCK`) para insertar la tarea en la lista `SCHEDULED_WHATSAPP_TASKS`. Al usar un "heap" (montículo), la tarea con la fecha más próxima siempre está en la posición 0, permitiendo acceso $O(1)$. Finalmente, activa el evento `WHATSAPP_SCHEDULER_EVENT` para despertar al hilo planificador si estaba dormido.

El bucle principal `_whatsapp_scheduler_loop` ejecuta la lógica de espera eficiente. 1. Inspecciona (sin extraer) la tarea más próxima en el heap. 2. Si no hay tareas, espera indefinidamente en `event.wait()`. 3. Si hay tarea, calcula los segundos faltantes (`next_eta - time.time()`). 4. Espera ese tiempo exacto. 5. Al despertar, verifica si se despertó por tiempo cumplido o porque se insertó una nueva tarea prioritaria. 6. Si es tiempo cumplido, extrae la tarea del heap, verifica nuevamente el tiempo (por seguridad) y ejecuta `automate_whatsapp_message`. Este diseño evita el "busy waiting" (bucles infinitos que consumen CPU) y asegura precisión en los tiempos de envío.

```
1 def _ensure_whatsapp_scheduler_running() -> None:
2     global WHATSAPP_SCHEDULER_THREAD
3     if WHATSAPP_SCHEDULER_THREAD and WHATSAPP_SCHEDULER_THREAD.is_alive():
4         return
5     WHATSAPP_SCHEDULER_THREAD = threading.Thread(
6         target=_whatsapp_scheduler_loop,
7         name="WhatsAppScheduler",
8         daemon=True,
9     )
10    WHATSAPP_SCHEDULER_THREAD.start()
11
12
13 def _schedule_whatsapp_message(
14     contact: str,
15     message: str,
16     repetitions: int,
17     send_at: datetime.datetime,
18 ) -> Tuple[str, datetime.datetime]:
19     global WHATSAPP_SCHEDULE_COUNTER
20     effective_dt = max(
21         send_at,
22         datetime.datetime.now() + datetime.timedelta(seconds=
23             WHATSAPP_SCHEDULE_MIN_LEAD_SECONDS),
24     )
25     WHATSAPP_SCHEDULE_COUNTER += 1
```

```
25     task_id = f"whatsapp-{int(effective_dt.timestamp())}-{
WHATSAPP_SCHEDULE_COUNTER}"
26     payload = {
27         "contact": contact,
28         "message": message,
29         "repetitions": max(1, min(repetitions, 25)),
30         "scheduled_for": effective_dt,
31         "task_id": task_id,
32         "requested_at": datetime.datetime.now(),
33     }
34     with WHATSAPP_SCHEDULER_LOCK:
35         heapq.heappush(SCHEDULED_WHATSAPP_TASKS, (effective_dt.
timestamp(), payload))
36         WHATSAPP_SCHEDULER_EVENT.set()
37         _ensure_whatsapp_scheduler_running()
38     return task_id, effective_dt
39
40
41 def _whatsapp_scheduler_loop() -> None:
42     while True:
43         try:
44             with WHATSAPP_SCHEDULER_LOCK:
45                 next_eta = SCHEDULED_WHATSAPP_TASKS[0][0] if
SCHEDULED_WHATSAPP_TASKS else None
46                 if next_eta is None:
47                     WHATSAPP_SCHEDULER_EVENT.wait()
48                     WHATSAPP_SCHEDULER_EVENT.clear()
49                     continue
50                 wait_seconds = max(0.0, next_eta - time.time())
51                 triggered = WHATSAPP_SCHEDULER_EVENT.wait(timeout=
wait_seconds)
52                 if triggered:
53                     WHATSAPP_SCHEDULER_EVENT.clear()
54                     continue
55                 with WHATSAPP_SCHEDULER_LOCK:
56                     if not SCHEDULED_WHATSAPP_TASKS:
57                         continue
58                     eta, payload = heapq.heappop(SCHEDULED_WHATSAPP_TASKS)
59                     now_ts = time.time()
60                     if eta - now_ts > 1.5:
61                         heapq.heappush(SCHEDULED_WHATSAPP_TASKS, (eta,
payload))
62                         continue
63                     success, detail = automate_whatsapp_message(
64                         payload["contact"],
65                         payload["message"],
66                         payload["repetitions"],
67                     )
68                     if success:
69                         debug_log(
70                             f"Mensaje programado enviado a {payload['contact']}
(tarea {payload['task_id']})."
71                         )
72                     else:
73                         debug_log(
74                             "No pude completar una tarea programada de WhatsApp
: "
75                             f"{detail or 'motivo desconocido'} (tarea {payload
```

```
76         ['task_id']))."  
77         )  
78         except Exception as exc:  
79             debug_log(f"El hilo programador de WhatsApp se reiniciará  
por un error: {exc}")  
time.sleep(2.0)
```

Listing 17: Scheduler Asíncrono de WhatsApp

Lógica de Auto-Respuesta y Análisis de Lenguaje Natural

2.5.18. Motor de Auto-Respuesta (Auto-Reply)

La función `auto_reply_whatsapp_chat` implementa un "agente autónomo temporal" dentro de WhatsApp. Su propósito es vigilar una conversación específica durante un tiempo determinado y responder automáticamente a los mensajes entrantes en nombre del usuario.

El flujo de control es el siguiente:

1. **Validación y Configuración:** Se verifica que el contacto no esté vacío y que la duración sea válida, acotándola entre los límites definidos (`WHATSAPP_AUTO_REPLY_MIN_SECONDS` y `MAX_SECONDS`). Se instancia el driver de Selenium.
2. **Inicialización de Estado:** Se utiliza un conjunto (`set`) llamado `processed_ids` para llevar un registro de los IDs únicos de los mensajes que ya han sido respondidos. Esto es fundamental para evitar bucles infinitos donde el bot se responda a sí mismo o responda repetidamente al mismo mensaje antiguo.
3. **Carga de Contexto:** Se abre el chat y se recolecta el historial reciente de mensajes. Se marcan todos los mensajes existentes como "procesados", excepto el último si es un mensaje entrante (de la otra persona), al cual se le podría querer responder de inmediato.
4. **Bucle de Vigilancia:** Se entra en un ciclo `while` que dura hasta que se agota el tiempo (`end_time`). En cada iteración:
 - Espera un tiempo de sondeo (`POLL_SECONDS`).
 - Escanea el DOM nuevamente para obtener el historial actualizado.
 - Filtra los mensajes para encontrar aquellos que sean `incoming` (entrantes) y cuyo ID no esté en `processed_ids`.
 - Si hay mensajes nuevos, invoca a `_build_auto_reply_message` (que consulta al LLM) para generar una respuesta.
 - Envía la respuesta físicamente usando `_send_messages_in_chat`.
 - Agrega el ID del mensaje respondido al conjunto `processed_ids`.

Este diseño permite que el agente mantenga una conversación fluida en tiempo real sin intervención humana.

```
1 def auto_reply_whatsapp_chat(contact: str, duration_seconds: int) ->
  Tuple[bool, str]:
2     if not contact.strip():
3         return False, "Necesito saber a quién responder en WhatsApp."
4     if duration_seconds <= 0:
5         return False, "El intervalo para responder debe ser mayor a
        cero."
6     duration_seconds = max(WHATSAPP_AUTO_REPLY_MIN_SECONDS, min(
        duration_seconds, WHATSAPP_AUTO_REPLY_MAX_SECONDS))
7     if not WHATSAPP_AUTOMATION_AVAILABLE:
8         return False, "No puedo controlar WhatsApp Web porque Selenium
        no está disponible."
9     driver = _create_whatsapp_driver()
10    if driver is None:
11        return False, "No pude abrir WhatsApp Web para hacerme cargo
        del chat."
12    responses_sent = 0
13    processed_ids: Set[str] = set()
14    try:
15        if not _wait_for_whatsapp_ready(driver):
16            return False, "No detecté una sesión activa de WhatsApp Web
        ."
17        if not _select_whatsapp_chat(driver, contact):
18            return False, f"No encontré el chat llamado '{contact}'."
19        history = _collect_recent_whatsapp_messages(driver,
        contact_hint=contact)
20        processed_ids.update(msg["id"] for msg in history if msg.get("
        incoming"))
21        pending_message = None
22        for candidate in reversed(history):
23            if candidate.get("incoming"):
24                pending_message = candidate
25            break
26        if pending_message:
27            processed_ids.discard(pending_message["id"])
28            reply_text = _build_auto_reply_message(contact, history,
        pending_message)
29            if reply_text:
30                if not _send_messages_in_chat(driver, reply_text, 1):
31                    return False, "No pude enviar la primera respuesta
        automática."
32                responses_sent += 1
33                processed_ids.add(pending_message["id"])
34            end_time = time.time() + duration_seconds
35            while time.time() < end_time:
36                time.sleep(max(0.8, WHATSAPP_AUTO_REPLY_POLL_SECONDS))
37                history = _collect_recent_whatsapp_messages(driver,
        contact_hint=contact)
38                if not history:
39                    continue
40                new_incoming = [msg for msg in history if msg.get("incoming
        ") and msg["id"] not in processed_ids]
41                if not new_incoming:
42                    continue
43                for incoming_msg in new_incoming:
44                    reply_text = _build_auto_reply_message(contact, history
        , incoming_msg)
45                if not reply_text:
```

```
46         processed_ids.add(incoming_msg["id"])
47         continue
48         if not _send_messages_in_chat(driver, reply_text, 1):
49             return False, "Falló el envío de una respuesta
automática."
50         responses_sent += 1
51         processed_ids.add(incoming_msg["id"])
52         if responses_sent == 0:
53             return True, f"Estuve pendiente del chat con {contact},
pero no hubo mensajes nuevos en ese lapso."
54         plural = "mensaje" if responses_sent == 1 else "mensajes"
55         return True, f"Respondí automáticamente {responses_sent} {
plural} en el chat con {contact}."
56     finally:
57         if not WHATSAPP_KEEP_BROWSER_OPEN:
58             try:
59                 driver.quit()
60             except Exception:
61                 pass
```

Listing 18: Bucle Principal de Auto-Respuesta

2.5.19. Extracción Heurística de Parámetros

Para interpretar comandos como `.envía un mensaje a Juan que diga Hola 5 veces`, el sistema utiliza un enfoque híbrido. Primero intenta métodos heurísticos (rápidos y deterministas) y, si fallan, recurre a un LLM.

La función `_extract_repetition_count` busca explícitamente números o palabras numéricas (`cinco`, `diez`) seguidas de la palabra `veces`.º `repeticiones`. Utiliza el diccionario `SPANISH_NUMBER_WORDS` definido anteriormente para convertir texto a enteros.

La función `_heuristic_whatsapp_parameters` intenta separar el `**contacto**` del `**mensaje**` usando lógica de cadenas. Primero, busca texto entre comillas (simples, dobles o tipográficas) usando `_strip_matching_quotes`. Si encuentra comillas, asume que el contenido es el mensaje. Si no hay comillas, busca patrones lingüísticos como `"que le diga [mensaje].º "dile [mensaje]"`. Una vez aislado el mensaje, asume que el resto de la oración contiene el nombre del contacto. Busca después de preposiciones como `.º "para"`. Implementa lógica para descartar falsos positivos, como horas (`.º las 5pm`) que podrían confundirse con nombres.

```
1 def _extract_repetition_count(text: str) -> int:
2     repetitions = 1
3     digit_match = re.search(r"(\d{1,2})\s*(?:veces|repeticiones|repetir
|repite)", text, flags=re.IGNORECASE)
4     if digit_match:
5         repetitions = int(digit_match.group(1))
6     else:
7         for word, value in SPANISH_NUMBER_WORDS.items():
8             if re.search(rf"\b{re.escape(word)}\s+(?:veces|repeticiones
)\b", text, flags=re.IGNORECASE):
9                 repetitions = value
10                break
11     return max(1, min(repetitions, 25))
12
13
14 def _strip_matching_quotes(text: str) -> str:
```

```
15     quote_pairs = {
16         "\"": "\"",
17         "’": "’",
18     }
19     text = text.strip()
20     if not text:
21         return text
22     start = text[0]
23     end = text[-1]
24     if start in quote_pairs and quote_pairs[start] == end:
25         return text[1:-1].strip()
26     if end in quote_pairs and quote_pairs[end] == start:
27         return text[1:-1].strip()
28     return text
29
30
31 def _heuristic_whatsapp_parameters(user_text: str) -> Tuple[str, str]:
32     contact = ""
33     message = ""
34     message_span: Optional[Tuple[int, int]] = None
35     if quote_match:
36         message = _strip_matching_quotes(quote_match.group(0))
37         message_span = (quote_match.start(), quote_match.end())
38     if not message:
39         pattern = re.search(r"(?:que\s+(?:le\s+)?(?:diga|digas|dice|
40         dile))\s+(.+)", user_text, flags=re.IGNORECASE)
41         if pattern:
42             message = pattern.group(1).strip()
43             message_span = pattern.span(1)
44     message = _strip_matching_quotes(message.strip(".,!?:;"))
45     search_area = user_text if not message_span else user_text[:
46     message_span[0]]
47     contact_iter = list(
48         re.finditer(
49             r"\b(?:a|para)\b\s+([\wÁÉÍÓÚÑáéíóúñ0-9 ., '-]{2,80})",
50             search_area,
51             flags=re.IGNORECASE,
52         )
53     )
54     for match in reversed(contact_iter):
55         candidate = match.group(1)
56         candidate = re.split(
57             r"\b(?:que|dile|diles|para|con|y|,|\.|durante|por)\b",
58             candidate,
59             1,
60             re.IGNORECASE,
61         )[0]
62         if not trimmed:
63             continue
64         if re.match(
65             r"^(?:las|la)\s+\d{1,2}(?:[:h\.] \d{2})?(?:\s*(?:am|pm|a\.m
66             \.lp\.m\.\.hrs?|horas?))?\b",
67             candidate,
68             flags=re.IGNORECASE,
69         ):
70             continue
71         contact = trimmed
72         break
```

```
70 return contact, message
```

Listing 19: Heurística de Extracción de Parámetros

2.5.20. Extracción Basada en LLM y Coordinación

Cuando la heurística falla (por ejemplo, en frases complejas sin comillas claras), el sistema recurre a `_parse_whatsapp_with_llm`. Esta función construye un prompt de sistema ("System Prompt") muy específico, instruyendo al modelo de lenguaje para que actúe como un extractor de entidades y devuelva ****únicamente JSON válido****. Se le pide que identifique el `contact`, el `message` exacto y el número de `repetitions`. El uso de LLM aquí permite entender contextos ambiguos que las expresiones regulares no pueden capturar.

La función `parse_whatsapp_command` es la coordinadora final. 1. Ejecuta la extracción heurística primero (rápida y barata). 2. Ejecuta la extracción de repeticiones. 3. Si falta el contacto o el mensaje después de la heurística, llama al LLM. 4. Combina los resultados, priorizando la heurística para evitar alucinaciones del modelo, pero rellenando los huecos con la salida del LLM. 5. Limpia los resultados finales de puntuación innecesaria.

```
1 def _parse_whatsapp_with_llm(user_text: str, default_repetitions: int)
2   -> Tuple[str, str, int]:
3     messages = [
4         {
5             "role": "system",
6             "content": (
7                 "Eres un extractor de parámetros para comandos de
8                 WhatsApp. "
9                 "Responde únicamente con JSON válido con la forma {\"
10                contact\": \"\", \"message\": \"\", \"repetitions\": 1}. "
11                "El contacto debe ser el nombre o número del
12                destinatario sin instrucciones adicionales. "
13                "El mensaje es el texto exacto que se debe enviar. "
14                "repetitions debe ser un entero entre 1 y 25. Si el
15                usuario no especifica, usa 1. "
16                "Si algún dato no aparece, deja la cadena vacía en ese
17                campo."
18            ),
19        },
20        {
21            "role": "user",
22            "content": user_text,
23        }
24    ]
25    raw = query_llm(messages).strip()
26    try:
27        data = json.loads(raw)
28        contact = str(data.get("contact", "")).strip()
29        message = _strip_matching_quotes(str(data.get("message", "")).
30        strip())
31        repetitions = data.get("repetitions", default_repetitions)
32        if isinstance(repetitions, str) and repetitions.isdigit():
33            repetitions = int(repetitions)
34        if not isinstance(repetitions, int):
35            repetitions = default_repetitions
36    except Exception:
37        return "", "", default_repetitions
```



```
31     return contact, message, max(1, min(int(repetitions), 25))
32
33
34 def parse_whatsapp_command(user_text: str) -> Tuple[str, str, int]:
35     heur_contact, heur_message = _heuristic_whatsapp_parameters(
36         user_text)
37     repetitions = _extract_repetition_count(user_text)
38     contact = heur_contact
39     message = heur_message
40     llm_contact = llm_message = ""
41     llm_repetitions = repetitions
42     if not contact or not message:
43         llm_contact, llm_message, llm_repetitions =
44         _parse_whatsapp_with_llm(user_text, repetitions)
45     if not contact and llm_contact:
46         contact = llm_contact
47     if not message and llm_message:
48         message = llm_message
49     if llm_repetitions:
50         repetitions = llm_repetitions
51     message = _strip_matching_quotes(message).strip()
52     return contact, message, repetitions
53
54 def _extract_auto_reply_duration_seconds(user_text: str) -> int:
55     match = WHATSAPP_AUTO_REPLY_DURATION_PATTERN.search(user_text.lower())
56     if not match:
57         return 0
58     value = match.group("value").strip().lower()
59     if value.isdigit():
60         minutes = int(value)
61     else:
62         minutes = SPANISH_NUMBER_WORDS.get(value, 0)
63     if minutes <= 0:
64         return 0
65     seconds = minutes * 60
66     seconds = max(WHATSAPP_AUTO_REPLY_MIN_SECONDS, seconds)
67     seconds = min(WHATSAPP_AUTO_REPLY_MAX_SECONDS, seconds)
68     return seconds
```

Listing 20: Parsing Híbrido (Heurística + LLM)

2.5.21. Extracción de Datos del DOM (Scraping) y Generación de Respuesta

La función `_collect_recent_whatsapp_messages` es responsable de leer la pantalla de WhatsApp. En lugar de buscar elementos uno por uno con Selenium (lo cual es lento), inyecta un bloque de JavaScript complejo usando `driver.execute_script`. Este script busca contenedores de mensajes usando múltiples selectores conocidos (`msg-container`, `div[role='row']`). Itera sobre los nodos encontrados y extrae metadatos críticos: el texto visible, el ID interno (`data-id`), y atributos como `data-pre-plain-text` o `aria-label` que indican quién envió el mensaje. De vuelta en Python, se procesa esta lista cruda para determinar si cada mensaje es `incoming` (entrante) o `outgoing` (saliente), basándose en clases CSS como `message-in` o analizando el texto de los metadatos.

Finalmente, `_build_auto_reply_message` utiliza este historial extraído para alimentar al LLM. Construye una transcripción de la conversación reciente. Crea un prompt de

sistema que define la personalidad (.Asistente que responde mensajes...). Envía el último mensaje recibido y el historial al modelo de lenguaje. La respuesta generada se limpia y se devuelve lista para ser escrita en el chat.

```
1 def _collect_recent_whatsapp_messages(  
2     driver: Any,  
3     limit: int = 12,  
4     contact_hint: Optional[str] = None,  
5 ) -> List[Dict[str, Any]]:  
6     script = """  
7     const limit = arguments[0];  
8     const selectors = ["div[data-testid='msg-container']", "div[role='row'  
9         ][data-id]"];  
10    let nodes = [];  
11    for (const selector of selectors) {  
12        const chunk = Array.from(document.querySelectorAll(selector));  
13        if (chunk.length) {  
14            nodes = chunk;  
15            break;  
16        }  
17    }  
18    if (!nodes.length) {  
19        nodes = Array.from(document.querySelectorAll("div[data-id]"));  
20    }  
21    const subset = nodes.slice(-limit);  
22    return subset.map((node) => ({  
23        text: node.innerText || "",  
24        dataId: node.getAttribute("data-id") || "",  
25        aria: node.getAttribute("aria-label") || "",  
26        pre: node.getAttribute("data-pre-plain-text") || "",  
27        classes: node.className || "",  
28    }));  
29    """  
30    try:  
31        raw_messages = driver.execute_script(script, max(1, limit))  
32    except Exception:  
33        return []  
34    if not isinstance(raw_messages, list):  
35        return []  
36    contact_lower = (contact_hint or "").strip().lower()  
37    messages: List[Dict[str, Any]] = []  
38    for idx, item in enumerate(raw_messages[-limit:]):  
39        text = str(item.get("text", "")).strip()  
40        if not text:  
41            continue  
42        data_id = str(item.get("dataId") or "").strip()  
43        aria_hint = str(item.get("aria") or "")  
44        pre_hint = str(item.get("pre") or "")  
45        classes = str(item.get("classes") or "").lower()  
46        msg_id = data_id or f"{pre_hint}|{text}|{idx}"  
47        incoming_flag = "message-in" in classes or "incoming" in  
48        classes  
49        outgoing_flag = "message-out" in classes or "outgoing" in  
50        classes  
51        pre_lower = pre_hint.lower()  
52        aria_lower = aria_hint.lower()  
53        if not incoming_flag and not outgoing_flag:  
54            if contact_lower and f"] {contact_lower}:" in pre_lower:
```

```
52         incoming_flag = True
53         elif contact_lower and f" {contact_lower}:" in pre_lower:
54             incoming_flag = True
55         elif re.search(r"\b(tú|you|yo|i):", pre_lower):
56             outgoing_flag = True
57         elif "te envió" in aria_lower or "te envío" in aria_lower
or "te mand" in aria_lower:
58             incoming_flag = True
59         elif "enviaste" in aria_lower or "you sent" in aria_lower:
60             outgoing_flag = True
61         if not incoming_flag and not outgoing_flag:
62             incoming_flag = True
63         is_incoming = incoming_flag and not outgoing_flag
64         messages.append(
65             {
66                 "id": msg_id,
67                 "text": text,
68                 "incoming": is_incoming,
69                 "meta": pre_hint,
70             }
71         )
72         return messages
73
74
75 def _build_auto_reply_message(
76     contact: str,
77     history: List[Dict[str, Any]],
78     target_message: Dict[str, Any],
79 ) -> str:
80     if not history:
81         return ""
82     transcript_lines: List[str] = []
83     for item in history[-8:]:
84         speaker = contact if item.get("incoming") else "Yo"
85         text = item.get("text") or ""
86         if not text:
87             continue
88         transcript_lines.append(f"{speaker}: {text}")
89     transcript = "\n".join(transcript_lines)
90     guidance = (
91         "Eres un asistente que responde mensajes de WhatsApp en nombre
del usuario. "
92         "Responde con claridad, tono empático y máximo dos frases. Si
falta contexto, pide una aclaración breve."
93     )
94     prompt = (
95         f"Último mensaje de {contact}: {target_message.get('text', '')}
\n"
96         f"Conversación reciente:\n{transcript}\n"
97         "Redacta la siguiente respuesta en español, sin viñetas ni
listas."
98     )
99     reply = query_llm(
100         [
101             {"role": "system", "content": guidance},
102             {"role": "user", "content": prompt},
103         ]
104     ).strip()
```

```
105     if not reply:
106         reply = "De acuerdo, gracias por avisar."
107     return reply
```

Listing 21: Scraping de Mensajes y Prompting para Respuestas

Procesamiento Temporal y Visión Artificial

2.5.22. Detección de Expresiones Temporales en Español

El manejo del tiempo es uno de los aspectos más complejos en los asistentes de voz debido a la ambigüedad del lenguaje natural. La función `_detect_spanish_time_expression` es el motor encargado de convertir frases como "mañana a las cinco." ^o "el jueves a las 3pm." ^{en} objetos `datetime` precisos.

El flujo lógico es secuencial y altamente defensivo:

1. **Extracción con Regex:** Se utiliza `WHATSAPP_TIME_PATTERN` para capturar horas, minutos y meridianos (AM/PM). Si no hay coincidencia, la función retorna `None` inmediatamente.
2. **Normalización Horaria:** Se convierten las horas al formato de 24 horas. Si se detecta "pm" la hora es menor a 12, se suma 12. Si es "12am", se convierte a 0.
3. **Análisis de Contexto (Día Relativo):** El código inspecciona el texto alrededor de la hora detectada (una ventana de 60 caracteres) buscando palabras clave:
 - "pasado mañana": Suma 2 días a la fecha actual.
 - "mañana": Suma 1 día.
 - "hoy": Mantiene la fecha actual.
 - Días de la semana ("lunes", "martes"): Calcula el desplazamiento (`offset`) necesario desde el día actual de la semana (`now.weekday()`) hasta el día objetivo.
4. **Construcción del Datetime:** Combina la fecha base calculada con la hora extraída.
5. **Validación de Futuro:** Si el usuario no especificó un día (ej: ".a las 3"), y la hora resultante ya pasó hoy, el sistema asume inteligentemente que se refiere a "mañanaz" suma un día extra. Además, asegura que haya un tiempo mínimo de ventaja (`WHATSAPP_SCHEDULE_MIN_LEAD_SECONDS`) para que el sistema pueda procesar la tarea.

A continuación, se presentan funciones auxiliares para formatear duraciones. `_format_duration_spanish` convierte segundos brutos en texto legible ("2 horas 5 minutos"). `_extract_timer_duration_seconds` realiza la inversa: busca patrones como "15 minutos." "media hora" y devuelve el total en segundos, manejando casos especiales como "media" (30) o palabras numéricas ("uno", "dos").

```
1 def _describe_schedule_day(target_dt: datetime.datetime) -> str:
2     today = datetime.date.today()
3     delta = (target_dt.date() - today).days
4     if delta == 0:
5         return "hoy"
6     if delta == 1:
7         return "mañana"
8     if delta == -1:
9         return "ayer"
10    weekday_name = SPANISH_WEEKDAY_NAMES[target_dt.weekday()]
11    return f"el {weekday_name} {target_dt.strftime('%d/%m')}"
12
13
14 def _detect_spanish_time_expression(user_text: str) -> Optional[Tuple[
15     datetime.datetime, bool]]:
16     if not user_text:
17         return None
18     lowered = user_text.lower()
19     match = WHATSAPP_TIME_PATTERN.search(lowered)
20     if not match:
21         return None
22     hour = int(match.group("hour") or 0)
23     minute_text = match.group("minute") or "0"
24     minute = int(minute_text)
25     meridian = (match.group("meridian") or "").replace(".", "").strip().lower()
26     if meridian:
27         if "p" in meridian and hour < 12:
28             hour += 12
29         if "a" in meridian and hour == 12:
30             hour = 0
31     if hour > 23 or minute > 59:
32         return None
33     now = datetime.datetime.now()
34     base_date = now.date()
35     explicit_reference = False
36     day_offset: Optional[int] = None
37     context_start = max(0, match.start() - 60)
38     context_end = min(len(lowered), match.end() + 60)
39     context_window = lowered[context_start:context_end]
40     if "pasado mañana" in context_window:
41         day_offset = 2
42         explicit_reference = True
43     elif "mañana" in context_window:
44         day_offset = 1
45         explicit_reference = True
46     elif "hoy" in context_window:
47         day_offset = 0
48         explicit_reference = True
49     if day_offset is None:
50         today_index = now.weekday()
51         for offset in range(0, 7):
52             weekday_name = SPANISH_WEEKDAY_NAMES[(today_index + offset)
53                 % 7]
54             if weekday_name in context_window:
55                 day_offset = offset
56                 explicit_reference = True
57                 break
```

```
56     if day_offset is None:
57         day_offset = 0
58     target_dt = datetime.datetime.combine(
59         base_date + datetime.timedelta(days=day_offset),
60         datetime.time(hour=hour, minute=minute, second=0, microsecond
61         =0),
62     )
63     rolled = False
64     while (target_dt - now).total_seconds() <
65     WHATSAPP_SCHEDULE_MIN_LEAD_SECONDS:
66         target_dt += datetime.timedelta(days=1)
67         rolled = True
68         if explicit_reference and (target_dt - now).total_seconds() >=
69         WHATSAPP_SCHEDULE_MIN_LEAD_SECONDS:
70             break
71         if not explicit_reference:
72             break
73     return target_dt, rolled
74
75 def _format_duration_spanish(seconds: int) -> str:
76     seconds = int(seconds)
77     if seconds < 60:
78         return f"{seconds} segundo{'s' if seconds != 1 else ''}"
79     minutes = seconds / 60
80     if minutes < 60:
81         whole = int(round(minutes))
82         return f"{whole} minuto{'s' if whole != 1 else ''}"
83     hours = minutes / 60
84     whole_hours = int(hours)
85     remaining_minutes = int(round((hours - whole_hours) * 60))
86     if remaining_minutes:
87         return f"{whole_hours}h {remaining_minutes}min"
88     return f"{whole_hours} hora{'s' if whole_hours != 1 else ''}"
89
90 def _extract_timer_duration_seconds(text: str) -> int:
91     text_lower = text.lower()
92     match = TIMER_DURATION_PATTERN.search(text_lower)
93     value: Optional[str] = None
94     unit: Optional[str] = None
95     if match:
96         value = match.group("value")
97         unit = match.group("unit")
98     else:
99         simple_match = re.search(r"(\d{1,3})\s+(segundos?|mins?|minutos
100         ?|horas?)", text_lower, flags=re.IGNORECASE)
101         if simple_match:
102             value = simple_match.group(1)
103             unit = simple_match.group(2)
104         if not value or not unit:
105             return 0
106         unit = unit.lower()
107         if value.isdigit():
108             amount = int(value)
109         else:
110             amount = SPANISH_NUMBER_WORDS.get(value.lower(), 0)
111         if value.lower() in {"media", "medio"} and "hora" in unit:
```

```
110         amount = 30
111         elif value.lower() == "media" and "min" in unit:
112             amount = 30
113         if amount <= 0:
114             return 0
115         if unit.startswith("seg"):
116             seconds = amount
117         elif unit.startswith("hora"):
118             seconds = amount * 3600
119         else:
120             seconds = amount * 60
121         return max(0, seconds)
```

Listing 22: Lógica de Parsing Temporal y Duraciones

2.5.23. Inferencia de Etiquetas y Generación de Enlaces de Calendario

La función `_infer_reminder_label` se encarga de limpiar el ruido de un comando de recordatorio para extraer la tarea real. Por ejemplo, de “recuérdame por favor comprar leche”, debe extraer solo “comprar leche”. Utiliza una lista negra de términos de “activación” (como “pon”, “crea”, “recordatorio”) y los elimina del string original. También maneja casos donde la tarea está citada explícitamente (recuérdame “llamar a mamá”).

La función `build_google_calendar_event_link` genera una URL preformateada para crear eventos en Google Calendar sin necesidad de usar la API de Google (que requiere autenticación OAuth compleja). Calcula las fechas de inicio y fin en formato UTC (Zulú), que es el estándar requerido por Google para la interoperabilidad de zonas horarias. Convierte los objetos `datetime` locales a UTC usando `astimezone`. Codifica los parámetros (título, fechas, descripción) usando `urllib.parse.urlencode`. El resultado es un enlace que, al abrirse, lleva al usuario directamente al formulario de “Crear Evento” con todos los campos llenos.

```
1 def _infer_reminder_label(user_text: str) -> str:
2     if quoted_match:
3         candidate = quoted_match.group(1).strip()
4         if candidate:
5             return candidate
6     removal_terms = [
7         "pon", "ponme", "crea", "creame", "crea me", "haz",
8         "programa", "agenda", "agendar", "agrega", "añade",
9         "recordatorio", "recordarme", "recuérdame", "recuerdame",
10        "alarma", "temporizador", "timer", "avísame", "avisame",
11        "google calendar", "calendar", "calendario", "evento",
12        "evento en", "evento para",
13    ]
14    cleaned = extract_focus_text(user_text, removal_terms)
15    cleaned = cleaned.strip()
16    if not cleaned:
17        cleaned = user_text
18    if "," in cleaned:
19        trailing = cleaned.split(",")[-1].strip()
20        if trailing:
21            cleaned = trailing
22    cleaned = re.sub(r"^(?:para|que|sobre|acerca|de|del|la|el|los|las)\s+", "", cleaned, flags=re.IGNORECASE)
23    return cleaned or "tu recordatorio"
```



```
25
26 def build_google_calendar_event_link(
27     title: str,
28     start_dt: datetime.datetime,
29     duration_minutes: int = CALENDAR_DEFAULT_DURATION_MINUTES,
30     description: Optional[str] = None,
31     location: Optional[str] = None,
32 ) -> str:
33     minimum_minutes = max(15, duration_minutes)
34     local_reference = datetime.datetime.now().astimezone()
35     local_tz = local_reference.tzinfo or datetime.timezone.utc
36
37     if start_dt.tzinfo is None:
38         start_dt = start_dt.replace(tzinfo=local_tz)
39     end_dt = start_dt + datetime.timedelta(minutes=minimum_minutes)
40     if end_dt.tzinfo is None:
41         end_dt = end_dt.replace(tzinfo=local_tz)
42
43     fmt = "%Y%m%dT%H%M%SZ"
44     start_utc = start_dt.astimezone(datetime.timezone.utc)
45     end_utc = end_dt.astimezone(datetime.timezone.utc)
46     params = {
47         "action": "TEMPLATE",
48         "text": title or "Evento",
49         "dates": f"{start_utc.strftime(fmt)}/{end_utc.strftime(fmt)}",
50     }
51     tz_hint = CALENDAR_TIMEZONE or (start_dt.tzinfo.tzname(start_dt) if
52                                     start_dt.tzinfo else "")
53     if tz_hint:
54         params["ctz"] = tz_hint
55     if description:
56         params["details"] = description
57     if location:
58         params["location"] = location
59     return "https://calendar.google.com/calendar/render?" + urllib.
60     parse.urlencode(params, quote_via=urllib.parse.quote)
```

Listing 23: Procesamiento de Recordatorios y Google Calendar

2.5.24. Integración con Modelos Locales (Ollama)

Para usuarios preocupados por la privacidad o sin conexión a internet, el sistema soporta inferencia local a través de Ollama. La función `query_local_llm` construye una petición HTTP POST hacia el endpoint local (usualmente `localhost:11434`). Prepara el payload JSON con el modelo seleccionado (`LOCAL_LLM_MODEL`) y los mensajes. Establece `stream: False` para recibir la respuesta completa de una vez, simplificando el manejo. Procesa la respuesta JSON, que puede variar ligeramente de estructura según la versión de Ollama, buscando el contenido en `message.content` o directamente en `response`. Incluye manejo de errores robusto: si el servidor local no responde o da timeout, la función captura la excepción y retorna una cadena vacía, permitiendo que el sistema principal recurra a otros métodos (como la nube) sin colapsar.

```
1 def local_llm_configured() -> bool:
2     return bool(LOCAL_LLM_PROVIDER and LOCAL_LLM_MODEL)
3
4
```

```
5 def query_local_llm(messages: List[Dict[str, str]]) -> str:
6     if not local_llm_configured():
7         return ""
8     if LOCAL_LLM_PROVIDER != "ollama":
9         debug_log(f"Proveedor local desconocido: {LOCAL_LLM_PROVIDER}")
10        return ""
11    url = f"{LOCAL_LLM_BASE_URL}/api/chat"
12    payload = {
13        "model": LOCAL_LLM_MODEL,
14        "messages": messages,
15        "stream": False,
16    }
17    try:
18        response = requests.post(url, json=payload, timeout=
LOCAL_LLM_TIMEOUT)
19        response.raise_for_status()
20        data = response.json()
21    except Exception as exc:
22        debug_log(f"Modelo local no disponible: {exc}")
23        return ""
24
25    if isinstance(data, dict):
26        message = data.get("message")
27        if isinstance(message, dict):
28            content = message.get("content")
29            if isinstance(content, str) and content.strip():
30                return content.strip()
31        choices = data.get("choices")
32        if isinstance(choices, list):
33            for choice in choices:
34                msg = choice.get("message") if isinstance(choice, dict)
35            else None
36                if isinstance(msg, dict):
37                    content = msg.get("content")
38                    if isinstance(content, str) and content.strip():
39                        return content.strip()
40        content = data.get("content")
41        if isinstance(content, str) and content.strip():
42            return content.strip()
43    return ""
```

Listing 24: Cliente HTTP para LLM Local (Ollama)

2.5.25. Subsistema de Visión Artificial (OpenCV)

Este bloque inicializa las capacidades visuales del agente. Primero, intenta cargar el clasificador en cascada de Haar (`haarcascade_frontalface_default.xml`) incluido en OpenCV. Este archivo pre-entrenado permite la detección rápida de rostros sin necesidad de redes neuronales pesadas. Si la carga falla (archivo faltante), `FACE.CASCADE` se establece en `None`, deshabilitando la detección de rostros pero permitiendo que el resto del sistema funcione.

La función `list_available_cameras` escanea los índices de dispositivos del sistema (del 0 al 4) para encontrar cámaras activas. Utiliza `cv2.VideoCapture` para intentar abrir cada índice; si tiene éxito (`isOpened()`), lo añade a la lista y libera el recurso inmediatamente. Esto permite al usuario seleccionar qué cámara usar si tiene múltiples conectadas.

`get_downloads_directory` es una utilidad multiplataforma para encontrar dónde guardar archivos generados (como documentos o reportes). Intenta localizar las carpetas estándar "Downloads." o "Descargas." en el directorio home del usuario. Si no existen, intenta crearlas o hace fallback al directorio raíz del usuario.

```
1 if cv2 is not None:
2     try:
3         _face_cascade_path = os.path.join(cv2.data.haarcascades, "
4         haarcascade_frontalface_default.xml")
5         FACE_CASCADE = cv2.CascadeClassifier(_face_cascade_path)
6         if FACE_CASCADE.empty():
7             FACE_CASCADE = None
8     except Exception as exc:
9         debug_log(f"No se pudo cargar el clasificador de rostros: {exc}
10        ")
11        FACE_CASCADE = None
12    else:
13        FACE_CASCADE = None
14
15 def list_available_cameras(limit: int = 5) -> List[int]:
16     if cv2 is None:
17         return []
18     detected: List[int] = []
19     backend = cv2.CAP_DSHOW if os.name == "nt" else cv2.CAP_ANY
20     for index in range(limit):
21         cap = cv2.VideoCapture(index, backend)
22         if cap is not None and cap.isOpened():
23             detected.append(index)
24         if cap is not None:
25             cap.release()
26     return detected
27
28 def get_downloads_directory() -> str:
29     home = os.path.expanduser("~")
30     candidates = [
31         os.path.join(home, "Downloads"),
32         os.path.join(home, "Descargas"),
33     ]
34     for candidate in candidates:
35         if os.path.isdir(candidate):
36             return candidate
37     fallback = candidates[0]
38     try:
39         os.makedirs(fallback, exist_ok=True)
40         return fallback
41     except OSError:
42         return home
```

Listing 25: Inicialización de Visión y Detección de Cámaras

2.5.26. Pipeline de Descripción de Imágenes (Image Captioning)

Para entender el contenido semántico de una imagen (un gato sentado en el sofá), el sistema utiliza la librería `transformers`. La función `_get_image_caption_pipeline` implementa el patrón Singleton con seguridad de hilos (thread-safety). Verifica si el pi-

pipeline ya está cargado en la variable global `_IMAGE_CAPTION_PIPELINE`. Si no, adquiere un bloqueo (`_IMAGE_CAPTION_LOCK`) para evitar que dos hilos intenten cargar el modelo simultáneamente (lo que consumiría el doble de RAM). Dentro del bloqueo, inicializa el pipeline `image-to-text` con el modelo especificado. Si la carga falla (por falta de internet o memoria), marca una bandera de fallo (`_IMAGE_CAPTION_FAILURE`) para no volver a intentar en el futuro y ahorrar recursos.

La función `_analyze_frame_snapshot` realiza un análisis matemático de bajo nivel sobre los píxeles de la imagen (array de NumPy). Calcula estadísticas como brillo promedio (media de escala de grises), saturación y tono (espacio de color HSV). Utiliza el algoritmo de Canny para detectar bordes y calcular la densidad de "textura" de la imagen. Implementa una heurística específica para detectar hardware: busca una alta concentración de color verde (típico de placas PCB) y líneas rectas paralelas (usando la Transformada de Hough). Finalmente, utiliza el `FACE_CASCADE` cargado anteriormente para contar cuántos rostros hay en la imagen. Toda esta información numérica se condensa en un diccionario que, más tarde, el LLM usará para "ver" la imagen a través de datos.

```
1 def _get_image_caption_pipeline() -> Optional[Any]:
2     global _IMAGE_CAPTION_PIPELINE, _IMAGE_CAPTION_FAILURE
3     if _IMAGE_CAPTION_FAILURE or not IMAGE_CAPTION_ENABLED or pipeline
4     is None:
5         return None
6     if _IMAGE_CAPTION_PIPELINE is not None:
7         return _IMAGE_CAPTION_PIPELINE
8     with _IMAGE_CAPTION_LOCK:
9         if _IMAGE_CAPTION_PIPELINE is not None:
10            return _IMAGE_CAPTION_PIPELINE
11        try:
12            _IMAGE_CAPTION_PIPELINE = pipeline(
13                "image-to-text",
14                model=IMAGE_CAPTION_MODEL,
15                max_new_tokens=80,
16            )
17        except Exception as exc:
18            debug_log(f"No pude inicializar el modelo de visión ({
19                IMAGE_CAPTION_MODEL}): {exc}")
20            _IMAGE_CAPTION_FAILURE = True
21            _IMAGE_CAPTION_PIPELINE = None
22            return None
23        return _IMAGE_CAPTION_PIPELINE
24
25 def _normalize_caption_output(caption_output: Any) -> Optional[str]:
26     if not caption_output:
27         return None
28     candidate: Any = caption_output
29     if isinstance(caption_output, list) and caption_output:
30         candidate = caption_output[0]
31     if isinstance(candidate, dict):
32         text = candidate.get("generated_text") or candidate.get("text")
33         or candidate.get("caption") or ""
34     else:
35         text = str(candidate)
36     cleaned = text.strip().strip(" .")
37     return cleaned.capitalize() if cleaned else None
```

```
38 def _semantic_caption_from_array(frame) -> Optional[str]:
39     if frame is None or cv2 is None or Image is None:
40         return None
41     captioner = _get_image_caption_pipeline()
42     if captioner is None:
43         return None
44     try:
45         rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
46         pil_image = Image.fromarray(rgb_frame)
47     except Exception as exc:
48         debug_log(f"No pude preparar la imagen para el modelo de visión
: {exc}")
49         return None
50     try:
51         output = captioner(pil_image)
52     except Exception as exc:
53         debug_log(f"Falló la generación de descripción semántica: {exc}
")
54         return None
55     return _normalize_caption_output(output)
56
57
58 def _analyze_frame_snapshot(frame) -> Dict[str, float]:
59     if cv2 is None or np is None:
60         return {}
61     resized = cv2.resize(frame, (640, 360)) if frame.shape[1] > 640
else frame
62     gray = cv2.cvtColor(resized, cv2.COLOR_BGR2GRAY)
63     brightness = float(np.mean(gray))
64     hsv = cv2.cvtColor(resized, cv2.COLOR_BGR2HSV)
65     hue = float(np.mean(hsv[:, :, 0]))
66     saturation = float(np.mean(hsv[:, :, 1]))
67     value = float(np.mean(hsv[:, :, 2]))
68     edges = cv2.Canny(gray, 80, 160)
69     edge_density = float(np.count_nonzero(edges)) / float(edges.size)
70     green_lower = np.array([35, 60, 40], dtype=np.uint8)
71     green_upper = np.array([95, 255, 255], dtype=np.uint8)
72     green_mask = cv2.inRange(hsv, green_lower, green_upper)
73     green_ratio = float(cv2.countNonZero(green_mask)) / float(
green_mask.size)
74     line_count = 0
75     try:
76         lines = cv2.HoughLinesP(edges, 1, np.pi / 180, threshold=60,
minLineLength=35, maxLineGap=8)
77         if lines is not None:
78             line_count = int(len(lines))
79     except Exception:
80         line_count = 0
81     faces = 0
82     if FACE_CASCADE is not None:
83         detections = FACE_CASCADE.detectMultiScale(gray, scaleFactor
=1.2, minNeighbors=5, minSize=(60, 60))
84         faces = len(detections)
85     return {
86         "brightness": brightness,
87         "hue": hue,
88         "saturation": saturation,
89         "value": value,
```

```
90     "edge_density": edge_density,
91     "faces": float(faces),
92     "green_ratio": green_ratio,
93     "line_count": float(line_count),
94 }
```

Listing 26: Análisis Matemático y Semántico de Imágenes

Interpretación Visual y Comunicación con LLM

2.5.27. Generación Heurística de Descripciones de Escena

Una vez que el sistema ha extraído métricas matemáticas de una imagen (brillo, bordes, color dominante), la función `_describe_scene_from_metrics` traduce estos datos a lenguaje natural. Esto es crucial porque el modelo de lenguaje (LLM) no "ve" la imagen directamente; recibe una descripción textual de lo que el sistema de visión "percibe".

La función calcula los promedios de todas las métricas recolectadas (útil cuando se analiza un video, donde hay múltiples cuadros). Luego, aplica umbrales lógicos:

- **Iluminación:** Si el brillo promedio es menor a 60 (en escala 0-255), se describe como "poca iluminación". Si es mayor a 180, "muy iluminado".
- **Color (Matiz/Hue):** Se utiliza el círculo cromático del modelo HSV. Rangos específicos de grados determinan si la escena es "rojiza", "fría" (azul), "verdosa", etc.
- **Textura:** La densidad de bordes (calculada previamente con Canny) determina la complejidad. Menos de 0.05 indica una escena simple/plana; más de 0.18 indica muchos detalles.
- **Hardware:** Se implementa una heurística específica para el dominio técnico del agente. Si hay mucho verde saturado (`green_ratio > 0.18`) y muchas líneas rectas, el sistema infiere la presencia de una "placa madre o circuito". Si hay muchas aristas pero menos verde, sugiere "objeto tecnológico".

Finalmente, combina esta descripción técnica con el "caption" semántico generado por la red neuronal (si está disponible), creando un prompt rico para el LLM: "Percibo un entorno con poca iluminación... también parece: un gato durmiendo".

```
1 def _describe_scene_from_metrics(metrics: List[Dict[str, float]],
2   semantic_caption: Optional[str] = None) -> str:
3     if not metrics:
4         return "No pude analizar la escena con suficiente información."
5     avg_brightness = sum(item["brightness"] for item in metrics) / len(
6         metrics)
7     avg_saturation = sum(item["saturation"] for item in metrics) / len(
8         metrics)
9     avg_hue = sum(item["hue"] for item in metrics) / len(metrics)
10    avg_edges = sum(item["edge_density"] for item in metrics) / len(
11        metrics)
12    max_faces = int(max(item["faces"] for item in metrics))
13    avg_green_ratio = sum(item.get("green_ratio", 0.0) for item in
14        metrics) / len(metrics)
```

```
10     avg_line_count = sum(item.get("line_count", 0.0) for item in
metrics) / len(metrics)
11
12     if avg_brightness < 60:
13         light_desc = "un entorno con poca iluminación"
14     elif avg_brightness > 180:
15         light_desc = "un entorno muy iluminado"
16     else:
17         light_desc = "una iluminación moderada"
18
19     if avg_saturation < 40:
20         color_desc = "con colores neutros"
21     else:
22         if avg_hue < 20 or avg_hue >= 170:
23             color_desc = "con tonos rojizos"
24         elif avg_hue < 45:
25             color_desc = "con matices anaranjados o amarillos"
26         elif avg_hue < 70:
27             color_desc = "con presencia de verdes"
28         elif avg_hue < 130:
29             color_desc = "con predominio de azules"
30         else:
31             color_desc = "con tonos fríos"
32
33     if avg_edges < 0.05:
34         texture_desc = "La escena se ve sencilla, con pocos contornos
definidos."
35     elif avg_edges > 0.18:
36         texture_desc = "Hay bastantes detalles y contornos marcados en
el entorno."
37     else:
38         texture_desc = "El entorno tiene una cantidad moderada de
detalles."
39
40     if max_faces > 1:
41         faces_desc = f"Detecto aproximadamente {max_faces} rostros en
el encuadre."
42     elif max_faces == 1:
43         faces_desc = "Veo un rostro presente en la escena."
44     else:
45         faces_desc = "No logro identificar rostros con claridad."
46
47     subject_notes = ""
48     pcb_hint = avg_green_ratio > 0.18 and avg_edges > 0.07 and
avg_line_count > 20
49     connector_hint = avg_green_ratio > 0.08 and avg_line_count > 15 and
avg_edges > 0.06
50     tech_hint = avg_edges > 0.16 and avg_green_ratio < 0.05
51     if pcb_hint:
52         subject_notes = (
53             " La combinación de verde saturado y muchas líneas rectas
se parece a una placa madre o circuito impreso."
54         )
55     elif connector_hint:
56         subject_notes = " Veo conectores y pistas paralelas, así que
luce como hardware electrónico."
57     elif tech_hint:
58         subject_notes = " Hay bastantes aristas y figuras geométricas,
```



```
parece un objeto tecnológico."
59
60 semantic_notes = ""
61 if semantic_caption:
62     semantic_notes = f" También parece: {semantic_caption}."
63
64 return f"Percibo {light_desc} {color_desc}. {texture_desc} {
    faces_desc}{subject_notes}{semantic_notes}"
```

Listing 27: Traducción de Métricas a Lenguaje Natural

2.5.28. Interacción con Hardware de Cámara y Archivos

La función `open_camera_preview_and_analyze` maneja el ciclo de vida de la cámara. Primero, verifica las dependencias. Selecciona el backend de captura (`CAP_DSHOW` en Windows para mayor rapidez de inicio). Abre la cámara y crea una ventana de visualización llamada "MiauCam". Entra en un bucle `while` que lee frames constantemente. Para optimizar el rendimiento y no saturar el procesador analizando 30 o 60 cuadros por segundo, implementa un muestreo: solo extrae métricas y actualiza el cuadro representativo cada 1.5 segundos o si la lista de métricas es pequeña. El bucle se rompe si el usuario presiona 'q', 'Esc', o si expira la duración predeterminada. Al finalizar, libera la cámara (`cap.release()`) y destruye la ventana. Finalmente, invoca al modelo de descripción semántica (`_semantic_caption_from_array`) sobre el último cuadro representativo y genera el resumen.

La función `analyze_image_file` es la contraparte para archivos estáticos. Valida la existencia del archivo, intenta decodificarlo con OpenCV y ejecuta el mismo pipeline de análisis de métricas y semántica que la cámara en vivo, devolviendo además la resolución de la imagen.

```
1 def open_camera_preview_and_analyze(camera_index: int = 0,
2   preview_duration: float = 8.0) -> Tuple[bool, str]:
3     if cv2 is None or np is None:
4         return False, "Necesito que instales las dependencias de visión
5         (opencv-python y numpy) para usar la cámara."
6     backend = cv2.CAP_DSHOW if os.name == "nt" else cv2.CAP_ANY
7     cap = cv2.VideoCapture(camera_index, backend)
8     if not cap or not cap.isOpened():
9         if cap:
10             cap.release()
11             return False, "No logré acceder a esa cámara."
12     cv2.namedWindow("MiauCam", cv2.WINDOW_NORMAL)
13     cv2.resizeWindow("MiauCam", 800, 450)
14     collected_metrics: List[Dict[str, float]] = []
15     representative_frame = None
16     last_capture = 0.0
17     start_time = time.time()
18     try:
19         while True:
20             ret, frame = cap.read()
21             if not ret:
22                 break
23             cv2.imshow("MiauCam", frame)
24             if len(collected_metrics) < 120:
25                 metrics = _analyze_frame_snapshot(frame)
26                 if metrics:
```

```
25         collected_metrics.append(metrics)
26         now = time.time()
27         if representative_frame is None or now - last_capture >=
1.5:
28             try:
29                 representative_frame = frame.copy()
30             except Exception:
31                 representative_frame = frame
32             last_capture = now
33             key = cv2.waitKey(1) & 0xFF
34             if key in (ord("q"), 27):
35                 break
36             if time.time() - start_time >= preview_duration:
37                 break
38         finally:
39             cap.release()
40             try:
41                 cv2.destroyAllWindows("MiauCam")
42             except Exception:
43                 try:
44                     cv2.destroyAllWindows()
45                 except Exception:
46                     pass
47         semantic_caption = _semantic_caption_from_array(
48         representative_frame) if representative_frame is not None else None
49         description = _describe_scene_from_metrics(collected_metrics,
50         semantic_caption)
51         return True, description
52
53 def analyze_image_file(path: str) -> Tuple[bool, str]:
54     if cv2 is None or np is None:
55         return False, "Necesito opencv-python y numpy instalados para
56         interpretar imágenes adjuntas."
57     if not os.path.exists(path):
58         return False, "No encuentro el archivo seleccionado."
59     image = cv2.imread(path)
60     if image is None:
61         return False, "No pude abrir la imagen, quizá el formato no es
62         compatible."
63     metrics = _analyze_frame_snapshot(image)
64     if not metrics:
65         return False, "No logré extraer detalles de la imagen."
66     semantic_caption = _semantic_caption_from_array(image)
67     description = _describe_scene_from_metrics([metrics],
68     semantic_caption)
69     height, width = image.shape[:2]
70     return True, f"{description} Resolución aproximada: {width}x{height}
71     píxeles."
```

Listing 28: Bucle de Cámara y Procesamiento de Archivos

2.5.29. Orquestador de Consultas LLM (Remote & Local)

La función `query_llm` es el cerebro cognitivo del agente. Su responsabilidad es enviar prompts y recibir respuestas textuales, gestionando la complejidad de múltiples proveedores y posibles fallos de red.

La función acepta una lista de mensajes (historial de chat) y un modelo opcional. Dentro, define una función anidada `invoke_remote` que encapsula la lógica específica de la librería `openai`. Si la URL base pertenece a OpenAI oficial, maneja la respuesta inspeccionando los atributos `output` o `choices` según la versión de la API. Si es un proveedor compatible (como OpenRouter), utiliza la interfaz estándar de `chat.completions.create`.

El sistema implementa una lógica de ****cascada de fallos (fallback)****:

1. **Preferencia Remota:** Si hay una API Key configurada y `LOCAL_LLM_STRICT` es falso, intenta primero con los modelos remotos.
2. **Secuencia de Modelos:** Intenta primero con el modelo principal (`target_model`). Si falla (error 500, timeout), intenta automáticamente con el modelo de respaldo (`LLM_FALLBACK_MODEL`).
3. **Fallback Local:** Si todos los intentos remotos fallan, o si no hay API Key, verifica si hay un modelo local configurado (Ollama). Si es así, llama a `query_local_llm`.
4. **Preferencia Local Estricta:** Si `LOCAL_LLM_STRICT` es verdadero, invierte el orden: intenta primero el local y solo usa el remoto si el local falla.

Esta arquitectura asegura que el agente siga funcionando incluso si se cae internet o si se agotan los créditos de la API, degradando suavemente a un modelo local o reportando errores específicos (como "Insufficient credits") al usuario.

```
1 def query_llm(messages: List[Dict[str, str]], model: Optional[str] =  
  None) -> str:  
2     target_model = model or LLM_MODEL  
3     errors: List[str] = []  
4  
5     def invoke_remote(chosen_model: str) -> str:  
6         try:  
7             if "openai.com" in LLM_BASE_URL:  
8                 response = client.responses.create(  
9                     model=chosen_model,  
10                    input=[{"role": message["role"], "content": message  
11                    ["content"]} for message in messages],  
12                    max_output_tokens=MAX_OUTPUT_TOKENS,  
13                )  
14                chunks: List[str] = []  
15                for item in getattr(response, "output", []) or []:  
16                    if getattr(item, "type", None) != "message":  
17                        continue  
18                    message_block = getattr(item, "message", None)  
19                    if not message_block:  
20                        continue  
21                    for content in getattr(message_block, "content",  
22                    []) or []:  
23                        if getattr(content, "type", None) == "text":  
24                            chunks.append(getattr(content, "text", ""))  
25                return " ".join(chunks).strip() if chunks else ""  
26                completion = client.chat.completions.create(  
27                    model=chosen_model,  
28                    messages=messages,  
29                    max_tokens=MAX_OUTPUT_TOKENS,  
30                )  
31            return completion.choices[0].message.content.strip()
```

```
30         except Exception as exc:
31             error_text = str(exc)
32             errors.append(error_text)
33             debug_log(f"Error en la solicitud al modelo ({chosen_model
34 }): {error_text}")
35             return ""
36
37         remote_models: List[str] = []
38         if LLM_API_KEY:
39             remote_models.append(target_model)
40             if LLM_FALLBACK_MODEL and LLM_FALLBACK_MODEL != target_model:
41                 remote_models.append(LLM_FALLBACK_MODEL)
42
43         def try_remote_sequence() -> str:
44             for selected in remote_models:
45                 reply = invoke_remote(selected)
46                 if reply:
47                     return reply
48             return ""
49
50         remote_first = bool(remote_models) and not LOCAL_LLM_STRICT
51         remote_after_local = bool(remote_models) and LOCAL_LLM_STRICT
52
53         if remote_first:
54             response = try_remote_sequence()
55             if response:
56                 return response
57
58         if local_llm_configured():
59             local_reply = query_local_llm(messages)
60             if local_reply:
61                 return local_reply
62
63         if remote_after_local:
64             response = try_remote_sequence()
65             if response:
66                 return response
67
68         for error_text in errors:
69             if "Insufficient credits" in error_text or "requires more
70 credits" in error_text:
71                 return (
72                     "No tengo crédito disponible en el modelo configurado."
73                     " Revisa tu cuenta de OpenRouter u OpenAI para agregar
74 saldo o proporciona otra clave."
75                 )
76             if "No endpoints found" in error_text and "openrouter" in
77 LLM_BASE_URL and not local_llm_configured():
78                 return (
79                     "No encuentro un endpoint válido para OpenRouter en
80 este momento."
81                     " Verifica tu conexión o habilita el modelo local con
82 Ollama."
83                 )
84
85         if local_llm_configured():
86             return (
87                 "Intenté usar el modelo local pero no logré obtener
```

```
82     respuesta."
      " Asegúrate de que Ollama esté ejecutándose con el modelo
      descargado."
83     )
84     return "Tengo dificultades para conectarme con mi modelo cognitivo
      ahora mismo."
```

Listing 29: Cliente Unificado de Modelos de Lenguaje

Servicios Externos y Motor de Investigación

2.5.30. Utilidades de Limpieza y Gestión de Historial

Antes de realizar consultas a APIs externas, es vital limpiar la entrada del usuario. Un comando como reproduce música de los Beatles”no sirve como consulta de búsqueda cruda; hay que extraer ”los Beatles”.

La función `limit_history` es una utilidad de gestión de memoria para el contexto del LLM. Recibe la lista de mensajes y un tamaño máximo. Si la lista excede el tamaño, devuelve una rebanada (*slice*) con los últimos N elementos (`history[-max_len:]`). Esto evita que el prompt crezca indefinidamente, lo que costaría más dinero y eventualmente excedería la ventana de contexto del modelo.

`extract_focus_text` es una función genérica de limpieza. Recibe el texto original y una lista de palabras clave a eliminar (”keywords”). Construye una expresión regular dinámica uniendo las palabras clave con el operador OR (`|`). Utiliza `re.sub` para borrar estas palabras (ignorando mayúsculas/minúsculas) y luego limpia los espacios sobrantes.

`clean_music_query` es una especialización de la anterior para el dominio musical. Define una lista extensa de patrones de ruido como ”ponme”, ”quiero escuchar”, “.en youtube”, canción”. Itera sobre estos patrones eliminándolos. También elimina preposiciones iniciales como ”de”, ”del”, ”la.” al principio de la frase resultante para dejar solo el nombre del artista o canción.

```
1 def limit_history(history: List[Dict[str, str]], max_len: int) -> List[
  Dict[str, str]]:
2     if len(history) <= max_len:
3         return history
4     return history[-max_len:]
5
6
7 def extract_focus_text(original: str, keywords: List[str]) -> str:
8     pattern = r"|".join([re.escape(word) for word in keywords])
9     cleaned = re.sub(pattern, "", original, flags=re.IGNORECASE)
10    cleaned = re.sub(r"\s+", " ", cleaned).strip()
11    return cleaned
12
13
14 def clean_music_query(text: str) -> str:
15     cleaned = text
16     removal_patterns = [
17         r"\bpon(?:er)?\b",
18         r"\breproduce\b",
19         r"\bponme\b",
20         r"\bplay\b",
21         r"\bescucha[r]? \b",
22         r"\bquiero\s+(?:o[í]r|escuchar)\b",
```

```
23     r"\bsuena\b",
24     r"\ben\s+youtube\b",
25     r"\bm[uú]sica\b",
26     r"\bcanci[oó]n\b",
27     r"\bplaylist\b",
28     r"\bde\s+la\s+can(?:ci[oó]n)?\b",
29     r"\bpuedes\b",
30     r"\bquiero\b",
31 ]
32 for pattern in removal_patterns:
33     cleaned = re.sub(pattern, " ", cleaned, flags=re.IGNORECASE)
34 cleaned = re.sub(r"\bde\b", " ", cleaned, flags=re.IGNORECASE)
35 cleaned = re.sub(r"\s*[-/]\s*", " ", cleaned)
36 cleaned = re.sub(r"\s+", " ", cleaned).strip()
37 cleaned = re.sub(r"^(?:de|del|la|el|los|las)\s+", "", cleaned,
38 flags=re.IGNORECASE)
39 if not cleaned:
40     return text.strip()
41 return cleaned
```

Listing 30: Limpieza de Texto y Gestión de Historial

2.5.31. Consultas a APIs de Entretenimiento (Música y Anime)

Para obtener datos estructurados sobre entretenimiento, el agente consulta APIs públicas gratuitas.

`fetch_music_recommendations` utiliza la API de iTunes Search. Aunque es una API de Apple, es pública y no requiere clave. Construye los parámetros GET: el término de búsqueda, el límite de resultados y el tipo de medio (`music`). Realiza la petición HTTP con un timeout de 10 segundos. Procesa el JSON de respuesta (`payload`). Itera sobre la lista `results` y extrae el nombre de la pista, el artista y la URL de previsualización (`previewUrl`). Si la API de iTunes falla (excepción), la función hace un "fallback inteligente": llama a `search_youtube` (documentada más adelante) para intentar encontrar canciones en YouTube en su lugar.

`fetch_anime_recommendations` consulta la API de Jikan (una interfaz para MyAnimeList). Configura parámetros como `sfw=true` (Safe For Work) para filtrar contenido adulto, y ordena los resultados por puntuación descendente (`order_by="score"`). Del JSON resultante, extrae títulos (prefiriendo el inglés si existe), año, puntuación y URL. Devuelve una lista de cadenas formateadas listas para ser presentadas al usuario.

```
1 def fetch_music_recommendations(query: str, limit: int = 3) -> List[str]:
2     results: List[str] = []
3     try:
4         params = {"term": query, "limit": limit, "media": "music"}
5         response = requests.get("https://itunes.apple.com/search",
6 params=params, timeout=10)
7         response.raise_for_status()
8         payload = response.json()
9         for entry in payload.get("results", []):
10             track = entry.get("trackName")
11             artist = entry.get("artistName")
12             preview = entry.get("previewUrl")
13             if not track or not artist:
14                 continue
```

```
14         snippet = f"{track} de {artist}"
15         if preview:
16             snippet += f" (escucha previa: {preview})"
17         results.append(snippet)
18         if len(results) == limit:
19             break
20     except Exception as exc:
21         debug_log(f"Error recuperando recomendaciones musicales: {exc}")
22 )
23 if results:
24     return results
25 return [entry["summary"] for entry in search_youtube(query, limit=
26 limit, filter_music=True)]
27
28 def fetch_video_recommendations(query: str, limit: int = 3) -> List[str
29 ]:
30     return [entry["summary"] for entry in search_youtube(query, limit=
31 limit)]
32
33 def fetch_anime_recommendations(query: str, limit: int = 4) -> List[str
34 ]:
35     params = {
36         "q": query,
37         "limit": limit,
38         "sfw": "true",
39         "order_by": "score",
40         "sort": "desc",
41     }
42     try:
43         response = requests.get("https://api.jikan.moe/v4/anime",
44 params=params, timeout=10)
45         response.raise_for_status()
46         payload = response.json()
47     except Exception as exc:
48         debug_log(f"Error consultando sugerencias de anime: {exc}")
49         return []
50     results: List[str] = []
51     for item in payload.get("data", []):
52         title = item.get("title") or item.get("title_english")
53         score = item.get("score")
54         year = item.get("year")
55         url = item.get("url")
56         if not title:
57             continue
58         snippet = title
59         if year:
60             snippet += f" ({year})"
61         if score:
62             snippet += f" - puntuación {score:.1f}"
63         if url:
64             snippet += f" - {url}"
65         results.append(snippet)
66         if len(results) == limit:
67             break
68     return results
```

Listing 31: Integración con iTunes y Jikan API

2.5.32. Estrategias de Búsqueda Web (Multi-Provider)

El agente implementa un motor de búsqueda web polimórfico para garantizar resultados incluso si un proveedor falla o bloquea las peticiones.

Se definen funciones privadas para cada proveedor:

- `_fetch_duckduckgo_search`: Usa la librería oficial `duckduckgo_search` (si está instalada). Retorna título, cuerpo y URL.
- `_fetch_duckduckgo_html`: Realiza scraping directo del HTML de DuckDuckGo (versión ligera). Utiliza `BeautifulSoup` para parsear los divs de resultados (`div.result`), extrayendo enlaces y snippets de texto.
- `_fetch_mojeeek_search`: Consulta el motor de búsqueda Mojeek mediante scraping básico de regex sobre el HTML de respuesta. Esto sirve como respaldo si DuckDuckGo bloquea la IP.
- `_fetch_duckduckgo_api`: Consulta la API “Instant Answer” de DuckDuckGo (`api.duckduckgo.com`). Es útil para definiciones directas (`AbstractText`), aunque menos eficaz para búsquedas generales.

Todas estas funciones implementan lógica para ignorar dominios bloqueados (`_is_blocked_search_url`) filtrando sitios de baja calidad definidos en la configuración global.

La función pública `fetch_web_snippets` orquesta la estrategia. Define una lista de tuplas (`nombre, función`). Itera sobre esta lista intentando ejecutar cada proveedor en secuencia. Si uno tiene éxito y devuelve resultados no vacíos, retorna inmediatamente esos datos y detiene la iteración. Si falla (excepción), registra el error y pasa al siguiente proveedor. Esta arquitectura de “Failover” hace al agente muy resistente a caídas de servicios externos.

```
1 def perform_web_search(query: str, limit: int = 3) -> List[str]:
2     snippets = fetch_web_snippets(query, limit)
3     if not snippets:
4         return []
5     return [f"{item['snippet']} - {item['url']}" for item in snippets]
6
7 def _is_blocked_search_url(url: str) -> bool:
8     try:
9         hostname = urllib.parse.urlparse(url).netloc.lower()
10    except ValueError:
11        return False
12    return any(hostname.endswith(domain) for domain in
13               WEB_SEARCH_BLOCKED_DOMAINS)
14
15 def _fetch_duckduckgo_search(query: str, limit: int) -> List[Dict[str,
16 str]]:
17     if DDGS is None:
18         raise RuntimeError("duckduckgo_search no está instalado")
19     results: List[Dict[str, str]] = []
20     with DDGS() as ddgs:
21         for item in ddgs.text(query, region="wt-wt", safesearch="
moderate", timelimit=None):
22             snippet = item.get("body") or item.get("snippet") or item.
get("title")
23             url = item.get("href") or item.get("url")
```

```
22         if not snippet or not url or _is_blocked_search_url(url):
23             continue
24         results.append(
25             {
26                 "title": item.get("title") or snippet[:80],
27                 "snippet": snippet,
28                 "url": url,
29             }
30         )
31         if len(results) >= limit:
32             break
33     return results
34
35 # ... (Omitimos _fetch_duckduckgo_html y _fetch_mojeeek por brevedad,
36 #     siguen logica similar de request+parse) ...
37
38 def fetch_web_snippets(query: str, limit: int = 5) -> List[Dict[str,
39     str]]:
40     providers: List[Tuple[str, Callable[[str, int], List[Dict[str, str]
41     ]]]] = [
42         ("ddg_html", _fetch_duckduckgo_html),
43         ("ddgs", _fetch_duckduckgo_search),
44         ("mojeeek", _fetch_mojeeek_search),
45         ("duckduckgo", _fetch_duckduckgo_api),
46         ("ddg_proxy", _fetch_duckduckgo_proxy),
47     ]
48     errors: List[str] = []
49     for name, provider in providers:
50         try:
51             results = provider(query, limit)
52             if results:
53                 return results[:limit]
54         except Exception as exc:
55             errors.append(f"{name}: {exc}")
56
57 if errors:
58     debug_log("Búsqueda web fallida: " + "; ".join(errors))
59
60 return []
```

Listing 32: Arquitectura de Búsqueda Web Multi-Proveedor

2.5.33. Investigación Aumentada por Recuperación (RAG)

La función `summarize_web_research` es el cerebro de la capacidad investigativa del agente. Implementa un patrón RAG (Retrieval-Augmented Generation) simplificado.

1. **Recuperación:** Llama a `fetch_web_snippets` para obtener información cruda de internet sobre la consulta del usuario.
2. **Enriquecimiento:** Si se detecta que es una consulta de videojuegos, puede inyectar contexto adicional desde las guías curadas localmente (`additional_context`).
3. **Expansión de Consulta:** Si la primera búsqueda da pocos resultados, utiliza una lista de `alt_queries` (consultas alternativas generadas previamente) para buscar sinónimos y agregar más fuentes al contexto.
4. **Construcción de Contexto:** Combina todos los fragmentos (`snippets`) encontrados en un solo bloque de texto, etiquetando cada uno con su URL fuente.
5. **Generación:** Construye un prompt para el LLM. Si `force_steps` es verdadero (común en guías de juegos), instruye al modelo para generar una lista paso a paso. Si no, pide un resumen conciso. En ambos casos, se exige explícitamente al modelo que base su respuesta **únicamente** en

la información recopilada (.evidencia dada”), reduciendo alucinaciones. 6. ****Retorno:**** Devuelve el texto generado por el LLM y la lista de URLs fuente para que el usuario pueda verificar.

```
1 def summarize_web_research(  
2     query: str,  
3     limit: int = 5,  
4     force_steps: bool = False,  
5     alt_queries: Optional[List[str]] = None,  
6     additional_context: Optional[List[Dict[str, str]]] = None,  
7 ) -> Tuple[str, List[str]]:  
8     snippets: List[Dict[str, str]] = []  
9     if limit != 0:  
10         snippets = fetch_web_snippets(query, limit)  
11         seen_urls: Set[str] = {item["url"] for item in snippets if item.get("url")}  
12         desired_snippet_count = max(4 if force_steps else 3, limit or 0)  
13  
14         if additional_context:  
15             for entry in additional_context:  
16                 url = entry.get("url")  
17                 if url and url not in seen_urls:  
18                     snippets.append(entry)  
19                     seen_urls.add(url)  
20  
21         if alt_queries and len(snippets) < desired_snippet_count:  
22             for alt_query in alt_queries:  
23                 if len(snippets) >= desired_snippet_count:  
24                     break  
25                 alt_query = alt_query.strip()  
26                 if not alt_query:  
27                     continue  
28                 extra = fetch_web_snippets(alt_query, limit or 5)  
29                 for entry in extra:  
30                     url = entry.get("url")  
31                     if url and url not in seen_urls:  
32                         snippets.append(entry)  
33                         seen_urls.add(url)  
34                     if len(snippets) >= desired_snippet_count:  
35                         break  
36  
37         if not snippets:  
38             return "", []  
39  
40         max_entries = max(desired_snippet_count, min(len(snippets), 6))  
41         snippets = snippets[:max_entries]  
42  
43         context_lines = []  
44         for idx, item in enumerate(snippets, 1):  
45             context_lines.append(  
46                 f"{idx}. Fuente: {item['url']}\nFragmento: {item['snippet'][:300]}"  
47             )  
48         context_block = "\n\n".join(context_lines)  
49  
50         if force_steps:  
51             user_prompt = (  
52                 f"Consulta del usuario: {query}.\n\nInformación recopilada
```

```
:\n{context_block}\n\n"
53         "Construye una guía paso a paso (3 a 6 pasos) usando solo
estos fragmentos."
54         " Para cada paso, resume la acción concreta y menciona
entre paréntesis una fuente corta como (Fuente 1)."
55         " Cierra con un recordatorio de verificar la versión del
juego si aplica."
56     )
57     else:
58         user_prompt = (
59             f"Consulta: {query}.\n\nInformación recopilada:\n{
context_block}\n\n"
60             "Redacta un resumen conciso (máximo 4 frases) basado ú
nicamente en la evidencia dada."
61             " Si hay incertidumbre, aclárala explícitamente."
62         )
63
64     messages = [
65         {
66             "role": "system",
67             "content": (
68                 "Eres un asistente que sintetiza resultados de
investigación en la web."
69                 " No inventes datos y cita con referencia numérica
entre paréntesis cuando corresponda."
70             ),
71         },
72         {"role": "user", "content": user_prompt},
73     ]
74     summary = query_llm(messages).strip()
75     sources = [item["url"] for item in snippets if item.get("url")]
76     return (summary, sources) if summary else ("", sources)
```

Listing 33: Implementación de RAG para Investigación Web

Búsqueda Multimedia y Navegación Contextual

2.5.34. Motor de Búsqueda de Video (Piped/YouTube)

Para buscar videos sin depender de la API oficial de YouTube (que requiere cuotas limitadas y claves API complejas), el agente utiliza instancias de **Piped**. Piped es un frontend de privacidad para YouTube que expone una API REST pública y gratuita.

La función `search_youtube` implementa una estrategia de alta disponibilidad:

1. Define una lista de instancias (PIPED_INSTANCES) previamente configurada.
2. Itera sobre cada instancia. Si una está caída o lenta (timeout), captura la excepción, registra el error y salta inmediatamente a la siguiente.
3. Realiza una petición GET al endpoint `/api/v1/search`.
4. Filtra los resultados para incluir solo elementos de tipo `stream` (videos), descartando canales o listas de reproducción.
5. Construye manualmente la URL de YouTube (`https://youtu.be/...`) usando el ID extraído.

6. Retorna una lista de diccionarios con título, canal y URL.

La función `open_youtube_video` orquesta la experiencia de usuario. Intenta primero obtener un video específico usando la búsqueda de Piped. Si encuentra uno, intenta abrirlo. Si la búsqueda de API falla, recurre a `fetch_first_youtube_video_id`, que realiza un *scraping* ligero del HTML de la página de resultados de YouTube para extraer el primer ID de video usando expresiones regulares (`videoId": "..."`). Si todo lo anterior falla, como último recurso, abre la página genérica de resultados de búsqueda de YouTube (`open_youtube_search_page`), asegurando que el usuario siempre obtenga *algo* relevante.

```
1 def search_youtube(query: str, limit: int = 3, filter_music: bool =
2   False) -> List[Dict[str, str]]:
3     params = {"q": query, "region": "MX"}
4     if filter_music:
5       params["filter"] = "music_songs"
6     headers = {"Accept": "application/json"}
7     errors: List[str] = []
8     for base_url in PIPED_INSTANCES:
9       try:
10         response = requests.get(f"{base_url}/api/v1/search", params
11         =params, headers=headers, timeout=10)
12         response.raise_for_status()
13         if "application/json" not in (response.headers.get("Content
14         -Type") or ""):
15           errors.append(f"Respuesta no JSON desde {base_url}")
16           continue
17         payload = response.json()
18       except Exception as exc:
19         errors.append(f"{base_url}: {exc}")
20         continue
21     results: List[Dict[str, str]] = []
22     for entry in payload:
23       if entry.get("type") != "stream":
24         continue
25       title = entry.get("title")
26       video_id = entry.get("videoId")
27       if not title or not video_id:
28         continue
29       channel = entry.get("uploaderName")
30       url = f"https://youtu.be/{video_id}"
31       summary = f"{title} ({channel}) - {url}" if channel else f"
32       {title} - {url}"
33       results.append(
34         {
35           "title": title,
36           "channel": channel or "",
37           "video_id": video_id,
38           "url": url,
39           "summary": summary,
40         }
41       )
42       if len(results) == limit:
43         break
44     if results:
45       return results
46     if errors:
```

```
43     debug_log("Error consultando YouTube/Piped: " + "; ".join(
44         errors))
45     return []
46
47 def open_youtube_video(query: str) -> Tuple[bool, Optional[str]]:
48     def _open_with_search_fallback(target_url: str, summary: Optional[
49         str]) -> Tuple[bool, Optional[str]]:
50         opened = launch_browser(target_url)
51         if opened:
52             return True, summary
53         debug_log(f"No pude abrir directamente {target_url}. Intentaré
54         con la búsqueda general de YouTube.")
55         fallback_opened, fallback_summary = open_youtube_search_page(
56             query)
57         if fallback_summary:
58             return fallback_opened, fallback_summary
59         return False, summary
60
61     matches = search_youtube(query, limit=1, filter_music=True)
62     if matches:
63         video = matches[0]
64         return _open_with_search_fallback(video["url"], video["summary"]
65     ])
66     video_id = fetch_first_youtube_video_id(query)
67     if video_id:
68         url = f"https://youtu.be/{video_id}"
69         summary = f"{query} - {url}"
70         return _open_with_search_fallback(url, summary)
71     return open_youtube_search_page(query)
72
73 def open_youtube_search_page(query: str) -> Tuple[bool, Optional[str]]:
74     if not query:
75         return False, None
76     url = "https://www.youtube.com/results?" + urllib.parse.urlencode({
77         "search_query": query})
78     opened = launch_browser(url)
79     description = f"busqué {query} en YouTube - {url}"
80     if not opened and HEADLESS_MODE:
81         return False, description
82     return opened, description
83
84 def fetch_first_youtube_video_id(query: str) -> Optional[str]:
85     if not query:
86         return None
87     params = {"search_query": query}
88     headers = {
89         "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)
90         AppleWebKit/537.36 (KHTML, like Gecko) Chrome/119.0 Safari/537.36",
91         "Accept-Language": "es-ES,es;q=0.9,en;q=0.8",
92     }
93     try:
94         response = requests.get("https://www.youtube.com/results",
95             params=params, headers=headers, timeout=10)
96         response.raise_for_status()
97         match = re.search(r'"videoId":' + "([a-zA-Z0-9_-]{11})", response.
```

```
text)
93     if match:
94         return match.group(1)
95     except Exception as exc:
96         debug_log(f"Error extrayendo video de YouTube: {exc}")
97     return None
```

Listing 34: Búsqueda de Video con Failover y Scraping

2.5.35. Utilidades de Navegación (Maps y Anime)

El agente incluye helpers para construir URLs complejas de servicios populares.

`open_google_maps_route` construye enlaces profundos ("deeplinks") para la navegación GPS. Acepta un destino y un origen opcional. Utiliza la API de URL universal de Google Maps (`googleusercontent.com...`), que tiene la ventaja de abrirse en la aplicación nativa de Maps en móviles o en el navegador en escritorio, sin necesidad de claves API.

`open_animeflv_search` maneja la idiosincrasia del sitio AnimeFLV, que ha cambiado su estructura de URL varias veces. Define una lista de plantillas de URL (`ANIMEFLV_SEARCH_URLS`) y prueba abrirlas secuencialmente.

`open_google_search` es la utilidad básica para búsquedas generales, codificando la consulta de manera segura para URLs.

```
1 def open_google_maps_route(destination: str, origin: Optional[str] =
  None) -> Tuple[bool, str]:
2     if not destination:
3         return False, ""
4     params = {"api": "1", "destination": destination, "hl": "es"}
5     if origin:
6         params["origin"] = origin
7     query = urllib.parse.urlencode(params)
8     url = f"https://www.google.com/maps/dir/{query}"
9     launched = launch_browser(url, new=2)
10    return launched, url
11
12
13 def open_google_search(query: str) -> Optional[str]:
14     if not query:
15         return None
16     url = "https://www.google.com/search?" + urllib.parse.urlencode({"q": query})
17     if HEADLESS_MODE:
18         return url
19     try:
20         success = webbrowser.open(url)
21         return url if success else None
22     except Exception as exc:
23         debug_log(f"Error al abrir búsqueda en Google: {exc}")
24         return None
25
26
27 def open_animeflv_search(query: str) -> Optional[str]:
28     if not query:
29         return None
30     encoded = urllib.parse.quote_plus(query)
31     for template in ANIMEFLV_SEARCH_URLS:
```



```
32     url = template.format(query=encoded)
33     if HEADLESS_MODE:
34         return url
35     try:
36         if webbrowser.open(url):
37             return url
38     except Exception as exc:
39         debug_log(f"Error al abrir AnimeFLV ({url}): {exc}")
40         continue
41     return None
```

Listing 35: Generación de Enlaces para Maps y AnimeFLV

2.5.36. Contexto de Videojuegos y Expansión de Consultas

Este bloque contiene lógica especializada para entender el dominio de los videojuegos, particularmente sagas complejas como Call of Duty Zombies.^o "GTA".

La función `extract_game_context` analiza el texto del usuario para identificar entidades conocidas. 1. Normaliza el texto a minúsculas. 2. Itera sobre conjuntos de conocimientos estáticos: `KNOWN_ZOMBIES_MAPS` (ej: ".origins", "Kino der Toten") y `KNOWN_GAME_SERIES`. 3. Utiliza un diccionario de alias (`GAME_SERIES_ALIASES`) para mapear "bo2.^a" a "black ops 2.^o" "gta v.^a" a "grand theft auto v". 4. Usa expresiones regulares para capturar menciones explícitas como "mapa de [nombre].^o" títulos citados entre comillas. Retorna un diccionario con listas de mapas y juegos detectados.

`build_game_alt_queries` utiliza este contexto para mejorar la búsqueda web (RAG). Si un usuario pregunta "¿cómo prendo la luz en kino?", el motor de búsqueda genérico podría fallar. Esta función: 1. Detecta "kino" como un mapa. 2. Genera variaciones permutadas: - "kino der toten pasos" "kino der toten guía" "kino der toten black ops 3 easter egg" 3. Si detecta palabras de "trucos" (`cheat_mode`), añade sufijos como "cheat codes.^o códigos". Esta expansión de consulta aumenta drásticamente la probabilidad de encontrar guías de calidad en la primera búsqueda.

```
1 def _normalize_term(term: str) -> str:
2     return re.sub(r"\s+", " ", term).strip()
3
4
5 def extract_game_context(text: str) -> Dict[str, List[str]]:
6     lowered = text.lower()
7     contexts: Dict[str, List[str]] = {"maps": [], "games": []}
8
9     def _append_unique(collection: List[str], value: str) -> None:
10         normalized = _normalize_term(value.lower())
11         if normalized and normalized not in collection:
12             collection.append(normalized)
13
14     for known_map in KNOWN_ZOMBIES_MAPS:
15         if known_map in lowered:
16             _append_unique(contexts["maps"], known_map)
17     for known_game in KNOWN_GAME_SERIES:
18         if known_game in lowered:
19             _append_unique(contexts["games"], known_game)
20     for alias, canonical in GAME_SERIES_ALIASES.items():
21         if alias in lowered:
22             _append_unique(contexts["games"], canonical)
23
```

```
24     map_patterns = re.findall(r"(?:mapa|map|mapa de|mapa del|en)\s+([\w\n'\-:]{3,40})", lowered)
25     for match in map_patterns:
26         _append_unique(contexts["maps"], match)
27
28
29     for segment in quoted_segments:
30         cleaned = segment.strip()
31         if cleaned:
32             _append_unique(contexts["maps"], cleaned)
33
34     game_pattern = re.findall(r"call of duty\s+[\w ]+", lowered)
35     for match in game_pattern:
36         _append_unique(contexts["games"], match)
37
38     gta_pattern = re.findall(r"gta\s*(?:[ivx0-9]+|san andreas|vice city|online|liberty city|v|iv|iii)?", lowered)
39     for match in gta_pattern:
40         _append_unique(contexts["games"], match)
41
42     return contexts
43
44
45 def build_game_alt_queries(query: str, text_lower: str) -> List[str]:
46     extras: List[str] = []
47     normalized = _normalize_term(query)
48     if not normalized:
49         return extras
50     seen: Set[str] = {normalized.lower(), text_lower}
51     contexts = extract_game_context(query or text_lower)
52     map_targets = contexts.get("maps") or []
53     game_targets = contexts.get("games") or []
54     cheat_mode = any(keyword in text_lower for keyword in
55 CHEAT_KEYWORDS)
56     suffixes = list(GAME_QUERY_SUFFIXES)
57     if cheat_mode:
58         suffixes += CHEAT_QUERY_SUFFIXES
59
60     def _add_candidate(candidate: str) -> None:
61         lowered = candidate.lower()
62         if lowered not in seen:
63             extras.append(candidate)
64             seen.add(lowered)
65
66     for suffix in suffixes:
67         _add_candidate(f"{normalized} {suffix}".strip())
68
69     for map_name in map_targets:
70         pretty_map = _normalize_term(map_name)
71         for suffix in suffixes:
72             _add_candidate(f"{pretty_map} {suffix}".strip())
73         if game_targets:
74             for game in game_targets:
75                 _add_candidate(f"{pretty_map} {game} guía")
76                 if cheat_mode:
77                     _add_candidate(f"{pretty_map} {game} cheat codes")
78
79     for game in game_targets:
```

```
79     _add_candidate(f"{game} guía")
80     _add_candidate(f"{game} tips")
81     if cheat_mode:
82         _add_candidate(f"{game} cheat codes")
83         _add_candidate(f"{game} lista de trucos")
84
85     for key in CURATED_GAME_GUIDES.keys():
86         if key in text_lower:
87             _add_candidate(f"{key} steps")
88
89     if cheat_mode and "helic" in text_lower:
90         _add_candidate(f"{normalized} helicopter cheat code")
91         _add_candidate(f"{normalized} helicóptero código de truco")
92
93     return extras[:8]
```

Listing 36: Extracción de Entidades de Juegos y Query Expansion

2.5.37. Detección de Guías Curadas (Conocimiento Estático)

Para ciertos juegos muy populares (como "Black Ops 2 Origins"), el agente tiene guías verificadas "hardcodeadas." cargadas desde JSON local, evitando buscar en la web información que ya "sabe" que es correcta.

`detect_curated_game_entries` verifica si la consulta del usuario coincide con alguna clave en `CURATED_GAME_GUIDES`. `format_curated_game_guide` toma esos datos estructurados (pasos, títulos, fuentes) y los formatea en un string legible con viñetas numéricas, listo para ser leído por el TTS o mostrado en el chat.

```
1 def needs_step_by_step_response(text_lower: str) -> bool:
2     triggers = [
3         "paso", "pasos", "cómo", "como ", "como puedo", "como hacer", "
    guía", "guia",
4         "tutorial", "instrucciones", "instrucción", "instruccion", "
    easter egg", "walkthrough", "resolver",
5         "haz", "hacer", "cómo consigo", "como consigo", "explica", "
    monta", "desbloquear",
6     ]
7     return any(trigger in text_lower for trigger in triggers)
8
9
10 def is_video_game_query(text_lower: str) -> bool:
11     return any(keyword in text_lower for keyword in VIDEO_GAME_KEYWORDS
12 )
13
14 def detect_curated_game_entries(text_lower: str) -> Optional[List[Dict[
    str, str]]]:
15     for key, entries in CURATED_GAME_GUIDES.items():
16         if key in text_lower:
17             return entries
18         tokens = [token for token in key.split() if token]
19         if tokens and all(token in text_lower for token in tokens):
20             return entries
21     return None
22
23
```

```
24 def format_curated_game_guide(query: str, entries: List[Dict[str, str  
    ]]) -> str:  
25     focus = query.strip() or "este reto"  
26     lines = [f"Guía verificada para {focus}:"]  
27     for idx, entry in enumerate(entries, 1):  
28         title = entry.get("title") or f"Paso {idx}"  
29         snippet = entry.get("snippet") or ""  
30         url = entry.get("url")  
31         step = f"{idx}. {title}. {snippet}".strip()  
32         if url:  
33             step += f" Fuente: {url}"  
34         lines.append(step)  
35     lines.append("Recuerda verificar la versión del juego o parches  
    recientes antes de intentarlo.")  
36     return "\n".join(lines)
```

Listing 37: Formateo de Guías Locales

Motores de Recomendación: Cocina y Hardware

2.5.38. Motor de Búsqueda de Recetas (Algoritmo Ponderado)

El sistema de recetas no utiliza una base de datos SQL, sino estructuras de datos en memoria optimizadas (diccionarios y conjuntos) cargadas previamente.

2.5.39. Extracción de Señales y Filtros

La función `_extract_recipe_tokens` actúa como un tokenizador ligero. Utiliza una expresión regular para extraer palabras alfanuméricas de al menos 3 caracteres. Esto elimina preposiciones cortas y ruido, dejando solo sustantivos y verbos significativos para la búsqueda.

`_infer_recipe_filters` es el traductor de intenciones. Recorre el diccionario global `RECIPE_FILTER_HINTS` (definido en la Parte 1). Si encuentra una palabra clave en el texto del usuario (ej: `çena`), añade las etiquetas normalizadas correspondientes (ej: `{çena, "ligera"}`) al conjunto de filtros activos. Esto permite que el sistema entienda sinónimos y contextos sin necesidad de IA.

`_match_recipe_aliases` busca coincidencias directas o difusas de nombres de platos. Itera sobre el índice de alias. Si el nombre exacto está en el texto, es un "match" directo. Si no, descompone el alias en tokens y verifica si *todos* los tokens significativos están presentes en la entrada del usuario (lógica AND). Esto permite detectar "pollo al horno" incluso si el usuario escribió "quiero cocinar un pollo rico al horno".

```
1 def _extract_recipe_tokens(normalized_text: str) -> List[str]:  
2     return [token for token in re.findall(r"[a-z0-9]+", normalized_text  
    ) if len(token) >= 3]  
3  
4  
5 def _infer_recipe_filters(normalized_text: str) -> Set[str]:  
6     filters: Set[str] = set()  
7     for needle, mapped in RECIPE_FILTER_HINTS.items():  
8         if needle and needle in normalized_text:  
9             filters.update(mapped)  
10    return filters  
11
```

```
12
13 def _match_recipe_alias(es(normalized_text: str) -> List[str]:
14     hits: List[str] = []
15     normalized_tokens = set(_extract_recipe_tokens(normalized_text))
16     for alias, recipe_id in CURATED_RECIPE_ALIASES.items():
17         if not alias:
18             continue
19         if alias in normalized_text:
20             hits.append(recipe_id)
21             continue
22         alias_tokens = [token for token in alias.split() if len(token)
23 >= 3]
24         if alias_tokens and all(token in normalized_tokens for token in
25 alias_tokens):
26             hits.append(recipe_id)
27     return hits
```

Listing 38: Tokenización y Detección de Filtros de Cocina

2.5.40. Algoritmo de Búsqueda y Ranking (Lookup)

La función `lookup_curated_recipes` implementa un motor de búsqueda híbrido.

1. **Prioridad 1: Coincidencia por Alias.** Primero llama a `_match_recipe_alias(es)`. Si encuentra recetas por nombre, las devuelve inmediatamente, asumiendo que es la intención más fuerte.
2. **Prioridad 2: Búsqueda Vectorial Simplificada.** Si no hay alias, utiliza un sistema de puntuación con Counter.
 - Extrae tokens del usuario, ignorando "stopwords" como "receta." o "cocinar".
 - Por cada token que coincida con una etiqueta en `CURATED_RECIPE_TAG_INDEX`, suma **2 puntos** a la receta asociada.
 - Por cada filtro detectado (ej: "vegana"), suma **3 puntos**. Esto da más peso a las restricciones dietéticas que a los ingredientes aleatorios.
3. **Ranking:** Ordena los resultados por puntuación descendente (`most_common`) y devuelve los top N.

`pick_random_recipes` es una utilidad para el "descubrimiento". Si se proporcionan filtros, calcula la intersección de los conjuntos de IDs que cumplen esos filtros y elige aleatoriamente de ese subconjunto. Si no hay candidatos, devuelve una lista vacía.

```
1 def lookup_curated_recipes(query: str, filters: Optional[Set[str]] =
2     None, limit: int = 3) -> List[Dict[str, Any]]:
3     if not CURATED_RECIPES or limit <= 0:
4         return []
5     normalized = _normalize_command_text(query)
6     filter_set = {tag for tag in (filters or set()) if tag}
7     seen: Set[str] = set()
8     matches: List[Dict[str, Any]] = []
9
10    for recipe_id in _match_recipe_alias(es(normalized):
11        if recipe_id not in seen:
12            recipe = CURATED_RECIPES.get(recipe_id)
```

```
12         if recipe:
13             matches.append(recipe)
14             seen.add(recipe_id)
15             if len(matches) >= limit:
16                 return matches
17
18     tokens = [token for token in _extract_recipe_tokens(normalized) if
19 token not in {"receta", "recetas", "cocina", "cocinar"}]
20     candidate_scores: Counter[str] = Counter()
21     for token in tokens[:20]:
22         for recipe_id in CURATED_RECIPE_TAG_INDEX.get(token, set()):
23             candidate_scores[recipe_id] += 2
24     for filter_tag in filter_set:
25         for recipe_id in CURATED_RECIPE_TAG_INDEX.get(filter_tag, set(
26 )):
27             candidate_scores[recipe_id] += 3
28
29     for recipe_id, _score in candidate_scores.most_common(limit * 2):
30         if recipe_id not in seen:
31             recipe = CURATED_RECIPES.get(recipe_id)
32             if recipe:
33                 matches.append(recipe)
34                 seen.add(recipe_id)
35                 if len(matches) >= limit:
36                     break
37     return matches
38
39 def pick_random_recipes(count: int, filters: Optional[Set[str]] = None,
40 rng: Optional[random.Random] = None) -> List[Dict[str, Any]]:
41     if not CURATED_RECIPES or count <= 0:
42         return []
43     rng = rng or random
44     candidate_ids: Set[str] = set()
45     if filters:
46         for filter_tag in filters:
47             candidate_ids.update(CURATED_RECIPE_TAG_INDEX.get(
48 filter_tag, set()))
49     if not candidate_ids:
50         candidate_ids = set(CURATED_RECIPES.keys())
51     pool = list(candidate_ids)
52     if not pool:
53         return []
54     sample_size = min(len(pool), max(1, count))
55     chosen = rng.sample(pool, sample_size)
56     return [CURATED_RECIPES[recipe_id] for recipe_id in chosen if
57 recipe_id in CURATED_RECIPES]
```

Listing 39: Búsqueda Ponderada de Recetas

2.5.41. Formateo de Respuesta (Recetas)

Una vez seleccionada la receta, el sistema debe presentarla al usuario. En lugar de volcar el JSON crudo, `format_recipe_detail` construye una narrativa legible. Verifica la existencia de campos opcionales (tiempo, porciones, dificultad). Concatena etiquetas clave (cocina, dieta) para dar contexto. Resume los ingredientes mostrando solo los primeros

4 para no saturar el chat, invitando a preguntar por más detalles. Enumera los pasos de preparación. Añade un consejo final ("tip") si existe en la base de datos.

`format_recipe_brainstorm` se usa cuando el usuario pide ideas generales. En lugar de detallar una sola receta, lista varias opciones con sus características principales (tiempo, tipo de dieta), fomentando la exploración.

```
1 def format_recipe_detail(recipe: Dict[str, Any], dataset_size: int) ->
  str:
2     name = recipe.get("name", "receta curada")
3     time_minutes = recipe.get("time_minutes")
4     difficulty = recipe.get("difficulty", "media")
5     servings = recipe.get("servings")
6     stats_parts = []
7     if isinstance(time_minutes, int):
8         stats_parts.append(f"Tiempo estimado: {time_minutes} min")
9     else:
10        stats_parts.append("Tiempo estimado: variable")
11    stats_parts.append(f"Dificultad {difficulty}")
12    if servings:
13        stats_parts.append(f"Rinde {servings} porciones")
14    signature_tags = [recipe.get("cuisine"), recipe.get("diet"), recipe
        .get("meal_type"), recipe.get("equipment")]
15    readable_tags = ", ".join(tag for tag in signature_tags if tag)
16    ingredients = recipe.get("ingredients") or []
17    ingredient_line = ""
18    if ingredients:
19        preview = "; ".join(ingredients[:4])
20        if len(ingredients) > 4:
21            preview += "..."
22        ingredient_line = f"Ingredientes base: {preview}."
23    steps = recipe.get("steps") or []
24    tips = recipe.get("tips") or []
25    lines = [
26        f"Receta entrenada ({dataset_size} disponibles): {name}.",
27
28    ]
29    if readable_tags:
30        lines.append(f"Etiquetas clave: {readable_tags}.")
31    if ingredient_line:
32        lines.append(ingredient_line)
33    if steps:
34        lines.append("Pasos:")
35        for idx, step in enumerate(steps, 1):
36            lines.append(f"{idx}. {step}")
37    if tips:
38        lines.append("Consejos: " + " ".join(tips))
39    lines.append("¿Quieres otra combinación? Pídemle más ideas por dieta
        , tiempo o ingrediente.")
40    return "\n".join(lines)
41
42
43 def format_recipe_brainstorm(recipes: List[Dict[str, Any]],
    dataset_size: int, filters: Optional[Set[str]] = None) -> str:
44     if not recipes:
45         return (
46             "No tengo recetas que coincidan con ese filtro todavía.
                Dime un ingrediente principal, una dieta o un tiempo y lo intento de
                nuevo."
```



```
47     )
48     focus_note = ""
49     if filters:
50         readable = ", ".join(sorted(filter_tag.replace("_", " ") for
filter_tag in filters if filter_tag))
51         if readable:
52             focus_note = f" con enfoque en {readable}"
53     lines = [
54         f"Ideas frescas{focus_note}: tengo {dataset_size} recetas
entrenadas y estas pueden inspirarte:",
55     ]
56     for idx, recipe in enumerate(recipes, 1):
57         time_minutes = recipe.get("time_minutes", "?")
58         diet = recipe.get("diet", "dieta")
59         cuisine = recipe.get("cuisine", "fusion")
60         lines.append(f"{idx}. {recipe.get('name', 'Receta')} ({
time_minutes} min, {diet}, {cuisine}).")
61         tip = (recipe.get("tips") or ["Ajusta las especias a tu gusto."
]) [0]
62         lines.append(f"    Tip: {tip}")
63     lines.append("Pídemle otra ronda si quieres más opciones o cambia el
filtro (ej. vegana, cena ligera, sin carne).")
64     return "\n".join(lines)
```

Listing 40: Generación de Texto para Recetas

2.5.42. Motor de Recomendación de Hardware

Este módulo sigue un patrón similar al de recetas pero introduce la variable `mode` (Desktop vs Laptop) como un filtro duro.

`_infer_hardware_filters` traduce términos técnicos ("4k", render", ".autocad") a etiquetas internas. `match_hardware_alias` busca nombres de configuraciones específicas (ej: "Build Gama Alta 2024") en la entrada del usuario. Aquí se aplica el filtro de modo: si el usuario pidió una "laptop", se descartan las coincidencias de "desktop" encontradas por alias.

`lookup_hardware_builds` implementa el algoritmo de puntuación:

1. **Filtros Duros:** Si se especifica `mode` (desktop/laptop), se ignoran todas las entradas que no coincidan, sin importar su puntuación.
2. **Pesos:**
 - Coincidencia de Filtro (ej: "gaming"): **+4 puntos**.
 - Coincidencia de Token (ej: ryzen"): **+1 punto**.
3. Esto asegura que si el usuario pide "PC para streaming", el sistema priorice configuraciones etiquetadas como "streaming" sobre aquellas que solo mencionan componentes aleatorios.

```
1 def _infer_hardware_filters(normalized_text: str) -> Set[str]:
2     filters: Set[str] = set()
3     for needle, mapped in HARDWARE_FILTER_HINTS.items():
4         if needle and needle in normalized_text:
5             filters.update(mapped)
```

```
6     return filters
7
8
9 def _match_hardware_aliases(normalized_text: str, mode: Optional[str])
-> List[str]:
10     hits: List[str] = []
11     for alias, build_id in CURATED_HARDWARE_ALIAS_INDEX.items():
12         if alias and alias in normalized_text:
13             entry = CURATED_HARDWARE_BUILDS.get(build_id)
14             if not entry:
15                 continue
16             entry_type = entry.get("type")
17             if mode and entry_type != mode:
18                 continue
19             hits.append(build_id)
20     return hits
21
22
23 def lookup_hardware_builds(
24     query: str,
25     filters: Optional[Set[str]] = None,
26     mode: Optional[str] = None,
27     limit: int = 3,
28 ) -> List[Dict[str, Any]]:
29     if not CURATED_HARDWARE_BUILDS or limit <= 0:
30         return []
31     normalized = _normalize_command_text(query)
32     filter_set = {tag for tag in (filters or set()) if tag}
33     matches: List[Dict[str, Any]] = []
34     seen: Set[str] = set()
35
36     alias_hits = _match_hardware_aliases(normalized, mode)
37     for build_id in alias_hits:
38         entry = CURATED_HARDWARE_BUILDS.get(build_id)
39         if entry and build_id not in seen:
40             matches.append(entry)
41             seen.add(build_id)
42             if len(matches) >= limit:
43                 return matches
44
45     candidate_scores: Counter[str] = Counter()
46
47     def _add_candidates(tag: str, weight: int) -> None:
48         build_ids = CURATED_HARDWARE_TAG_INDEX.get(tag, set())
49         for build_id in build_ids:
50             if build_id in seen:
51                 continue
52             entry = CURATED_HARDWARE_BUILDS.get(build_id)
53             if not entry:
54                 continue
55             if mode and entry.get("type") != mode:
56                 continue
57             candidate_scores[build_id] += weight
58
59     for filter_tag in filter_set:
60         _add_candidates(filter_tag, 4)
61
62     tokens = [token for token in _extract_recipe_tokens(normalized) if
```

```
token not in HARDWARE_TOKEN_STOPWORDS]
63 for token in tokens[:30]:
64     _add_candidates(token, 1)
65
66 for build_id, _score in candidate_scores.most_common(limit * 2):
67     if build_id in seen:
68         continue
69     entry = CURATED_HARDWARE_BUILDS.get(build_id)
70     if entry:
71         matches.append(entry)
72         seen.add(build_id)
73     if len(matches) >= limit:
74         break
75 return matches
76
77
78 def pick_random_hardware_builds(
79     count: int,
80     mode: Optional[str] = None,
81     filters: Optional[Set[str]] = None,
82     rng: Optional[random.Random] = None,
83 ) -> List[Dict[str, Any]]:
84     if not CURATED_HARDWARE_BUILDS or count <= 0:
85         return []
86     rng = rng or random
87     candidate_ids: Set[str] = set()
88     filter_set = {tag for tag in (filters or set()) if tag}
89     for filter_tag in filter_set:
90         candidate_ids.update(CURATED_HARDWARE_TAG_INDEX.get(filter_tag,
91             set()))
92     if not candidate_ids:
93         candidate_ids = set(CURATED_HARDWARE_BUILDS.keys())
94     if mode:
95         candidate_ids = {
96             build_id
97             for build_id in candidate_ids
98             if CURATED_HARDWARE_BUILDS.get(build_id, {}).get("type") ==
99             mode
100         }
101     pool = list(candidate_ids)
102     if not pool:
103         pool = [
104             build_id
105             for build_id, entry in CURATED_HARDWARE_BUILDS.items()
106             if not mode or entry.get("type") == mode
107         ]
108     if not pool:
109         return []
110     sample_size = min(len(pool), max(1, count))
111     chosen = rng.sample(pool, sample_size)
112     return [CURATED_HARDWARE_BUILDS[build_id] for build_id in chosen if
113         build_id in CURATED_HARDWARE_BUILDS]
```

Listing 41: Búsqueda y Filtrado de Hardware

2.5.43. Formateo Dinámico de Detalles de Hardware

Aquí se encuentra una de las lógicas más sofisticadas de generación de texto sin LLM. La función `format_hardware_component_detail` no solo muestra especificaciones; genera una explicación contextual de *por qué* ese componente es bueno para esa configuración.

Funciona detectando el "slot" componente de interés (CPU, GPU, RAM, etc.). Luego, utiliza plantillas de texto que insertan referencias cruzadas a otros componentes de la misma build. Por ejemplo, si el usuario pregunta por la **GPU**, la función explica que "se elige para cumplir el objetivo de [resolución] sin cuellos de botella frente a [CPU]". Si pregunta por la **Fuente de Poder (PSU)**, explica que "tiene margen para el consumo combinado de [GPU]". Esta interconexión de datos crea la ilusión de un razonamiento profundo sobre la arquitectura del computador, cuando en realidad es una interpolación inteligente de campos de la base de datos estructurada.

```
1 def format_hardware_build_detail(build: Dict[str, Any], dataset_size:
2     int) -> str:
3     title = build.get("title", "Configuración recomendada")
4     release_year = build.get("release_window", "N/A")
5     use_cases = ", ".join(build.get("use_cases", [])) or "uso general"
6     budget = build.get("budget_tier", "personalizado")
7     system = build.get("recommended_os", "Windows 11 Pro")
8     header = f"Configuración entrenada ({dataset_size} combinaciones) -
9     {title} ({release_year})."
10    lines = [header, f"Perfil: {use_cases} | Presupuesto: {budget}."]
11    if build.get("type") == "desktop":
12        resolution = build.get("resolution", "multi monitor")
13        lines.append(f"Objetivo: {resolution} | PSU: {build.get('psu',
14        'N/A')}] | Enfriamiento: {build.get('cooling', 'estándar')}.")
15    else:
16        default_display = '16"'
17        lines.append(
18            f"Pantalla: {build.get('display', default_display)} | Bateria: {build.get('battery', '80 Wh')} | Peso: {build.get('weight', '2
19        kg')}]")
20    components = build.get("components") or []
21    if components:
22        lines.append("Componentes sugeridos:")
23        for idx, component in enumerate(components, 1):
24            lines.append(f"{idx}. {component}")
25    summary = build.get("summary")
26    if summary:
27        lines.append(summary)
28    lines.append(f"Sistema recomendado: {system}.")
29    lines.append("Puedo ajustar la lista si cambias resolución,
30    presupuesto o prefieres otra marca.")
31    return "\n".join(lines)
32
33 def format_hardware_brainstorm(
34     builds: List[Dict[str, Any]],
35     dataset_size: int,
36     mode: Optional[str] = None,
37     filters: Optional[Set[str]] = None,
38 ) -> str:
39     if not builds:
40         return "No encontré coincidencias directas; dame presupuesto,
```

```
resolución o marca preferida y recalcuulo."
38 focus = "pc" if mode == "desktop" else "laptop" if mode == "laptop"
    else "equipo"
39 filter_note = ""
40 if filters:
41     readable = ", ".join(sorted(filter_tag.replace("_", " ") for
filter_tag in filters if filter_tag))
42     if readable:
43         filter_note = f" con foco en {readable}"
44     lines = [
45         f"Otras ideas ({dataset_size} combinaciones registradas) para
tu {focus}{filter_note}:",
46     ]
47     for idx, build in enumerate(builds, 1):
48         lines.append(
49             f"{idx}. {build.get('title', 'Opción')} ({build.get('
release_window', 'N/A')}, {build.get('budget_tier', 'personalizado'
)}))."
50         )
51         lines.append(f"    Uso destacado: {' '.join(build.get('
use_cases', [])) or 'multipropósito'}." )
52         lines.append("Pide detalles de alguna opción para desglosar
componentes o comparar.")
53     return "\n".join(lines)
54
55
56 def _humanize_use_cases(use_cases: Optional[List[str]]) -> str:
57     if not use_cases:
58         return "uso general"
59     cleaned = [tag.replace("_", " ") for tag in use_cases if tag]
60     return ", ".join(cleaned) if cleaned else "uso general"
61
62
63 def _detect_hardware_component_focus(normalized_text: str) -> Optional[
str]:
64     if not normalized_text:
65         return None
66     if not any(trigger in normalized_text for trigger in
HARDWARE_DETAIL_TRIGGERS):
67         return None
68     for slot, keywords in HARDWARE_COMPONENT_KEYWORDS.items():
69         for keyword in keywords:
70             if keyword and keyword in normalized_text:
71                 return slot
72     return None
73
74
75 COMPONENT_DISPLAY_NAMES = {
76     "cpu": "Procesador",
77     "gpu": "GPU",
78     "ram": "RAM",
79     "storage": "Almacenamiento",
80     "motherboard": "Placa base",
81     "psu": "Fuente",
82     "cooling": "Refrigeración",
83     "case": "Gabinete",
84     "display": "Pantalla",
85     "battery": "Batería",
```

```
86     "weight": "Peso",
87 }
88
89
90 def _extract_component_value(build: Dict[str, Any], slot: str) -> str:
91     if slot == "storage":
92         storage_items = build.get("storage") or []
93         return ", ".join(storage_items)
94     if slot == "ram":
95         return build.get("ram", "")
96     if slot == "display":
97         return build.get("display", "")
98     if slot == "battery":
99         return build.get("battery", "")
100     if slot == "weight":
101         return build.get("weight", "")
102     return build.get(slot, "")
103
104
105 def format_hardware_component_detail(build: Dict[str, Any], slot: str,
106 dataset_size: int) -> str:
107     display_name = COMPONENT_DISPLAY_NAMES.get(slot, slot.upper())
108     component_value = _extract_component_value(build, slot)
109     title = build.get("title", "configuración recomendada")
110     if not component_value:
111         return (
112             f"No tengo fichas detalladas del {display_name.lower()}
113             para {title}. "
114             "Puedo proponer otra pieza si me dices qué priorizas (
115             potencia, silencio o presupuesto)."
116         )
117     use_case_desc = _humanize_use_cases(build.get("use_cases"))
118     resolution = build.get("resolution", "resoluciones altas")
119     budget_label = (build.get("budget_tier") or "personalizado").
120     replace("_", " ")
121     lines = [
122         f"Detalle entrenado ({dataset_size} combinaciones) - {
123         display_name} en {title}:",
124         f"- Especificacion propuesta: {component_value}.",
125     ]
126     if slot == "cpu":
127         gpu = build.get("gpu", "la GPU prevista")
128         lines.append(
129             f"- Mantiene el perfil {use_case_desc} sin cuellos de
130             botella frente a {gpu} para el objetivo {resolution}."
131         )
132         motherboard = build.get("motherboard")
133         if motherboard:
134             lines.append(
135                 f"- La placa {motherboard} ya está seleccionada para
136                 futuras actualizaciones o ajustes de voltaje."
137             )
138         cooling = build.get("cooling")
139         if cooling:
140             lines.append(f"- {cooling} sostiene temperaturas estables
141             incluso bajo cargas prolongadas.")
142     elif slot == "gpu":
143         psu = build.get("psu", "la fuente recomendada")
```

```
136         lines.append(  
137             f"- Se elige para cumplir el objetivo {resolution} en  
escenarios de {use_case_desc} sin exigir overclock extremo."  
138         )  
139         lines.append(f"- La fuente {psu} deja margen para picos y  
asegura eficiencia dentro del presupuesto {budget_label}.")  
140         elif slot == "ram":  
141             lines.append(  
142                 f"- La capacidad propuesta cubre multitarea y proyectos de  
{use_case_desc} sin swaps innecesarios."  
143             )  
144             motherboard = build.get("motherboard")  
145             if motherboard:  
146                 lines.append(f"- Validado con {motherboard} para aprovechar  
perfiles XMP/EXPO estables.")  
147             elif slot == "storage":  
148                 storage_items = build.get("storage") or []  
149                 if storage_items:  
150                     primary = storage_items[0]  
151                     lines.append(f"- {primary} se usa como unidad principal  
para SO y apps sensibles.")  
152                     if len(storage_items) > 1:  
153                         lines.append(  
154                             f"- El resto ({', '.join(storage_items[1:])})  
separa bibliotecas, capturas o proyectos pesados."  
155                         )  
156                     lines.append("- Mantiene un balance entre velocidad y costo sin  
salirse del presupuesto.")  
157             elif slot == "motherboard":  
158                 cpu = build.get("cpu", "el CPU propuesto")  
159                 lines.append(f"- Garantiza compatibilidad directa con {cpu} y  
con la RAM declarada.")  
160                 lines.append("- Ofrece suficientes fases y puertos para  
upgrades sin cambiar toda la plataforma.")  
161             elif slot == "psu":  
162                 gpu = build.get("gpu", "la GPU prevista")  
163                 lines.append(  
164                     f"- Tiene margen para el consumo combinado de {gpu} y el  
resto del build, evitando trabajar al 100% continuo."  
165                 )  
166                 lines.append("- Certificacion 80+ asegura eficiencia y menos  
calor en sesiones prolongadas.")  
167             elif slot == "cooling":  
168                 cpu = build.get("cpu", "el CPU propuesto")  
169                 lines.append(f"- Mantiene a {cpu} dentro de un delta térmico  
seguro incluso con boost sostenido.")  
170                 case = build.get("case")  
171                 if case:  
172                     lines.append(f"- El gabinete {case} tiene flujo pensado  
para aprovechar este sistema de refrigeracion.")  
173             elif slot == "case":  
174                 cooling = build.get("cooling", "la solución térmica sugerida")  
175                 lines.append(f"- Flujo y espacio pensados para {cooling},  
facilitando airflow limpio.")  
176                 lines.append("- Incluye gestión de cables y espacio para  
futuras GPU sin restricciones.")  
177             elif slot == "display":  
178                 battery = build.get("battery")
```



```
179         lines.append(f"- Se calibra con el perfil {use_case_desc} para  
ofrecer la claridad adecuada.")  
180         if battery:  
181             lines.append(f"- Coordinado con la bateria de {battery}  
para sostener sesiones unplugged.")  
182         elif slot == "battery":  
183             weight = build.get("weight")  
184             lines.append(f"- Otorga autonomia acorde al perfil {  
use_case_desc} sin inflar demasiado el peso.")  
185             if weight:  
186                 lines.append(f"- Junto al peso de {weight} sigue siendo  
viable para movilidad diaria.")  
187             elif slot == "weight":  
188                 lines.append(f"- Mantiene la movilidad para tareas de {  
use_case_desc} sin sacrificar rigidez del chasis.")  
189                 battery = build.get("battery")  
190                 if battery:  
191                     lines.append(f"- Se equilibra con la bateria ({battery})  
para no comprometer autonomía.")  
192             else:  
193                 summary = build.get("summary")  
194                 if summary:  
195                     lines.append(summary)  
196             lines.append("¿Quieres que compare otra pieza o prioricemos  
silencio, fps o presupuesto?")  
197             return "\n".join(lines)
```

Listing 42: Generación Contextual de Explicaciones de Hardware

Arquitectura Cognitiva y Control de Flujo

2.5.44. Gestión de Estado Emocional y Cerebro del Asistente

El agente mantiene un estado interno persistente para simular empatía. La clase `EmotionState` es un contenedor de datos.

- **current:** Almacena la etiqueta emocional actual (ej: `.alegre`, `.empático`).
- **intensity:** Un valor flotante (0.0 a 1.0) que modula qué tan fuerte es esa emoción.
- **register_user_message** y **register_assistant_message:** Analizan el texto entrante/saliente buscando palabras clave positivas o negativas (`_score_text`) para ajustar dinámicamente la intensidad. Si el usuario dice "gracias." o "genial", la intensidad sube; si dice "triste." o "mal", el agente cambia a modo `.empático`.

La clase `AssistantBrain` abstrae la interacción con el Modelo de Lenguaje (LLM).

- Mantiene el historial de conversación (`self.history`) limitado a `max_history` turnos para no exceder la ventana de contexto del modelo.
- **_system_prompt:** Construye dinámicamente las instrucciones para el LLM, inyectando el estado emocional actual (`self.emotion_state.describe()`). Esto obliga al modelo de lenguaje a "actuar" según el ánimo calculado (ej: Responde con tono alegre con intensidad 0.8).

- `generate_reply`: Es el método principal. Combina el prompt del sistema, recuerdos recuperados de la memoria a largo plazo (si los hay) y el historial reciente para generar la respuesta final.

```
1 @dataclass
2 class EmotionState:
3     current: str = "neutral"
4     intensity: float = 0.0
5     history: List[str] = field(default_factory=list)
6
7     def describe(self) -> str:
8         return f"{self.current} con intensidad {self.intensity:.2f}"
9
10    def register_user_message(self, text: str) -> None:
11        score = self._score_text(text)
12        self._update_state(score)
13        self.history.append(f"U:{text}")
14        self.history = self.history[-20:]
15
16    def register_assistant_message(self, text: str) -> None:
17        score = self._score_text(text)
18        self._update_state(score * 0.5)
19        self.history.append(f"A:{text}")
20        self.history = self.history[-20:]
21
22    def _score_text(self, text: str) -> float:
23        positive = {"gracias", "feliz", "bien", "genial", "increíble",
24        "alegre", "contento", "maravilloso", "perfecto"}
25        negative = {"triste", "mal", "horrible", "enojado", "enfadado",
26        "deprimido", "cansado", "frustrado", "molesto"}
27        text_lower = text.lower()
28        score = 0.0
29        for word in positive:
30            if word in text_lower:
31                score += 1.0
32        for word in negative:
33            if word in text_lower:
34                score -= 1.0
35        return score
36
37    def _update_state(self, score: float) -> None:
38        decay = 0.1
39        self.intensity = max(0.0, self.intensity * (1 - decay))
40        if score > 0.5:
41            self.current = "alegre"
42            self.intensity = min(1.0, self.intensity + score * 0.3)
43        elif score < -0.5:
44            self.current = "empático"
45            self.intensity = min(1.0, self.intensity + abs(score) *
46            0.3)
47        else:
48            if self.intensity < 0.2:
49                self.current = "neutral"
50
51    def boost(self, target: str = "alegre", minimum: float = 0.55,
52    extra: float = 0.3) -> None:
53        self.current = target
54        self.intensity = max(self.intensity, minimum)
```

```
51         self.intensity = min(1.0, self.intensity + extra)
52
53
54 class AssistantBrain:
55     def __init__(self, emotion_state: EmotionState, model: str =
LLM_MODEL) -> None:
56         self.emotion_state = emotion_state
57         self.model = model
58         self.history: List[Dict[str, str]] = []
59         self.max_history = 10
60
61     def _system_prompt(self) -> str:
62         return (
63             "Eres Miau, un asistente emocional multimodal."
64             " Razona internamente paso a paso antes de contestar, pero
entrega solo la respuesta final en voz natural."
65             " Revisa tus recuerdos relevantes antes de responder para
mantener coherencia y haz explícito cuando falte contexto."
66             " Adapta tu tono al estado emocional indicado, manteniendo
empatía y claridad."
67             f" Estado emocional actual: {self.emotion_state.current}
con intensidad {self.emotion_state.intensity:.2f}."
68             " Responde en español, con frases fluidas y sin usar
formato markdown, listas explícitas ni código."
69             " Cuando corresponda, ofrece consejos concretos y explica
brevemente tu razonamiento o supuestos."
70         )
71
72     def generate_reply(self, user_message: str, memory_context:
Optional[List[str]] = None) -> str:
73         system_message = {"role": "system", "content": self.
_system_prompt()}
74         messages = [system_message]
75         if memory_context:
76             formatted = "\n".join(f"- {item}" for item in
memory_context)
77             memory_prompt = {
78                 "role": "system",
79                 "content": (
80                     "Memorias relevantes de esta conversación:\n"
81                     f"{formatted}\n"
82                     "Úsalas solo si encajan con la pregunta actual y
evita contradicciones."
83                 ),
84             }
85             messages.append(memory_prompt)
86             messages.extend(limit_history(self.history, self.max_history))
87             messages.append({"role": "user", "content": user_message})
88             response = query_llm(messages, model=self.model)
89             self.history.append({"role": "user", "content": user_message})
90             self.history.append({"role": "assistant", "content": response})
91             self.history = limit_history(self.history, self.max_history)
92             self.emotion_state.register_assistant_message(response)
93         return response
```

Listing 43: Clases EmotionState y AssistantBrain

2.5.45. Clasificador de Intenciones (Router Semántico)

La clase `IntentPredictor` determina qué "herramienta" debe usar el agente para responder. Implementa un enfoque híbrido para latencia mínima:

1. **Heurística (Reglas):** Primero, escanea el texto buscando palabras clave de alta prioridad. Si encuentra "whatsapp", clasifica inmediatamente como `whatsapp_message`. Si encuentra reproducez "música", clasifica como `music_playback`. Esto cubre el 90% de los casos comunes en microsegundos.
2. **Clasificación LLM (Fallback):** Si ninguna regla coincide y `use_llm` es verdadero, consulta al modelo de lenguaje. Le envía el texto y una lista de etiquetas válidas (`INTENT_LABELS`), pidiéndole que clasifique la intención en formato JSON. Esto permite detectar intenciones sutiles que las reglas simples pierden.

```
1 INTENT_LABELS = {
2     "greeting", "web_search", "music_recommendation", "music_playback",
3     "video", "document", "suggestion", "opinion", "anime_suggestion",
4     "anime_watch", "route", "camera", "time", "date", "emotion",
5     "mode_switch", "whatsapp_message", "reminder", "general",
6 }
7
8 class IntentPredictor:
9     def __init__(self, use_llm: bool = True) -> None:
10         self.use_llm = use_llm
11
12     def predict(self, text: str, hints: Optional[Dict[str, bool]] =
13 None) -> str:
14         lowered = text.lower().strip()
15         if not lowered:
16             return "general"
17
18         hints = hints or {}
19         # Prioridad a pistas externas (ej: si se detecto patron de ruta
20 antes)
21         if hints.get("route"):
22             return "route"
23
24         # Reglas heurísticas rápidas
25         if any(keyword in lowered for keyword in GREETING_KEYWORDS):
26             return "greeting"
27         if any(keyword in lowered for keyword in REMINDER_KEYWORDS):
28             return "reminder"
29         # ... (Multiples if-checks para deteccion por palabras clave)
30         ...
31         if any(phrase in lowered for phrase in ["quiero enseñarte", "
32 abre la cam"]):
33             return "camera"
34
35         # Fallback a LLM si no hay coincidencia exacta
36         if not self.use_llm:
37             return "general"
38
39         system_prompt = (
40             "Eres un detector de intenciones... Responde con JSON... "
41             + ", ".join(sorted(INTENT_LABELS))
42         )
```

```
39     # ... (Llamada a query_llm y parseo de JSON) ...
40     return "general"
```

Listing 44: Predictor de Intenciones

2.5.46. Interfaz de Hardware de Voz

La clase `VoiceInterface` encapsula la complejidad de la librería `speech_recognition`.

- `__init__`: Configura umbrales de energía estáticos para detectar silencio vs habla.
- `_prepare_source`: Realiza una calibración de ruido ambiental. Esto es vital: el micrófono .escucha. el silencio durante unos segundos (`VOICE_FULL_CAL_DURATION`) para establecer qué nivel de decibelios se considera ruido de fondo”.
- `listen`: Es el bucle de escucha. Implementa reintentos (`max_attempts`). Utiliza `recognizer.listen` para capturar audio y lo envía a la API de Google Speech Recognition (`recognize_google`) para convertirlo a texto. Si hay errores de red o no se entiende el audio, ajusta dinámicamente el umbral de energía (`_soften_threshold`) para ser más sensible en el siguiente intento.

```
1 def microphone_available() -> bool:
2     try:
3         return bool(sr.Microphone.list_microphone_names())
4     except OSError:
5         return False
6
7 class VoiceInterface:
8     def __init__(self, device_index: Optional[int] = None) -> None:
9         self.recognizer = sr.Recognizer()
10        self.recognizer.dynamic_energy_threshold = False
11        self.recognizer.energy_threshold = VOICE_ENERGY_THRESHOLD
12        # ... (Configuracion de tiempos de pausa y silencio)
13        self.device_index = device_index
14
15    def listen(self, timeout: Optional[int] = None, ...) -> Optional[
16    str]:
17        # ... (Logica de reintentos y calibracion)
18        try:
19            with sr.Microphone(device_index=self.device_index) as
20            source:
21                self._prepare_source(source, quick=quick_calibration)
22                audio = self.recognizer.listen(source, timeout=...,
23                phrase_time_limit=...)
24                text = self.recognizer.recognize_google(audio, language="es
25                -MX")
26                return text
27            except sr.WaitTimeoutError:
28                # Manejo de silencio
29            except sr.UnknownValueError:
30                # Manejo de audio ininteligible
31            # ...
32            return None
```

Listing 45: Abstracción de Micrófono y Reconocimiento

2.5.47. Núcleo del Asistente (AssistantCore)

Esta es la clase controladora principal (Controller.^{en} MVC). Coordina todos los subsistemas.

- `__init__`: Inicializa el Cerebro, el Estado Emocional, la Interfaz de Voz y el sistema de Memoria. Arranca el hilo de salida de voz (`_speech_thread`) que consume una cola de mensajes para no bloquear la ejecución principal mientras el asistente habla.
- `run`: El bucle principal para el modo CLI (Consola). Lee entrada (teclado o voz), verifica condiciones de salida y llama a `handle_text`.
- `handle_text`: Punto de entrada para mensajes. Puede recibir contexto visual (descripción de imágenes) o contexto de respuesta. Llama a `_process_input`.
- `_process_input`: Ejecuta el pipeline cognitivo completo:
 1. Registra el mensaje en el estado emocional.
 2. Verifica comandos de sistema (salir, cambiar modo).
 3. Usa `IntentPredictor` para clasificar la intención.
 4. Despacha la ejecución a un manejador específico (`handle_weather`, `handle_whatsapp`, etc.) o al cerebro general (LLM) si no hay herramienta específica.
 5. Registra la interacción en la memoria a largo plazo.

Los métodos `handle_*` (ej: `handle_whatsapp_request`) contienen la lógica específica para invocar las funciones de automatización documentadas en partes anteriores y formatear la respuesta final para el usuario.

```
1 class AssistantCore:
2     def __init__(self, voice_enabled: bool = True) -> None:
3         self.emotion_state = EmotionState()
4         self.brain = AssistantBrain(self.emotion_state)
5         self.voice_interface = VoiceInterface() if voice_enabled else
None
6         self.memory = MemoryManager()
7         self.intent_predictor = IntentPredictor()
8         # ... (Inicializacion de hilos de voz y colas)
9
10    def run(self) -> None:
11        print("Escribe 'salir' cuando quieras terminar.")
12        while self.keep_running:
13            try:
14                if self.mode == "voz" and self.voice_enabled:
15                    user_text = self.voice_interface.listen()
16                else:
17                    user_text = input("Tú: ").strip()
18            except KeyboardInterrupt:
19                break
20
21            if user_text:
22                self.handle_text(user_text)
23        self.shutdown()
24
25    def _process_input(self, user_text: str) -> None:
26        # 1. Registro emocional
```

```
27     self.emotion_state.register_user_message(user_text)
28
29     # 2. Prediccion de intencion
30     intent = self.intent_predictor.predict(user_text)
31
32     # 3. Despacho (Routing)
33     handlers = {
34         "whatsapp_message": self._handle_whatsapp_request,
35         "web_search": self._handle_web_search,
36         # ... (Mapeo de todos los intents a sus funciones)
37     }
38
39     if intent in handlers:
40         handlers[intent](user_text)
41     else:
42         # Fallback al cerebro conversacional (LLM generico)
43         response = self.brain.generate_reply(user_text)
44         self._emit(response)
45
46     # 4. Memorizacion
47     self._remember_interaction(user_text, intent)
```

Listing 46: Clase Controladora Principal AssistantCore

2.5.48. Interfaz Gráfica (Tkinter) y Punto de Entrada

La clase ChatGUI envuelve al AssistantCore en una ventana moderna.

- `__init__`: Configura la ventana raíz, estilos (colores oscuros, fuentes), y divide la pantalla en paneles: Chat (Scrollable), Entrada de Texto, Panel de Respuesta/Adjuntos y Barra de Estado.
- `_append_message`: Instancia un ChatBubble (documentado en Parte 1) y lo añade al canvas de chat con animación.
- `_process_in_background`: Ejecuta la lógica del asistente en un hilo secundario (`threading.Thread`). Esto es crucial para que la interfaz gráfica no se congele mientras el asistente piensa, consulta internet o procesa una imagen.
- `_start_voice_session`: Gestiona el bucle de micrófono en la GUI, actualizando la barra de estado (`.Es escuchando...`) y enviando el texto transcrito al chat automáticamente.

Finalmente, el bloque `if __name__ == "__main__"` es el punto de entrada. Verifica argumentos de línea de comandos (`--cli`, `--gui`). Si `GUI_AVAILABLE` es cierto y no se forzó CLI, inicia la interfaz gráfica. De lo contrario, inicia el bucle de consola `run_cli_assistant`.

```
1 class ChatGUI:
2     def __init__(self) -> None:
3         self.root = tk.Tk()
4         self.root.geometry("780x620")
5         # ... (Configuracion de estilos y widgets: Canvas, Entry,
6         Buttons)
7
7     # Instancia del nucleo logico dentro de la GUI
8     self.assistant = AssistantCore(voice_enabled=True)
```



```
9         # Redireccion de la salida del asistente para que pinte
        burbujas en vez de prints
10         self.assistant.set_output_handler(self._handle_assistant_output
        )
11
12     def _send_message(self) -> None:
13         text = self.input_var.get().strip()
14         if not text: return
15
16         # Muestra el mensaje del usuario inmediatamente
17         self._append_message("Tú", text)
18         self.input_var.set("")
19
20         # Lanza el procesamiento en hilo aparte para no bloquear la UI
21         threading.Thread(target=self._process_in_background, args=(text
        ,)).start()
22
23     def _process_in_background(self, text: str) -> None:
24         # Llama al nucleo del asistente. La respuesta llegara via
        callback (_handle_assistant_output)
25         self.assistant.handle_text(text)
26
27 if __name__ == "__main__":
28     if should_launch_gui():
29         try:
30             gui = ChatGUI()
31             gui.run()
32         except RuntimeError:
33             run_cli_assistant()
34     else:
35         run_cli_assistant()
```

Listing 47: Interfaz Gráfica y Main